

Introduction to Computer Hardware & Programming Languages

Understanding the machines we use and the languages we use to command them

A FOUNDATIONS OF COMPUTING PRIMER

AGENDA

What We'll Cover



Computer Hardware Basics

CPU, memory, storage, and the components that power every machine



Input & Output Devices

How computers receive information and communicate results



How Hardware Works Together

The journey of data through a computer system



Programming Languages

Machine code, assembly, and high-level languages explained



Compiled vs Interpreted

Two ways source code becomes running programs



The Full Stack

Connecting hardware and software into one working system

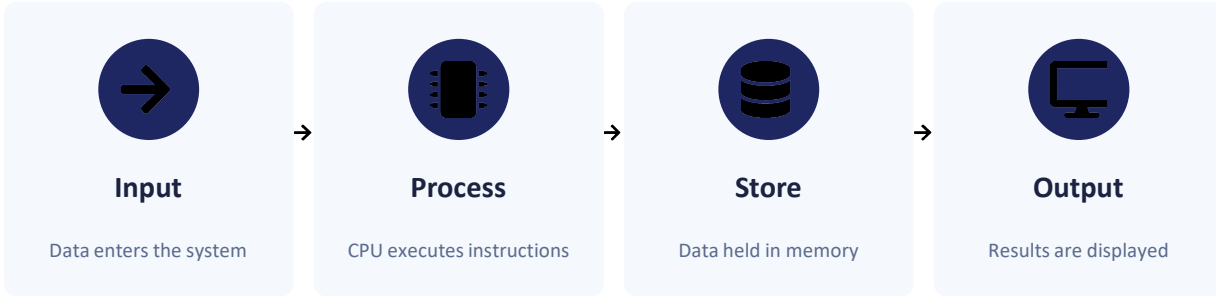


A machine that takes INPUT, processes it, and produces OUTPUT

FUNDAMENTALS

What Is a Computer?

A computer is an electronic device that accepts data (input), processes it according to a set of instructions, stores it, and produces results (output). Every computer, from a smartphone to a supercomputer, follows this same basic cycle.



Every computer — a phone, a laptop, a data-center server — repeats this input-process-output cycle billions of times per second.

FUNDAMENTALS

Hardware vs. Software

Two halves of every computer system — the physical parts, and the instructions that bring them to life.



Hardware

The physical, tangible components you can touch

- Processor (CPU) — executes instructions
- Memory (RAM) — short-term working space
- Storage (SSD/HDD) — long-term data
- Input/output devices — keyboard, screen



Software

The instructions that tell hardware what to do

- Operating system — manages resources
- Applications — browsers, editors, games
- Programming languages — build the code
- Drivers — let software reach hardware

HARDWARE · PROCESSING

The CPU — The Brain of the Computer

The Central Processing Unit fetches, decodes, and executes instructions — every calculation, decision, and action in a program passes through it.

Control Unit (CU)

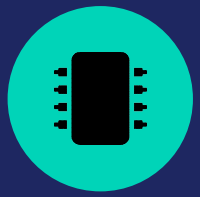
Directs the flow of data and instructions between components

Arithmetic Logic Unit (ALU)

Performs math and logical comparisons

Registers

Ultra-fast internal storage for data in active use



Clock Speed

3–5 GHz

billions of cycles executed every second on a modern processor

Cores

4–24

Threads

8–32

Independent tasks a modern CPU can juggle in parallel across its cores

HARDWARE · MEMORY

Memory Hierarchy

Not all memory is equal — a trade-off between speed, cost, and capacity shapes every computer's design.



RAM is volatile

Its contents disappear the instant power is lost — that's why unsaved work vanishes on a crash.

Storage is persistent

SSDs and HDDs keep data safely even when the computer is switched off.

Cache bridges the gap

A small pool of ultra-fast memory keeps the CPU fed while it waits on slower RAM.

HARDWARE · STORAGE

Storage Devices: HDD vs. SSD

Both store data permanently — they differ in how they read and write it.

Hard Disk Drive (HDD)

MECHANISM

Spinning magnetic platters + moving read/write head

SPEED

Slower — mechanical parts limit access time

DURABILITY

More vulnerable to shock and wear

COST

Cheaper per gigabyte

BEST FOR

Bulk, low-cost archival storage

Solid State Drive (SSD)

MECHANISM

Flash memory chips — no moving parts

SPEED

Much faster — near-instant data access

DURABILITY

More resistant to shock and vibration

COST

More expensive per gigabyte

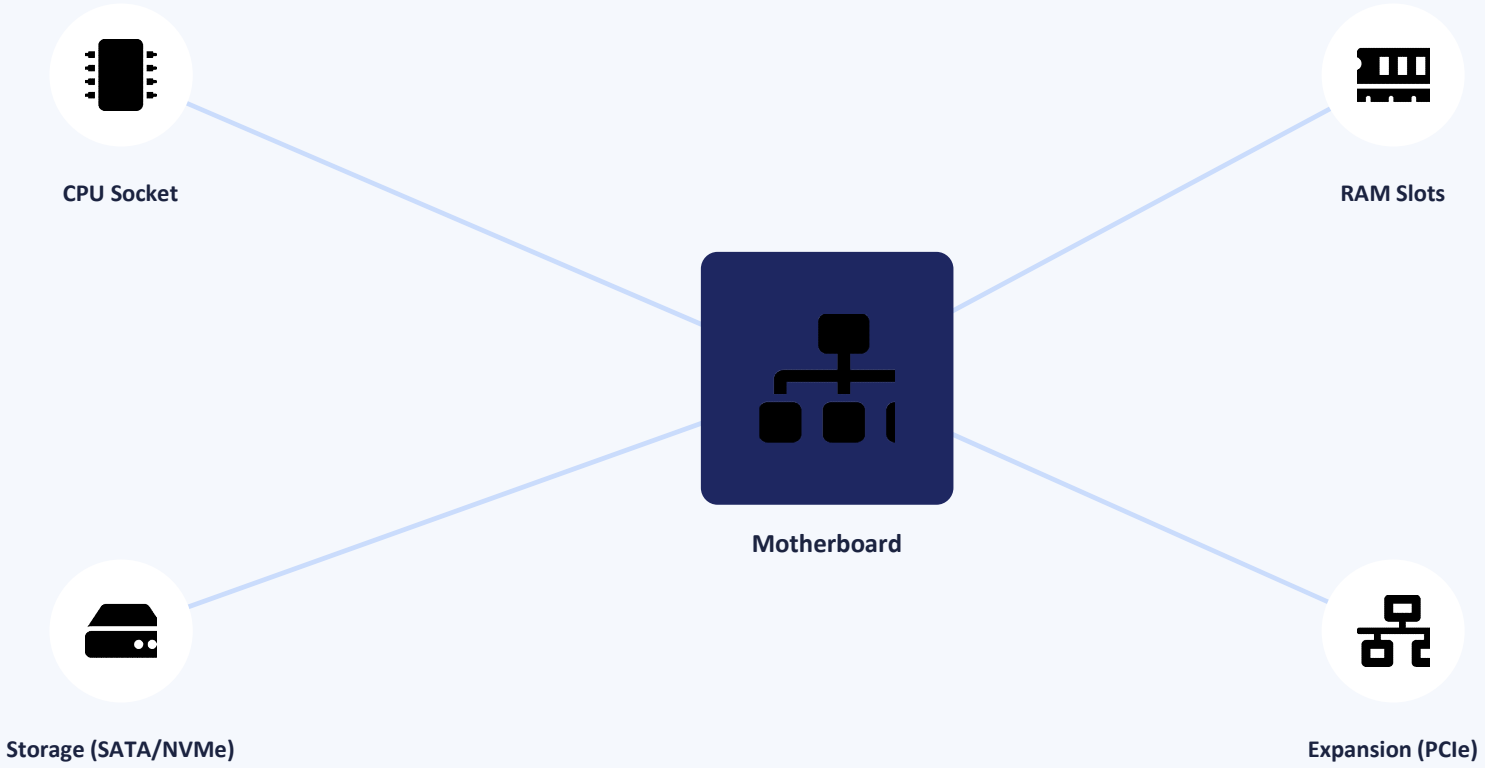
BEST FOR

Operating systems and everyday performance

HARDWARE · CONNECTIVITY

The Motherboard & System Buses

The motherboard is the circuit board that connects every component; buses are the electrical pathways that let them talk to each other.



Data and Address buses carry information; the Control bus carries the timing and command signals that keep every transfer in sync.

HARDWARE · I/O

Input Devices

Hardware that lets a person or another system send data and commands into the computer.



Keyboard

Text and command entry



Mouse / Trackpad

Pointing and selection



Touchscreen

Direct on-screen input



Microphone

Audio and voice input



Scanner

Digitizes printed material



Webcam

Captures video input

HARDWARE · I/O

Output Devices

Hardware that presents the results of processing back to the user.



Monitor

Visual display of images and text



Printer

Produces physical, printed copies



Speakers

Converts signals into sound



Headphones

Private audio output



Projector

Large-scale shared display



Indicator Lights

Simple status and alert signals

HARDWARE · CONNECTIVITY

Ports & Peripherals

Ports are the physical sockets that let external peripherals connect to, and communicate with, the computer.



USB — Universal — connects drives, keyboards, mice, and more



HDMI / DisplayPort — Carries video and audio to monitors and projectors



Ethernet — Wired connection to a local network or the internet



Audio Jack — Analog connection for headphones and microphones

Peripheral

Any external device that connects to the computer to add functionality — without being a core internal component.

- External hard drives
- Printers & scanners
- Webcams & microphones
- Game controllers

HARDWARE · SYSTEMS

How It All Works Together

Follow a single action — pressing a key — as it moves through the entire hardware system.

1



Input

A key press is sent as an electrical signal

→

2



CPU

The processor interprets and acts on the signal

→

3



RAM

Active data is held for immediate use

→

4



Storage

Relevant files are read from or written to disk

→

5



Output

The result appears on screen

This entire round trip — from keypress to pixel on screen — happens in a fraction of a millisecond.

PART TWO

Programming Languages

How we translate human ideas into instructions a machine can execute

SOFTWARE · CONCEPTS

Why We Need Programming Languages

A CPU only understands electrical signals — 1s and 0s. Programming languages give humans a readable way to describe logic, which is then translated down to that binary level.



This translation is handled automatically by compilers and interpreters — tools we'll meet shortly.

SOFTWARE · CONCEPTS

Levels of Programming Languages

Languages sit on a spectrum from what the machine speaks natively to what humans find easiest to read.

LOWEST LEVEL

Machine Language

```
10110000
01100001
```

Raw binary the CPU executes directly. Fast, but unreadable to humans and specific to each processor type.

MID LEVEL

Assembly Language

```
MOV AL, 61h
ADD AL, BL
```

Human-readable mnemonics that map almost one-to-one to machine instructions. Still hardware-specific.

HIGHEST LEVEL

High-Level Language

```
total = a + b
```

Reads close to English. Portable across machines and far faster to write, like Python, Java, or C++.

SOFTWARE · CONCEPTS

Compiled vs. Interpreted Languages

Two different strategies for turning source code into something a CPU can execute.

Compiled

Source code → translated entirely into machine code ahead of time, producing a standalone executable file.

```
Source Code (.c)
↓
Compiler
↓
Executable (.exe)
↓
Runs directly on CPU
```

Examples: C, C++, Rust, Go • Fast execution, platform-specific builds

Interpreted

Source code → read and executed line-by-line at run time by an interpreter program, with no separate build step.

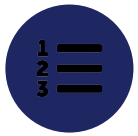
```
Source Code (.py)
↓
Interpreter (reads live)
↓
Executes line-by-line
↓
Immediate results
```

Examples: Python, JavaScript, Ruby • Easier to test, generally slower to run

SOFTWARE · CONCEPTS

Programming Paradigms

A paradigm is a style or approach to structuring code and thinking about problems.



Procedural

Step-by-step instructions grouped into reusable procedures or functions.

C, Pascal



Object-Oriented

Models real-world entities as objects that bundle data with behavior.

Java, C++, Python



Functional

Builds programs by composing pure functions and avoiding shared state.

Haskell, Elixir



Scripting

Lightweight code that automates tasks or glues other systems together.

JavaScript, Bash

SOFTWARE · LANDSCAPE

Popular Languages in Practice

A quick tour of widely used languages and where each one shines.



Python

Data science, automation, backend, AI



Java

Enterprise systems, Android apps



JavaScript

Web pages, browsers, interactive UI



C / C++

Operating systems, games, embedded devices



SQL

Querying and managing databases



Swift / Kotlin

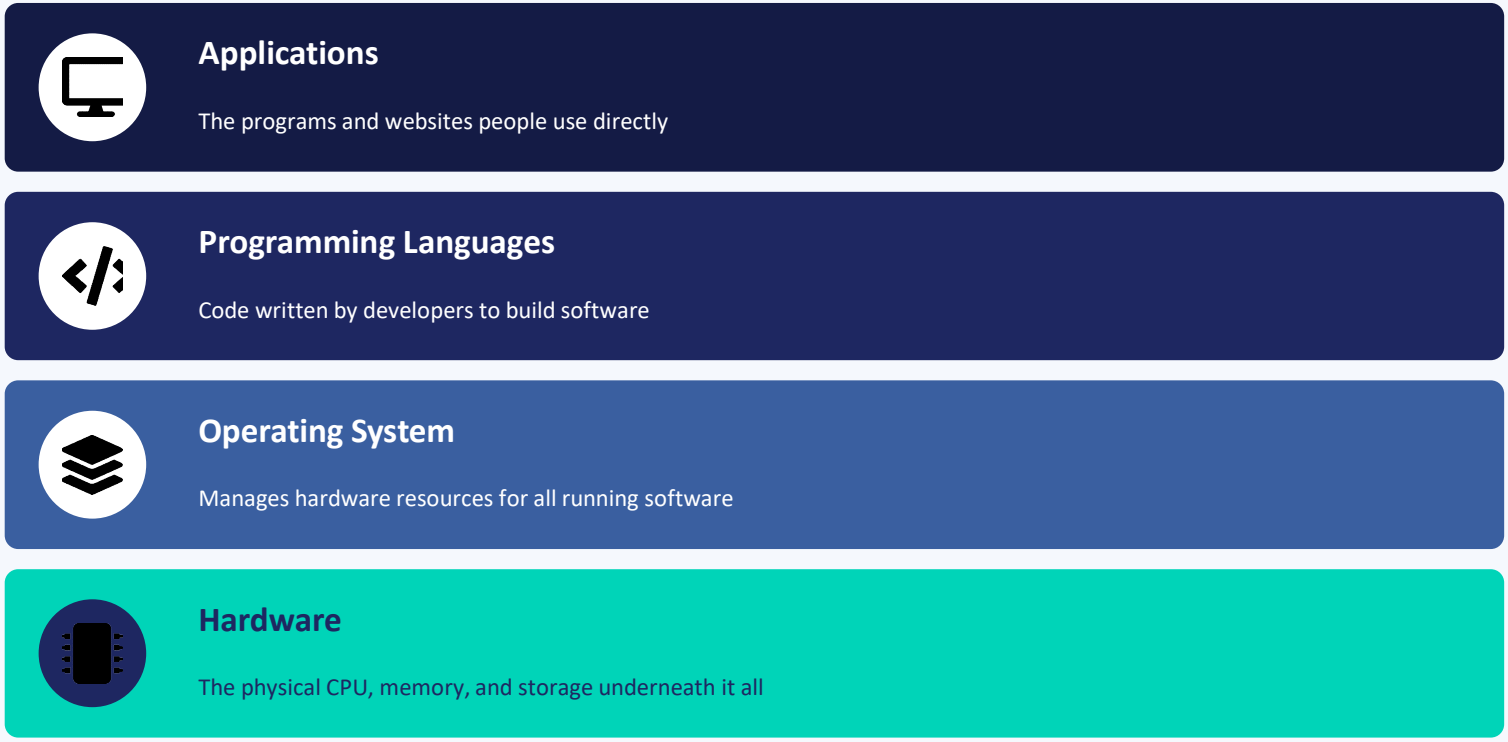
Native iOS and Android apps

BRINGING IT TOGETHER

Hardware & Software: The Full Stack

Every layer depends on the one beneath it — from silicon transistors to the app on your screen.

↑ USER-FACING



↓ PHYSICAL LAYER

RECAP

Key Takeaways

Hardware is the physical machine — CPU, memory, and storage — that carries out every instruction.

Programming languages translate human logic into the binary instructions hardware understands.

Compiled and interpreted languages offer different trade-offs between speed and flexibility.

Hardware and software form a stack — each layer depends on the one below it to function.

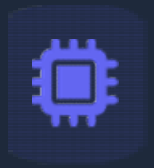
The Foundation of Computing

A Comprehensive Guide to Modern Computer Hardware and Programming Languages

Computer Hardware

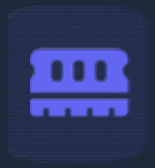
The physical backbone of computing systems, evolving from basic microprocessors to today's highly advanced AI-ready silicon architectures.

Core Hardware Components



Processing

The "brain" of the computer. Modern CPUs and GPUs handle billions of calculations per second, directing the flow of information across the system.



Memory (RAM)

The short-term workspace. Volatile memory holds the data your programs actively use, providing lightning-fast access for the processor.



Storage

The long-term vault. Non-volatile drives (like NVMe SSDs) retain the operating system, applications, and user files permanently.

The Central Processing Unit (CPU)

- **The Control Unit:** Directs operations within the processor, managing the flow of data across the system.
- **The ALU:** The Arithmetic Logic Unit performs all mathematical and logical operations.
- **Multi-core Architecture:** Modern CPUs possess multiple cores, allowing simultaneous execution of vast tasks.
- **The 2026 Shift: NPU Integration:** Most flagship CPUs now incorporate dedicated Neural Processing Units to handle on-device AI tasks, offloading work from standard cores.



Understanding Memory vs. Storage

Volatile Memory (RAM)

Random Access Memory is temporary. It clears when the PC loses power. By 2026, **DDR5** is the universal standard, offering significantly higher bandwidth and lower power consumption. It dictates how many tasks a computer can fluidly juggle at once without stuttering.

Non-Volatile Storage (SSD)

Solid State Drives hold data permanently. **PCIe Gen 5 NVMe SSDs** have become the benchmark, reducing application boot times to mere milliseconds. Unlike older mechanical hard drives (HDDs), SSDs have no moving parts, ensuring durability and silence.

The Speed of Modern Storage

10k+

Megabytes per second (MB/s)

The Era of Instant Access

Top-tier PCIe Gen 5 NVMe SSDs in 2026 exceed read/write speeds of 10,000 MB/s. To put this in perspective, older traditional Hard Disk Drives maxed out around 150 MB/s.

This leap eliminates rendering bottlenecks in professional workflows and ensures that complex operating systems load instantaneously.

The GPU Revolution

Beyond Rendering Graphics

Graphics Processing Units (GPUs) were originally designed strictly to render 3D visuals. Today, their architecture makes them the powerhouse of modern computing.

Unlike CPUs which have a few powerful cores, GPUs possess thousands of smaller cores designed for parallel processing. This makes them indispensable not just for lifelike gaming ray-tracing, but for training the massive AI models dominating the 2026 tech landscape.

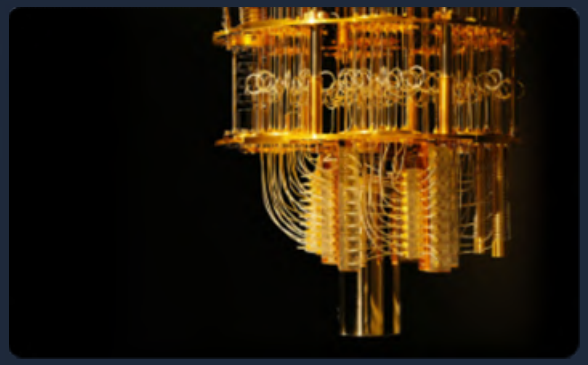


Hardware Trajectories in 2026



Edge AI Computing

Integrating AI processing power directly into localized devices to reduce latency and cloud dependency.



Quantum Progression

Quantum systems moving from theoretical labs to commercial cloud availability for complex problem-solving.



Advanced Thermals

Liquid cooling and advanced thermodynamic materials are now standard to manage the heat output of extreme processors.

Programming Languages

The crucial bridge between human logic and machine execution;
telling the hardware exactly how to perform.

The Hierarchy of Code



1. Machine Code

The lowest level. Pure binary (0s and 1s) that the CPU directly understands. Extremely fast to execute, but impossible for humans to effectively write or read.



2. Assembly Language

One step above binary. Uses short mnemonics (like ADD or MOV). It provides granular hardware control but is still highly complex and hardware-specific.



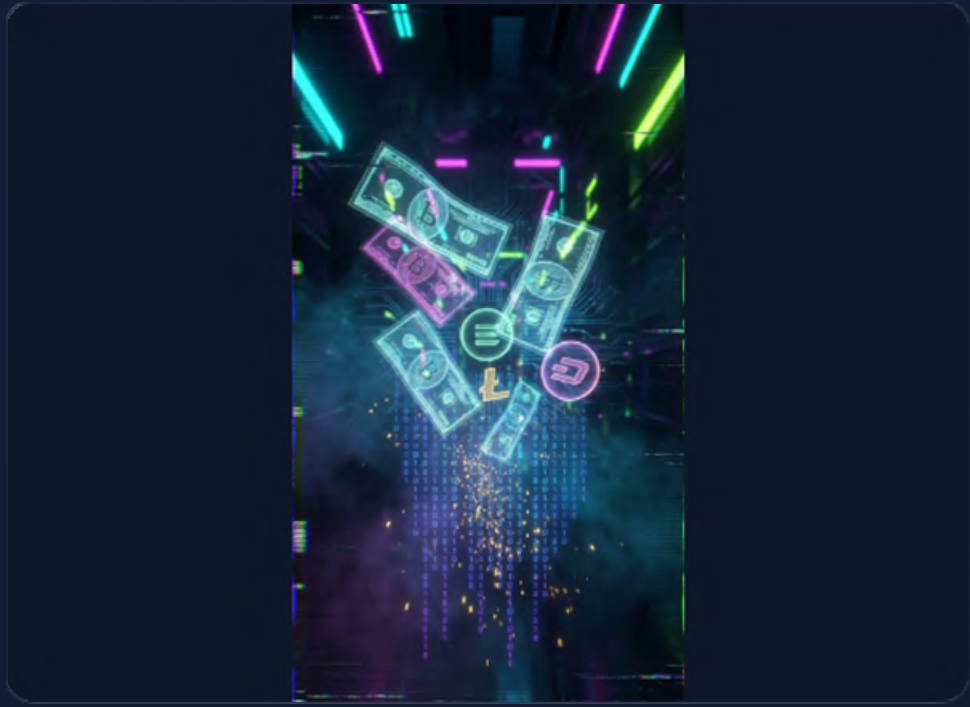
3. High-Level Languages

Languages like Python, Java, and C++. They use English-like syntax, allowing developers to focus on logic rather than hardware management.

Translation: Compilers & Interpreters

Computers cannot directly read high-level languages. The code must be translated into machine instructions.

- **Compilers:** Read the entire source code file and translate it into a standalone executable file before it ever runs (e.g., C++, Rust). This results in faster execution times.
- **Interpreters:** Translate the code line-by-line while the program is actively running (e.g., Python, JavaScript). This allows for easier debugging and cross-platform flexibility, though it can be slower.



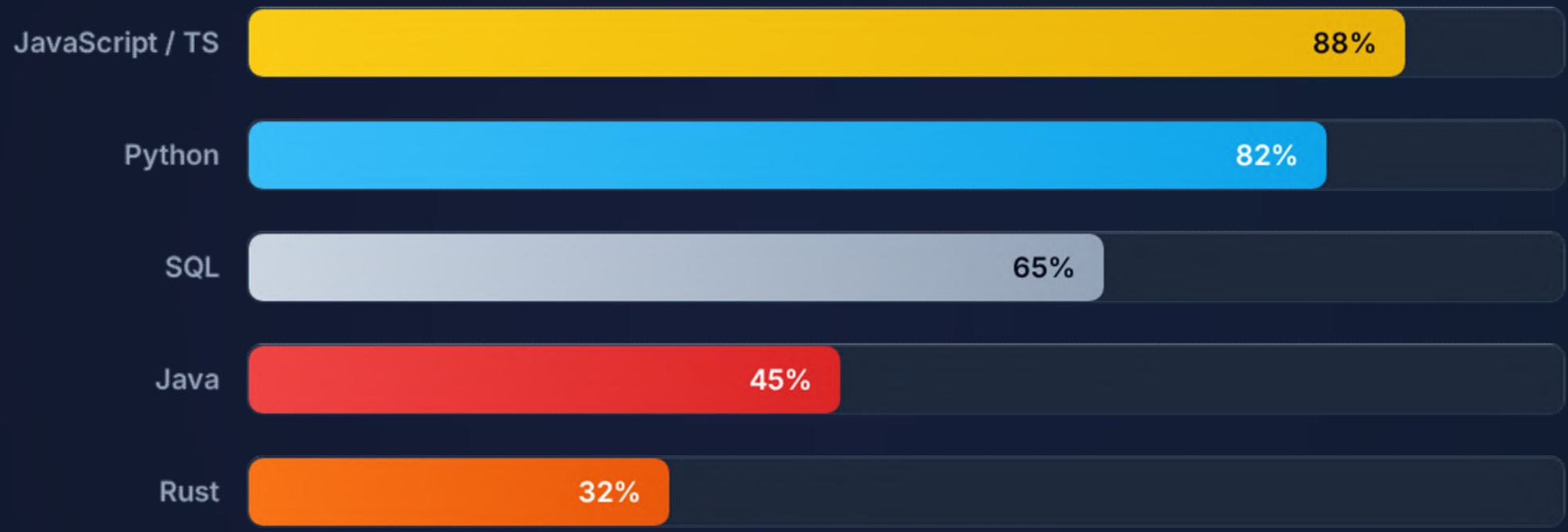
Core Programming Paradigms

- **Procedural Programming:** Writing a list of instructions for the computer to follow step-by-step. It relies on procedures or routines (functions) to manipulate data. Best for straightforward scripts and tasks (e.g., C).
- **Object-Oriented Programming (OOP):** Organizes code into reusable "objects" that contain both data and functions. This mimics real-world modeling and makes maintaining large codebases significantly easier (e.g., Java, C#).
- **Functional Programming:** Treats computation as the evaluation of mathematical functions and avoids changing state or mutable data. Highly predictable and excellent for concurrent operations (e.g., Haskell, Clojure).

Language Matrix 2026

Language	Primary Use Case	Key Benefit	Learning Curve
Python	AI, Data, Orchestration	Rapid prototyping, vast AI libraries	Beginner Friendly
JavaScript / TS	Web Development	Dynamic, interactive full-stack apps	Moderate
Rust	Systems & Infrastructure	Unmatched memory safety & speed	Steep
Java / Kotlin	Enterprise & Android	Stability, large-scale processing	Moderate

Developer Usage Trends (2026)



JavaScript maintains web dominance, while Python secures the runner-up position fueled by AI demands. Rust, while smaller in overall percentage, boasts the highest growth trajectory year-over-year.

The Enduring Web Foundation

JavaScript

The undisputed king of the browser. It breathes life into static HTML/CSS, providing the interactivity users expect today. With runtimes like Node.js, JavaScript is also a fully capable backend solution, allowing developers to create massive full-stack applications using a single language.

TypeScript (The Evolution)

Developed by Microsoft, TypeScript is a strict syntactical superset of JavaScript. It adds optional static typing, which catches errors during development rather than at runtime. In 2026, most enterprise-level front-end development is done strictly in TypeScript for maintainability.

Python: The AI Core

The AI Orchestrator

Despite not being optimized for raw execution speed, Python remains the absolute heavyweight champion of AI, Machine Learning, and Big Data in 2026.

Its strength lies in its extensive, mature ecosystem (libraries like PyTorch and TensorFlow) and its clean syntax. Developers use Python to prototype rapidly and act as an "orchestrator," dictating high-level logic while offloading the heavy math to C or Rust modules.



Rust: Safety & Performance



The Modern Standard for Infrastructure

Rust has broken into the top-tier of systems programming by solving a decade-old problem: memory leaks. Through its unique "ownership" model, Rust guarantees memory safety at compile time without relying on a garbage collector.

This allows it to run as fast as C/C++ while eliminating entire classes of security vulnerabilities. In 2026, it is the premier choice for embedded hardware, cloud infrastructure, and mission-critical applications.

Generational Tech Evolution

1970s: The Foundations

C language establishes the foundation for modern operating systems and hardware interaction.

1990s: The Web & Enterprise

Java solidifies enterprise architecture, while JavaScript is born to handle browser interactivity.

2010s: Safety & Scaling

TypeScript scales JavaScript, while Rust emerges to prioritize systems-level memory security.

2020s: The AI Integration

Python drives the AI boom; developers utilize AI co-pilots natively to accelerate software creation.

> *Code is the universal dialect of the future; it is the exact point where human creativity commands machine capability.*
>
> — The Modern Developer

Problem Solving & Programming in C

A Comprehensive Foundation in Computational Logic, Algorithms, and Software Design

Programming is not about typing code. It is about systematically breaking down problems and designing logical solutions.

— Course Philosophy

Course Overview

Introducing the principles of problem-solving and programming through the C language, from basic flowcharts to advanced memory management.

Curriculum Focus

1. Logical Thinking Skills

The course begins with the absolute basics of algorithms, flowcharts, and structured program design. Before writing any code, students will learn how to approach computational problems methodically and design step-by-step solutions.

2. C Language Implementation

Students will translate their logic into functional code using C. The curriculum progresses from core concepts (variables, control statements) to advanced techniques (pointers, dynamic memory, file handling), building a robust programming foundation.

Course Pre-requisites



Prior Coding Experience Needed

Accessible to All

This course has **no specific pre/co-requisites**. It is designed from the ground up for beginners who have never written a line of code.

We start with computational thinking and logical structuring before diving into the syntax of the C programming language, ensuring a smooth learning curve for all students.

Sustainable Development Goals (SDGs)



SDG 4: Quality Education

Providing inclusive and equitable quality education by equipping students with essential 21st-century digital problem-solving and coding skills.



SDG 8: Economic Growth

Promoting sustained, inclusive economic growth by preparing a technically proficient workforce capable of securing decent, high-value tech jobs.



SDG 9: Innovation

Building resilient infrastructure and fostering innovation by training students to develop efficient software solutions for real-world problems.

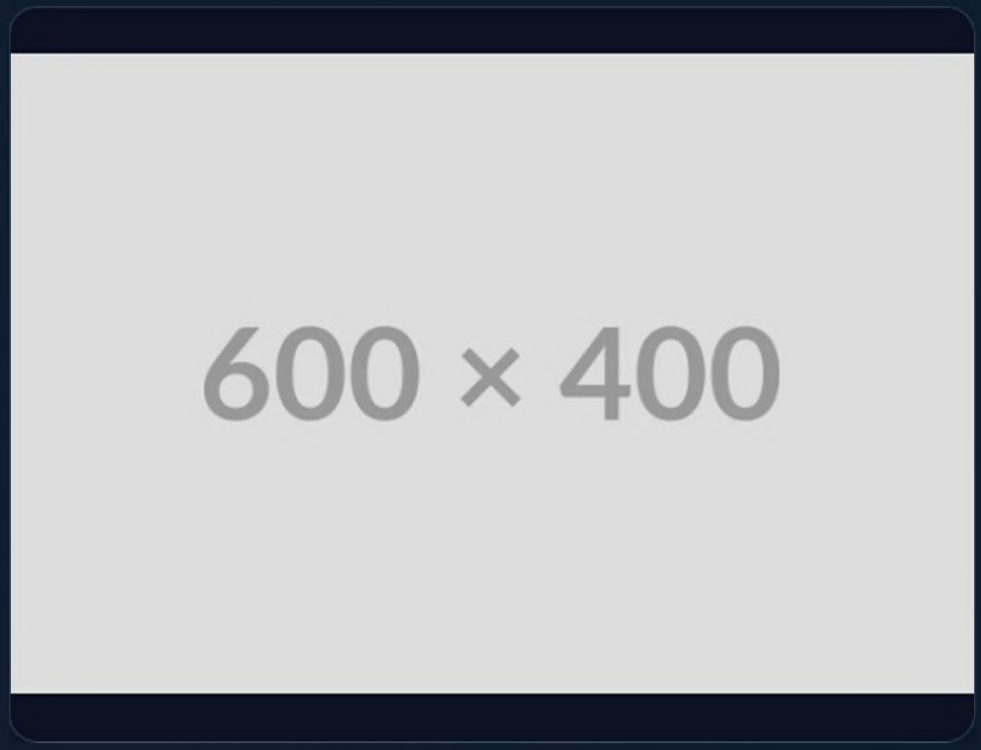
Core Modules

A structured journey from foundational logic to advanced data manipulation.

Basics of Problem Solving

Before writing code, we must understand how to structure a solution. This module focuses on the architecture of problem-solving.

- **Algorithms:** Defining a finite sequence of well-defined instructions to solve a specific class of problems.
- **Flowcharts:** Creating graphical representations of algorithmic processes to visualize decision paths.
- **Structured Design:** Breaking down complex problems into smaller, manageable, and logical sub-problems.



Core C Programming Concepts



Variables & Types

Understanding how computer memory stores different types of data, including integers, floating-point numbers, and characters.



Operators

Utilizing arithmetic, relational, and logical operators to manipulate data and evaluate conditions within the program.



Control Statements

Directing the flow of execution using decision-making structures (if, else, switch) and iteration loops (for, while, do-while).

Foundational Outcomes

Outcome A9501.1: Basic Constructs

By completing the introductory modules, students will be able to **use basic programming constructs and control statements** to design and code fundamental solutions for standard computational problems.

Outcome A9501.2: Sequential Data

Moving beyond single variables, students will learn to **develop programs using arrays and strings** to efficiently store, access, and manipulate sequential and grouped data in memory.

Handling Sequential Data

When dealing with large amounts of related data, creating individual variables is inefficient. We use sequential structures.

- **Arrays:** A collection of items stored at contiguous memory locations. Crucial for organizing lists of data of the same type.
- **Multi-dimensional Arrays:** Expanding arrays into matrices for complex grid-based data handling.
- **Strings:** One-dimensional arrays of characters terminated by a null character. Essential for text processing and manipulation.



Modular Programming

Functions & Recursion

As programs grow in complexity, writing all code in a single block becomes unmanageable. Modular programming is the solution.

Students will learn to write **Functions**—reusable blocks of code that perform specific tasks. This promotes code reusability, easier debugging, and logical separation.

We will also explore **Recursion**, a powerful technique where a function calls itself to solve smaller instances of the same problem.



Handling Complex Data

- **The Need for Complexity:** Arrays can only store data of the same type. Real-world entities (like a 'Student' with a name, ID, and GPA) require storing mixed data types together.
- **Structures (struct):** A user-defined data type in C that allows combining data items of different kinds. It allows developers to create complex, logical data models representing real-world objects.
- **Unions:** Similar to structures, but all members share the exact same memory location. This is an advanced technique used for strict memory conservation in embedded systems.
- **Outcome A9501.3:** By mastering these, students will **implement modular programming using functions, structures, and unions** to manage complex problems and data.

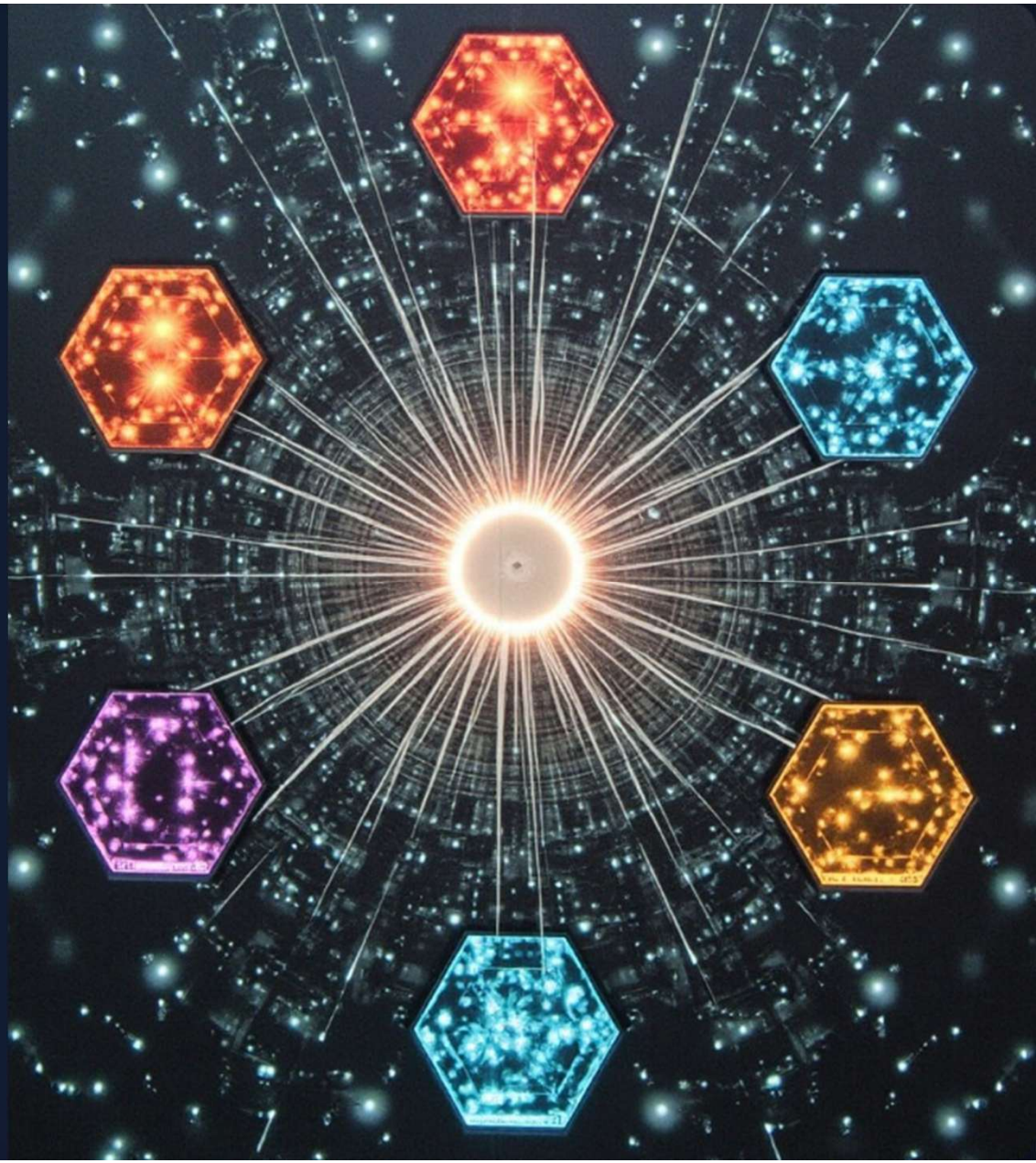
Advanced Memory

Pointers in C

Pointers are the defining feature of the C language, providing direct access to the hardware's memory architecture.

A pointer is a variable that stores the **memory address** of another variable. This allows for pass-by-reference function calls, dynamic memory allocation (malloc/free), and the creation of advanced data structures like linked lists.

Mastering pointers significantly enhances program efficiency and hardware-level control.



File Handling Techniques

Variables and arrays store data temporarily in RAM. To persist data after the program closes, we must interact with the file system.

- **Data Persistence:** Learning to open, create, read, write, and close files on the local disk using C standard library functions.
- **Text vs. Binary:** Understanding the difference between human-readable text files and raw binary data files for efficient storage.
- **Outcome A9501.4:** Students will successfully **make use of pointers and file handling** to effectively manage, persist, and process data.



The Capstone Outcomes

Outcome A9501.4: Memory & Storage

Through the advanced modules, students master the architecture of the machine, learning to **make use of pointers and file handling** to manipulate memory addresses and ensure long-term data persistence.

Outcome A9501.5: Algorithmic Efficiency

In the final phase, students evaluate performance to **choose appropriate searching and sorting techniques**, organizing data to optimize retrieval speed and computational efficiency.

Searching & Sorting Algorithms



Searching

Implementing algorithms like Linear Search and Binary Search to locate specific elements within vast arrays of data efficiently.



Sorting

Studying fundamental sorting techniques (Bubble, Selection, Insertion) to organize randomized data into logical, sequential order.



Performance

Analyzing the time and space complexity of these algorithms to understand which technique is optimal for different data scenarios.

Course Outcomes (CO) Summary

ID	Objective Description	Key Focus Area
A9501.1	Use basic constructs & control statements to design solutions.	Logic & Syntax
A9501.2	Develop programs using arrays and strings.	Sequential Data
A9501.3	Implement modular programming (functions, structures).	Architecture
A9501.4	Make use of pointers and file handling.	Memory & Storage
A9501.5	Choose appropriate searching and sorting techniques.	Efficiency & Speed

The Student Journey

1. Design

Learning to think computationally, utilizing algorithms and flowcharts before coding.

2. Implement

Writing functional C programs using variables, arrays, and control structures.

3. Structure

Scaling code through modular functions and custom complex data structures.

4. Evaluate

Optimizing programs with pointers, file I/O, and advanced sorting algorithms.

Translators: Compilers & Interpreters

Bridging the Gap Between Human Logic and Machine Execution

What is a Translator?

A **translator** is a crucial piece of software in computer science. Its primary role is to convert source code written in a high-level programming language into low-level machine code (binary).

Because computer processors can only understand operations in raw binary sequences of 0s and 1s, the translator acts as the essential intermediary that makes human-readable software executable by hardware.



The Language Gap

High-Level Languages

Designed for human convenience. They use abstract syntax, English-like keywords, and mathematical logic (e.g., Python, C++, Java). They are easy for humans to read, write, and maintain, but entirely incomprehensible to CPUs.

Machine Code

The native language of the processor. It consists entirely of binary sequences. While it executes blazingly fast, writing raw machine code is extremely error-prone, tedious, and impractical for modern complex software development.

The Three Main Types



Assemblers

Translates assembly language (low-level, symbolic instructions) directly into machine code. A 1-to-1 conversion process used close to the hardware.



Compilers

Scans the entire high-level source code file at once, generating a standalone executable machine code file before any execution begins.



Interpreters

Reads, translates, and executes the high-level source code line-by-line in real-time, without generating an intermediate executable file.

Compilers

Understanding the Mechanics of Batch Translation

Defining the Compiler

A compiler processes the **entire source code block simultaneously**. It acts like a book translator who reads an entire novel in English and writes a brand new book in French.

The output is an independent *object code* or *executable file* (like a .exe file). Once compiled, the original source code and the compiler itself are no longer needed to run the program.



Advantages of Compilers

- **Rapid Execution Speed:** Since the code is already translated into machine code, the hardware can execute it immediately without any translation delay during runtime.
- **Source Code Protection:** The generated executable is distributed instead of the source code, protecting intellectual property and making reverse-engineering difficult.
- **Comprehensive Error Checking:** The compiler scans the entire codebase, identifying syntax and structural errors all at once before the program is allowed to run.

Disadvantages of Compilers

- **Slower Translation Time:** Compiling a massive codebase can take minutes or even hours, slowing down the development and testing feedback loop.
- **Higher Memory Usage:** The process requires enough system memory to hold both the source code and the newly generated object code simultaneously during compilation.
- **Complex Debugging:** Because all errors are dumped at the end of the compilation phase, tracing a specific error back to its exact root cause can be challenging for developers.

Phases of Compilation

1. Lexical Analysis

Converts raw code characters into meaningful tokens (keywords, identifiers).

2. Syntax Analysis

Organizes tokens into a syntax tree, checking grammatical rules.

3. Code Optimization

Refines the intermediate representation to execute faster and use less memory.

4. Target Generation

Produces the final, hardware-specific machine code (binary).

Interpreters

The Power of On-the-Fly Execution

Defining the Interpreter

Unlike a compiler, an interpreter processes the source code **line-by-line or statement-by-statement**. It acts like a live language translator during a UN conference.
It reads one instruction, translates it, executes it immediately, and then moves to the next. No standalone executable file is ever created or stored on the disk.



Advantages of Interpreters



Easy Debugging

Execution halts immediately when an error is encountered. Developers know exactly which line caused the crash, making fixes significantly faster.



Platform Agnostic

Source code can run on any operating system without modification, as long as the target machine has the corresponding interpreter installed.



Lower Memory

Because it processes one instruction at a time and creates no intermediate object code, overall memory footprint during translation is highly efficient.

Disadvantages of Interpreters



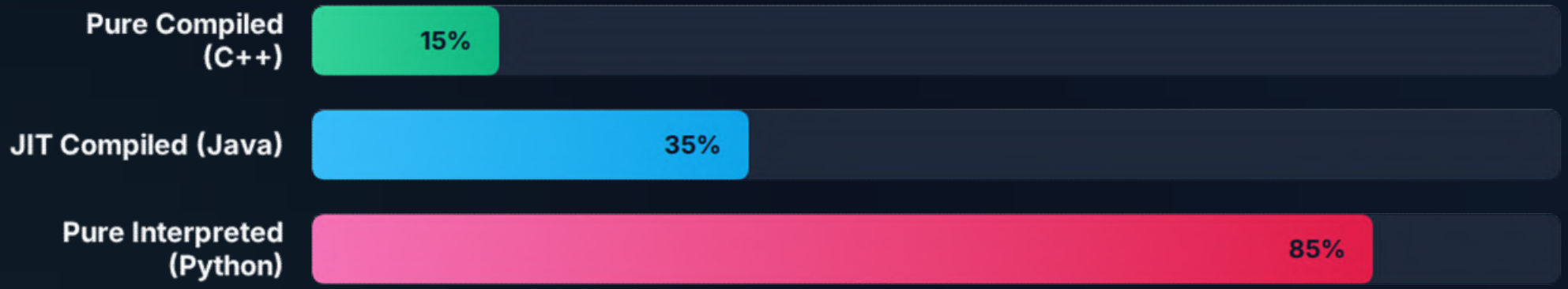
The Performance Trade-off

- **Slower Execution:** The CPU spends time translating code while the program runs, leading to noticeable performance drops in heavy computation.
- **Dependency:** The end-user must have the interpreter (like Python or Ruby runtime) installed to run the program.
- **Exposed Code:** Source code must be shared to run the application, which offers zero intellectual property protection.

Compiler vs. Interpreter

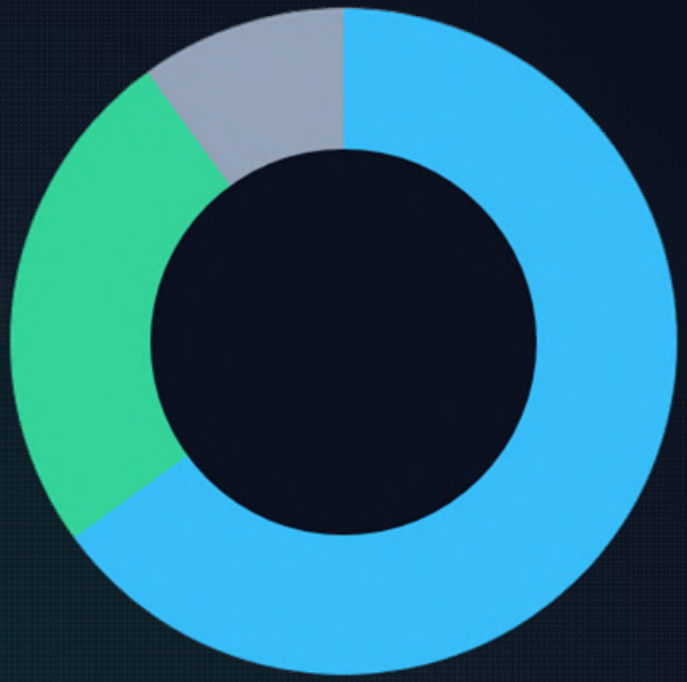
Feature	Compiler	Interpreter
Translation Method	Translates entire program at once	Translates line-by-line
Execution Speed	Very Fast (Pre-translated)	Slower (Translated at runtime)
Memory Requirement	High (Stores object code)	Low (No object code stored)
Debugging	Difficult (Errors shown at end)	Easier (Halts at exact error line)
Typical Languages	C, C++, Go, Rust	Python, Ruby, JavaScript, PHP

Runtime Performance Overhead



Lower numbers represent less runtime overhead. Compiled languages allocate hardware resources directly to execution, whereas interpreted languages dedicate significant runtime to continuous translation.

Language Paradigm in Web Development



- Interpreted / JIT (JS, Python, Ruby) - 65%
- Compiled (Go, Rust, C++) - 25%
- WebAssembly / Others - 10%

Despite the performance cost, interpreted and JIT languages dominate web ecosystems due to rapid development cycles, vast frameworks, and dynamic typing flexibility.

Hybrid Systems

The Best of Both Worlds: Just-In-Time (JIT) Compilation

Just-In-Time (JIT) Compilation

Hybrid

Compile + Interpret

How it Works (e.g., Java, C#)

1. The source code is first **compiled** into an intermediate, platform-independent format called *Bytecode*.

2. At runtime, a Virtual Machine (like the JVM) **interprets** the Bytecode.

3. **The JIT Magic:** The engine monitors execution. Frequently run ("hot") paths of Bytecode are instantly compiled down to native machine code on the fly for maximum speed.

Real-World Applications



Python (Interpreted)

Dominates Data Science and AI. Developers prefer the rapid iteration and simple syntax, trading raw speed for development speed.



Java (Hybrid/JIT)

The backbone of enterprise servers. Offers portability ("Write once, run anywhere") combined with near-native JIT performance.



C++ (Compiled)

Used in high-frequency trading, game engines, and OS kernels where maximum hardware control and execution speed are non-negotiable.

Problem Solving Techniques Using Programming

A Structured Approach to Algorithmic Thinking and Code Implementation

The Programmer's Reality

80%

Thinking & Planning

Coding is the Final Step

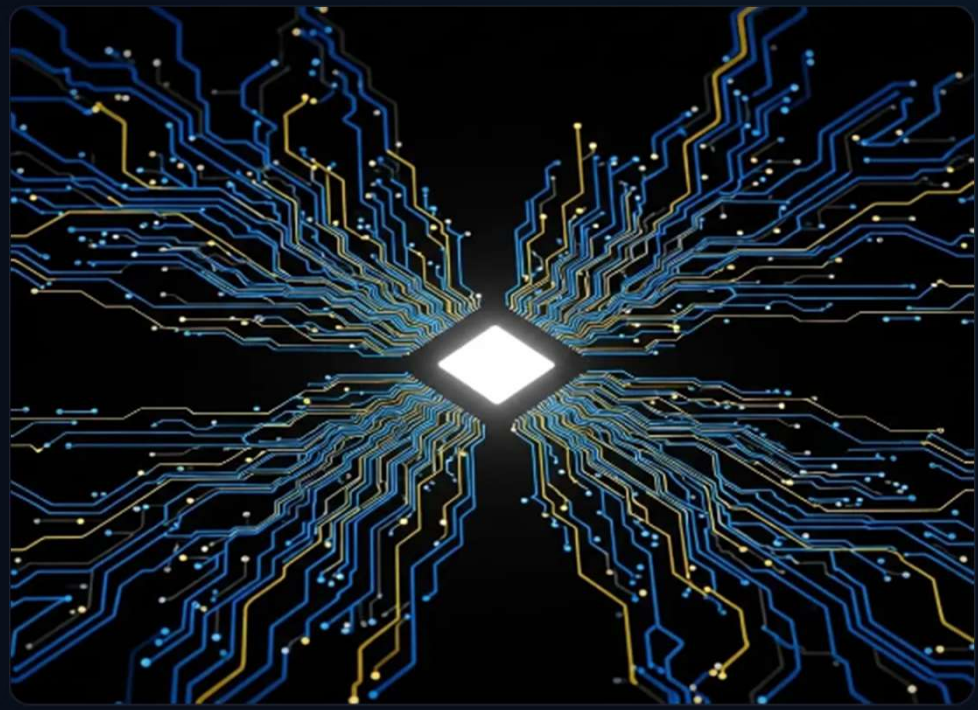
Programming is fundamentally about solving problems, not just typing syntax. The actual translation of logic into code should only occur after a solution has been thoroughly designed.

Mastering problem-solving techniques reduces bugs, improves efficiency, and allows developers to tackle highly complex systems with confidence.

Computational Thinking

Computational Thinking is the thought process involved in formulating a problem and expressing its solution in a way that a computer—human or machine—can effectively carry out.

- It bridges the gap between human intuition and machine logic.
- It allows us to scale solutions to massive datasets.
- It relies on logic, abstraction, and efficiency.



The Problem Solving Lifecycle

1. Understand

Read the problem. Identify inputs, outputs, constraints, and edge cases. Do not assume.

2. Plan & Design

Deconstruct the problem. Write algorithms, pseudocode, or draw flowcharts before coding.

3. Implement

Translate the logical plan into syntax using your chosen programming language.

4. Evaluate

Test for correctness, debug errors, and optimize for time/space complexity.

Phase 1: Deconstruct

Breaking down the insurmountable into the achievable.

Core Analytical Skills

Decomposition

The process of breaking down a complex, large-scale problem into smaller, more manageable sub-problems. If a task feels too difficult, it hasn't been broken down enough. You solve the macro by conquering the micro.

Pattern Recognition

Observing data or sub-problems and finding trends, similarities, or rules. Recognizing patterns allows you to use loops, reusable functions, or previously learned algorithmic strategies to solve new issues rapidly.

Tools for Planning



Pseudocode

Writing logic in plain English rather than strict syntax. It focuses on logic flow without getting bogged down by missing semicolons or brackets.



Flowcharts

Visual diagrams representing the sequence of steps and decisions needed. Excellent for mapping out complex conditional logic and loops.



Abstraction

Filtering out ignoring the characteristics that we don't need in order to concentrate on those that we do. Removing unnecessary noise.

Phase 2: Algorithms

Applying strategic paradigms to process data.

What is an Algorithm?

An algorithm is a finite sequence of well-defined, computer-implementable instructions to solve a class of problems or perform a computation.

- **Correctness:** It must produce the right output for every valid input.
- **Efficiency:** It should utilize minimal CPU time and memory space.
- **Finiteness:** It must eventually terminate; it cannot run infinitely without purpose.

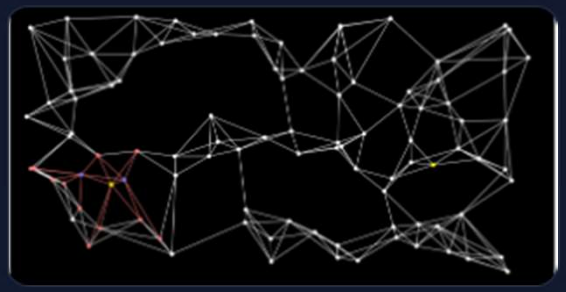


Algorithmic Paradigms



Brute Force

The most straightforward approach. Generates all possible solutions and checks which one works. Slow, but guarantees a find if it exists.



Greedy Algorithm

Makes the locally optimal choice at each step with the hope of finding a global optimum. Fast, but doesn't always yield the absolute best result.



Divide & Conquer

Recursively breaks down a problem into sub-problems of the same type, solves them independently, and combines their results.

Iteration vs. Recursion

Iteration (Loops): Repeatedly executes a block of code (like `for` or `while` loops) until a condition is met. Highly memory efficient but can be complex to write for intricate data structures.

Recursion: A function that calls itself to solve smaller instances of the same problem. Requires a base case to prevent infinite loops. Often yields cleaner code but uses more stack memory.



"First, solve the problem. Then, write the code."

John Johnson

Phase 3: Optimization

Making code not just work, but work efficiently.

Advanced Techniques

Dynamic Programming

An optimization over plain recursion. Whenever we see a recursive solution that has repeated calls for same inputs, we optimize it using *Memoization* (caching). It stores results of sub-problems to avoid computing them again.

Backtracking

An algorithmic-technique for solving problems incrementally. It systematically builds candidates to the solutions, and abandons a candidate ("backtracks") as soon as it determines that the candidate cannot possibly yield a valid solution.

Measuring Efficiency: Big O Notation

Complexity	Name	Description	Example Algorithm
O(1)	Constant	Time stays same regardless of data size.	Accessing an array index.
O(log n)	Logarithmic	Time increases slightly as data grows heavily.	Binary Search.
O(n)	Linear	Time grows proportionately with data.	Simple `for` loop traversal.
O(n^2)	Quadratic	Time grows exponentially based on data.	Nested loops (Bubble Sort).

Phase 4: Validation

Testing, Tracing, and Refining the implementation.

Debugging & Testing



Syntax vs Logic

Syntax errors prevent code from running entirely (typos). Logic errors allow code to run, but produce the wrong output. Logic errors are where real problem-solving applies.



Trace Tables

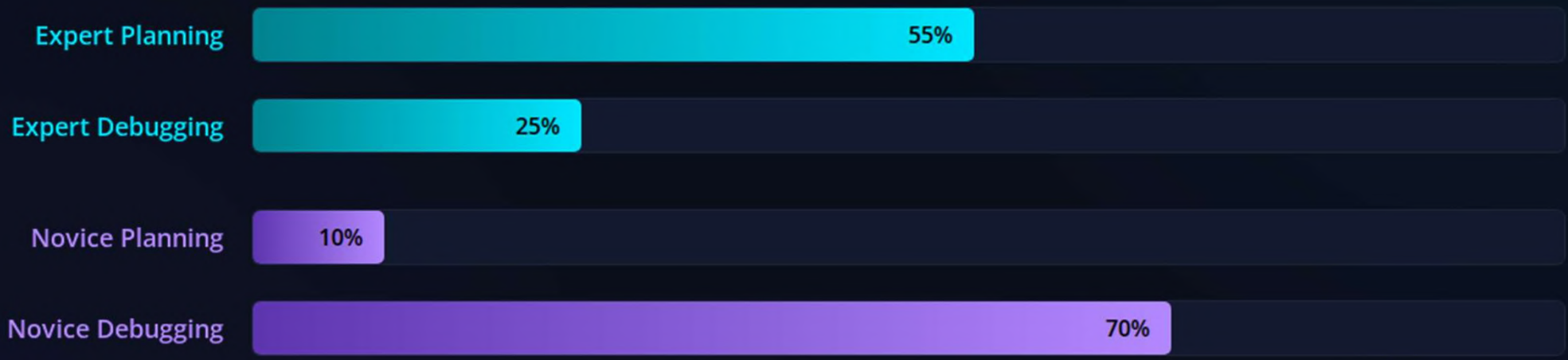
A manual testing technique used to track the value of variables as each line of code executes. Crucial for understanding complex loops and identifying exactly where logic fails.



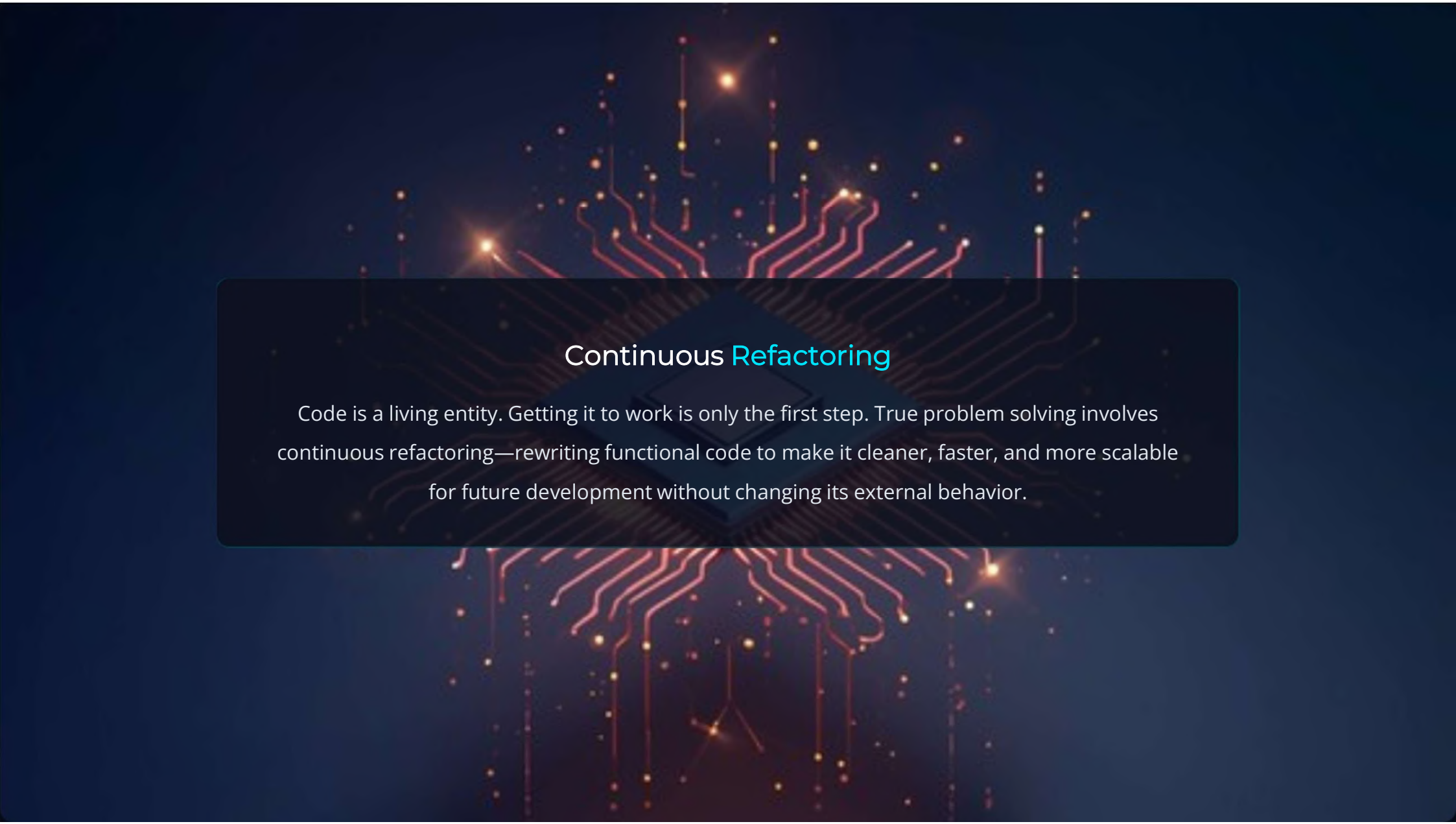
Edge Cases

Testing the absolute limits of your program. What happens if input is zero? Negative? Extremely large? A robust solution handles edge cases gracefully.

Time Allocation: Novice vs. Expert



Experts spend heavily on design, drastically reducing time stuck in "debug hell." Novices rush to code, resulting in brittle architecture and massive debugging time.


Continuous Refactoring
Code is a living entity. Getting it to work is only the first step. True problem solving involves continuous refactoring—rewriting functional code to make it cleaner, faster, and more scalable for future development without changing its external behavior.

Demystifying Algorithms

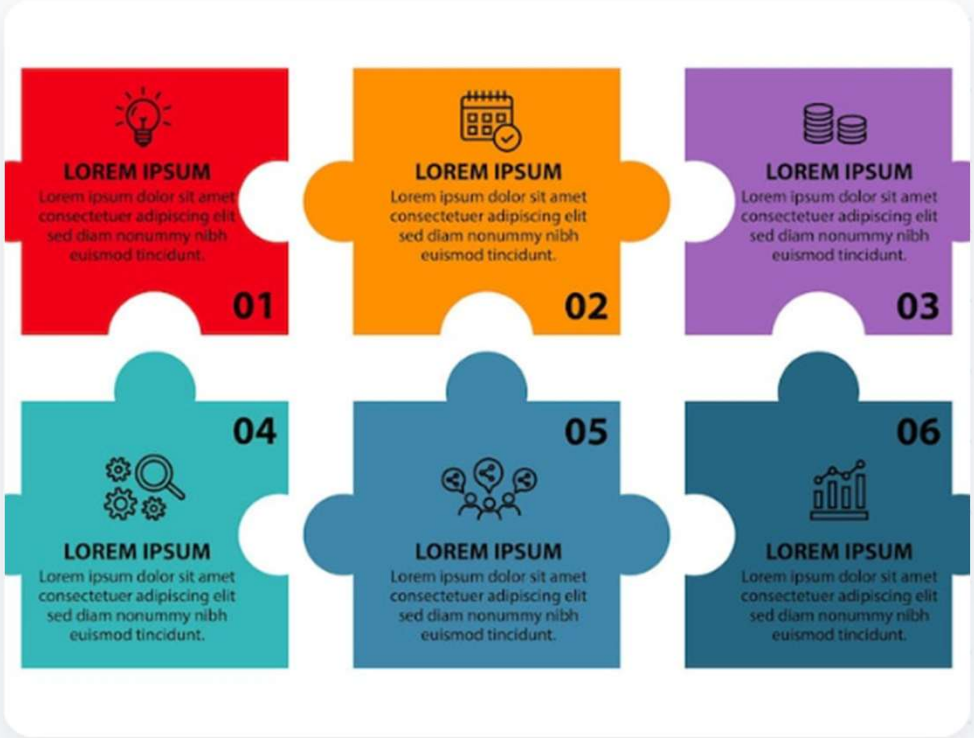
From Core Computer Science Concepts to Mastering Everyday Tasks

An algorithm is nothing more than a recipe. It's a sequence of unambiguous instructions for solving a problem.

Programming Wisdom

What Exactly is an Algorithm?

Source: elements.envato.com



At its core, an **algorithm** is a finite sequence of rigorously defined instructions. It takes a specific set of inputs, processes them through a logical flow, and produces a desired output.

While the term is heavily used in computer science to describe how software operates, algorithmic thinking is fundamentally a human trait used to solve problems efficiently in daily life.

Core Characteristics



Unambiguous

Every single step must be perfectly clear and exact. There can be no room for subjective interpretation or guesswork.



Finiteness

The algorithm must eventually stop. It cannot run in an infinite loop; it must reach a distinct endpoint or solution.



Feasibility

The instructions must be possible to execute with the resources available. A theoretical step that cannot be performed breaks the algorithm.

Writing an Algorithm

The structured framework for solving any problem.

The 4-Step Process

1. Define

Clearly understand the problem you are trying to solve.

2. Identify IO

Determine exactly what goes in (Input) and what must come out (Output).

3. Sequence

Draft the step-by-step logic required to turn inputs into outputs.

4. Refine

Test the steps, look for edge cases, and optimize for speed.

Phase 1: Setup

Step 1: Understand the Problem

Before writing a single step, you must fully grasp the goal. Ask yourself: What is the exact definition of success here? Are there any constraints (e.g., time limits, budget, specific tools)?

Step 2: Define Inputs & Outputs

Inputs: The raw materials, data, or state you start with.

Outputs: The finalized product, calculated result, or changed state you end with. The algorithm is simply the bridge between the two.

Phase 2: Execution

Step 3: Draft the Sequence

Write down the chronological steps. Use plain language (pseudocode) or visual flowcharts. Use logic structures like conditional statements (If/Then) and loops (Repeat until).

Step 4: Review and Refine

Walk through your steps manually with test data. Does it account for missing inputs? Can a step be skipped or simplified to make the whole process faster? Refinement is where good algorithms become great.

The Golden Rule

1

Action Per Step

Keep Instructions Atomic

The most common mistake when writing algorithms is combining multiple complex actions into a single step.

To ensure unambiguity, a single step should represent a single, clear action. Instead of "Make the sandwich", write "1. Take two slices of bread. 2. Spread peanut butter on one slice."

5 Daily Life Examples

Algorithmic thinking happens long before you touch a keyboard.

Ex 1: Making Coffee

Goal: Brew a cup of coffee.

- **Step 1:** Check if there is water in the kettle.
- **Step 2:** IF empty, fill kettle with water.
- **Step 3:** Turn on kettle to boil.
- **Step 4:** Place filter in dripper and add 15g coffee grounds.
- **Step 5:** Pour boiling water over grounds.
- **Step 6:** Wait for dripping to stop. Serve.



Ex 2: GPS Navigation

Goal: Drive from Home to Work.

- **Input:** Current Location, Destination.
- **Step 1:** Calculate all possible route combinations.
- **Step 2:** Fetch real-time traffic data for routes.
- **Step 3:** Apply pathfinding algorithm (like Dijkstra's) to find shortest time.
- **Output:** Step-by-step turn instructions to the driver.

Source: www.shutterstock.com



Ex 3: Doing Laundry

Source Source: www.cleanipedia.com



1. Conditional Logic

IF item is white, put in Basket A. ELSE IF colored, put in Basket B.



2. Execution Sequence

Add detergent -> Select temperature based on color -> Press Start.



3. Looping Task

REPEAT (pick up item, fold it, place in pile) UNTIL basket is empty.

Ex 4: Dictionary Search



Source: www.nytimes.com

The "Binary Search" Algorithm

When finding a word (e.g., "Lion"), you don't read page 1, then page 2. You use an algorithm:

- Open dictionary to the exact middle.
- IF "Lion" is alphabetically before the current page, ignore the right half.
- Open to the middle of the remaining left half.
- REPEAT until the word is found.

Ex 5: Organizing a Bookshelf

Goal: Sort a messy shelf alphabetically by author. (Insertion Sort)

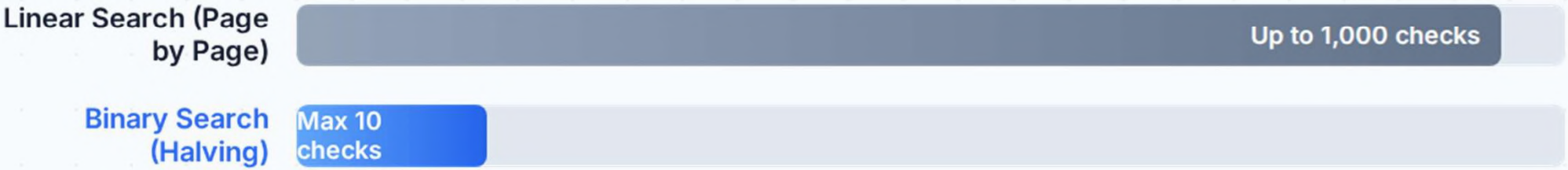
1. Start with the second book on the left.
2. Compare it to the first book. IF it comes earlier alphabetically, swap them.
3. Move to the third book. Compare it backwards through the sorted books and insert it into its correct alphabetical position.
4. Repeat this process for every book until you reach the end of the shelf.

Mapping Life to Code Concepts

Daily Life Action	Algorithmic Concept	Definition
Following a Recipe	Sequence	Running instructions in a strict, top-to-bottom order.
Checking Traffic before driving	Conditionals (If/Else)	Making decisions and branching paths based on state.
Folding a whole basket of laundry	Iteration (Loops)	Repeating a block of instructions until a condition is met.
Finding a word in a dictionary	Binary Search	Efficiently finding an item in a sorted list by halving it.

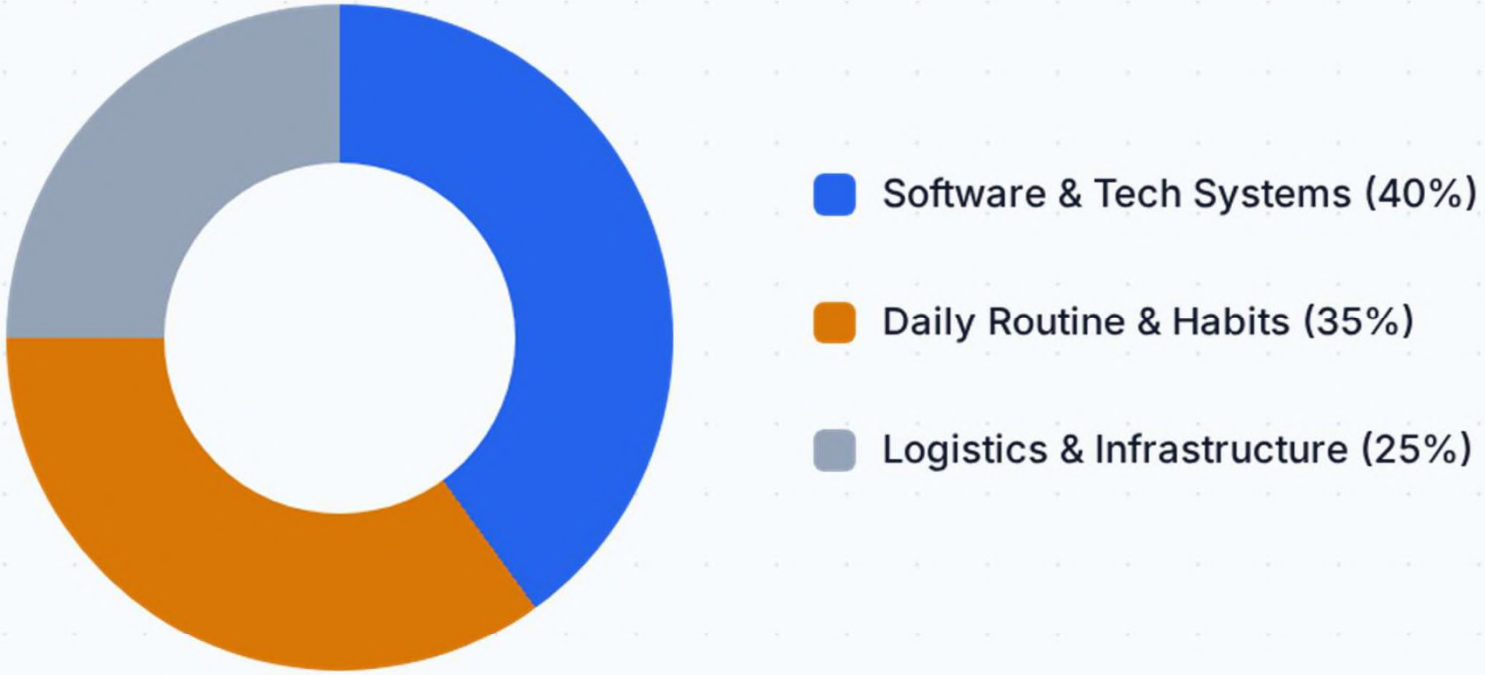
Why Use Algorithms? Efficiency

Time taken to find a word in a 1,000-page dictionary:

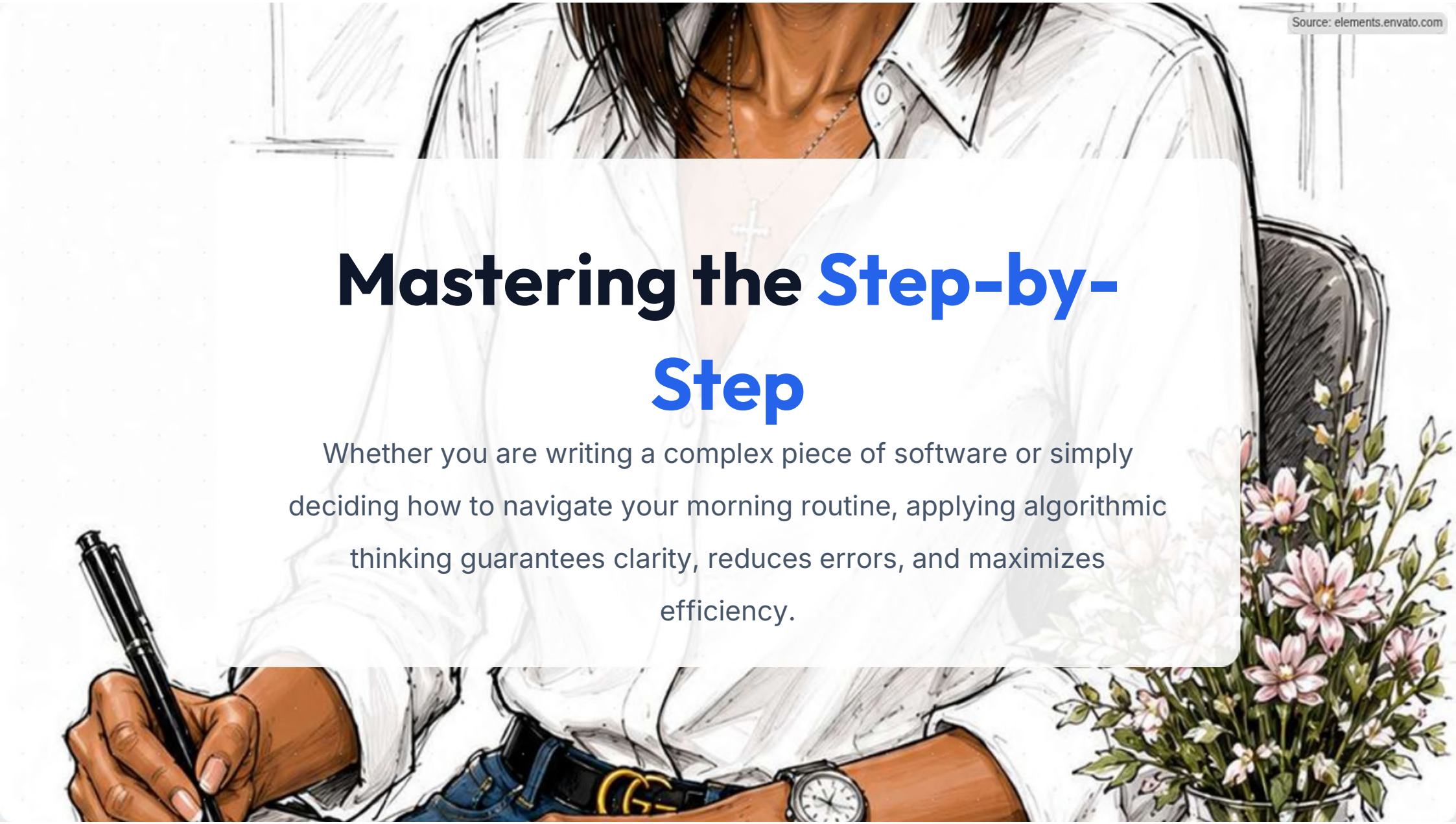


A good algorithm doesn't just solve the problem; it optimizes the resources (time and energy) required to reach the solution. Working smarter, not harder.

Algorithmic Impact



Algorithmic thinking extends far beyond computer screens. It is the underlying framework for how human society organizes, optimizes, and executes daily functions.



Mastering the Step-by-Step

Whether you are writing a complex piece of software or simply deciding how to navigate your morning routine, applying algorithmic thinking guarantees clarity, reduces errors, and maximizes efficiency.

W '

g i y n gp- t s g k

EA t W

w D E Gp
D G

h o Rl D E G
y Ri E Gg E w Dq D i CCG
g x Rw D E E E G

FAfi 2 b

b n

```
INIT SET Wfi
/ GG SET total TO 0 :G
COMPUTE CALCULATE Wp
G
```

HAl 2 s e h

g Gz
G

e l

fi READ D GET D INPUT
D D G
READ studentName

s w Rr

Gl

/ GGD `userAge` :G

e s

fi `PRINT` D `DISPLAY` D `OUTPUT` D

D G

```
PRINT "Validation Successful"
```

l Ae p 4d o 5

k G W

`IF [condition]` W G

`THEN` W G

`ELSE` l `ELSEIF` W G

`ENDIF` W D G

KAI 2 o

d o 4hs v 5

fi

G

z W`FOR count = 1 TO 10` G

i W`FOR each item IN list` G

g `ENDFOR` G

d o 4 klog5

fi

G

l W`WHILE condition IS true` G

y

G

l `ENDWHILE` G

LAp e 4h 5

' D G

e Rfi `FUNCTION` `MODULE` D G

g Rfi `CALL` D D G

s Rfi `RETURN` G

MAI d

g w G

d	t d	t l
fl p	`SET score TO 0`	`score = 0`
fi p	`READ userName`	`userName = input()`
i s	`IF score > 10 THEN PRINT "Win"`	`if score > 10:` `  print("Win")`
p s	`FOR i = 5 DOWNTO 1`	`for i in range(5, 0, -1):`
t y	`RETURN finalValue`	`return finalValue`



Everyday Algorithms

20 Real-World Problems and their Step-by-Step Computational Solutions



Q1: How do you make a cup of tea?

Can you define the exact step-by-step instructions?

Algorithm: Sequence / Sequential

1. **Boil Water:** Turn on the kettle and wait for the water to reach 100°C.
2. **Prepare Cup:** Place a tea bag or tea leaves into an empty cup.
3. **Pour:** Pour the boiling water into the cup over the tea.
4. **Steep:** Wait for exactly 3 minutes.
5. **Finish:** Add milk and sugar to taste, stir, and serve. *(Note: Steps must be executed in this exact order to succeed).*



Q2: How does an ATM dispense cash?

What decisions must the machine make before giving you money?

Algorithm: If-Else / Decision Making

1. **Input:** Read user's inserted card and requested PIN.
2. **IF** PIN is invalid **THEN** eject card and stop. **ELSE** continue to next step.
3. **Input:** Read requested withdrawal amount from user.
4. **IF** Account Balance >= Requested Amount **THEN** dispense cash and deduct from balance.
5. **ELSE** display error "Insufficient Funds" and return to main menu.



Q3: How do you choose what to wear?

How does your brain process the weather to make a choice?

Algorithm: Multiple Conditions (Else-If)

1. **Check:** Look at the weather forecast and current temperature.
2. **IF** it is raining **THEN** wear a raincoat and boots.
3. **ELSE IF** temperature < 15°C **THEN** wear a heavy winter jacket.
4. **ELSE IF** temperature > 25°C **THEN** wear a light t-shirt and shorts.
5. **ELSE** (default action) wear a standard long-sleeve shirt and jeans.



Q4: How do you find the most expensive item out of three choices?

Without trying to remember all prices at once.

Algorithm: Comparison / Maximum

1. **Initialize:** Look at Item A and Item B.
2. **Compare:** **IF** Price(A) > Price(B) **THEN** set TempMax = Price(A). **ELSE** set TempMax = Price(B).
3. **Compare Again:** Look at Item C.
4. **IF** Price(C) > TempMax **THEN** set FinalMax = Price(C). **ELSE** FinalMax = TempMax.
5. **Output:** The FinalMax is the most expensive item.



Q5: How do you spot your friend in a classroom?

Assuming the seating is completely random and unassigned.

Algorithm: Linear Search

1. **Start:** Look at the very first desk in the front row.
2. **Check:** Is the person sitting here my friend?
3. **IF** YES, walk over to them and **STOP** searching.
4. **IF** NO, move your eyes to the exact next consecutive desk.
5. **Loop:** Repeat steps 2-4 until you find your friend or check every desk.



Q6: How do you find a specific book in a library?

Assuming the shelf is already perfectly sorted alphabetically.

Algorithm: Binary Search

1. **Start:** Go to the exact middle book on the shelf and read its title.
2. **Check:** **IF** it is the book you want, take it and **STOP**.
3. **Left Half:** **IF** your book comes alphabetically *before* the middle book, completely ignore the right half of the shelf.
4. **Right Half:** **IF** your book comes *after*, ignore the left half.
5. **Loop:** Find the middle of the remaining half and repeat Steps 2-4 until found.



Q7: How do you pick the fastest checkout line?

Scanning multiple open registers at the supermarket.

Algorithm: Minimum / Selection

1. **Baseline:** Look at Line 1. Count the people and store this number as 'ShortestCount'. Store 'Line 1' as 'BestLine'.
2. **Next:** Move to Line 2 and count the people.
3. **Update:** IF Line 2 count < 'ShortestCount', update 'ShortestCount' to this new number and set 'BestLine' to Line 2.
4. **Loop:** Repeat Steps 2-3 for every open register in the store.
5. **Action:** Walk to the register currently stored as 'BestLine'.



Q8: How do you arrange students by height?

Organizing a scattered group from shortest to tallest.

Algorithm: Sorting / Selection Sort

1. **Scan:** Look at the entire unsorted group of students.
2. **Find:** Identify the absolute shortest student in the group.
3. **Swap:** Move that shortest student to the very first position (Position 1) in the new line.
4. **Scan Again:** Look at the *remaining* unsorted students to find the next shortest.
5. **Loop:** Place them in Position 2. Repeat this finding and placing process until no students remain.



Q9: How do cashiers give the minimum coins for change?

If they owe you 65 cents, what is their logic?

Algorithm: Greedy Algorithm

1. **Identify:** Note the total change required (e.g., 65¢) and look at the largest coin available (Quarter / 25¢).

2. **Take:** **WHILE** the largest coin is <= remaining change, hand one over and subtract its value from the total (65 - 25 = 40. Then 40 - 25 = 15).

3. **Switch:** When the current coin is too big (25 > 15), move down to the next largest coin denomination (Dime / 10¢).

4. **Repeat:** Hand over one Dime (15 - 10 = 5). Move to next coin (Nickel / 5¢). Hand over Nickel (5 - 5 = 0).

5. **Stop:** When remaining change equals 0, the algorithm is complete.



Q10: How does GPS find the fastest route?

Out of millions of possible road combinations.

Algorithm: Shortest Path / Graph Algorithm

1. **Map Data:** Treat every intersection as a 'Node' and every connecting road as an 'Edge'.
2. **Assign Weight:** Assign a numerical value (Time) to each Edge based on distance, speed limit, and live traffic.
3. **Explore:** Start at the Origin. Calculate the total time to reach all directly connected neighbor Nodes.
4. **Update:** Move to the neighbor with the lowest time. Update times to its neighbors. Never revisit a Node if a slower path is found.
5. **Finish:** Once the Destination Node is reached, trace the lowest-time path backwards to establish the route.



Q11: How does a delivery app handle your food order?

Tracking the meal without human intervention.

Algorithm: Sequence + Conditions + States

1. **State 1 (Cart):** Wait for user to trigger payment. **IF** payment succeeds, update database to 'Paid'.

2. **State 2 (Prep):** Send order to restaurant system. **WAIT UNTIL** chef presses 'Food Ready' button.

3. **State 3 (Pickup):** Ping nearest driver GPS. **WAIT UNTIL** driver GPS matches restaurant GPS.

4. **State 4 (Delivery):** Update app UI to show map tracking.

5. **State 5 (Delivered):** **IF** driver marks complete OR GPS matches customer house, close order and prompt for tip.



Q12: How does the portal decide if you can take exams?

Processing attendance and exemptions automatically.

Algorithm: Conditional / Decision Algorithm

1. **Calculate:** Retrieve 'Classes Attended' and 'Total Classes'. Calculate Percentage = (Attended / Total) * 100.

2. **Primary Check:** **IF** Percentage >= 75% **THEN** set status to ELIGIBLE and exit algorithm.

3. **Secondary Check:** **ELSE IF** student has a 'Valid Medical Certificate' uploaded in the database...

4. **Exception:** ...**THEN** set status to ELIGIBLE and exit algorithm.

5. **Default:** **ELSE** set status to NOT ELIGIBLE and block exam hall ticket generation.



Q13: How does a website validate your new password?

Ensuring security before you click submit.

Algorithm: Boolean Logic / Conditions

1. **Evaluate Length:** Let Cond1 = (Password length is >= 8 characters). Results in True or False.

2. **Evaluate Numbers:** Let Cond2 = (Password contains at least one digit 0-9).

3. **Evaluate Symbols:** Let Cond3 = (Password contains a special character like @, #, $).

4. **Logic Gate:** **IF** (Cond1 **AND** Cond2 **AND** Cond3) == TRUE...

5. **Output:** ...**THEN** enable Submit button. **ELSE** show error message detailing which condition is False.



Q14: How does a washing machine clean clothes?

Moving through phases without you touching a button.

Algorithm: State-Based Algorithm

1. **State: FILL.** Open water valve. **WAIT UNTIL** water level sensor returns 'FULL'. Close valve.
2. **State: WASH.** Turn on drum motor to agitate. Set timer for 30 minutes. **WAIT UNTIL** timer == 0.
3. **State: DRAIN.** Open drain pump. **WAIT UNTIL** water level sensor returns 'EMPTY'.
4. **State: SPIN.** Turn on drum motor at high speed (1000 RPM) for 10 minutes to extract water.
5. **State: END.** Stop motor, unlock door mechanism, and beep 3 times.



Q15: How does a traffic light run continuously?

Managing intersections 24/7 without stopping.

Algorithm: Loop / Repetition

1. **Start Condition:** **WHILE** (System Power == ON) do the following steps:
2. **Action 1:** Turn RED light ON. Turn other lights OFF. Wait for exactly 60 seconds.
3. **Action 2:** Turn GREEN light ON. Turn RED OFF. Wait for exactly 60 seconds.
4. **Action 3:** Turn YELLOW light ON. Turn GREEN OFF. Wait for exactly 5 seconds.
5. **Loop Reset:** Reached end of loop block. Automatically jump back to Step 1 and repeat infinitely.



Q16: How does your phone pull up a specific song instantly?

Out of a library of 10,000 downloaded tracks.

Algorithm: Searching / Indexing

1. **Input:** User types a song name (the "Key").
2. **Hash Function:** The system converts the text string into a mathematical hash code.
3. **Index Lookup:** The system looks up this hash code in a pre-built table (an Index) that stores exact memory addresses.
4. **Direct Access:** The OS jumps directly to the hard drive sector indicated by the index, entirely skipping over the other 9,999 songs.
5. **Execution:** Load the audio file into memory and press Play.



Q17: How do students fill a college bus efficiently?

Allocating resources without overflowing.

Algorithm: Iteration + Allocation

1. **Initialize:** Let 'StudentsWaiting' = total line count. Let 'EmptySeats' = 50.
2. **Condition:** **WHILE** (StudentsWaiting > 0) **AND** (EmptySeats > 0) do steps 3-4:
3. **Action:** Allow the first student in line to board and assign them the next available seat.
4. **Update State:** Subtract 1 from 'StudentsWaiting'. Subtract 1 from 'EmptySeats'. Loop back to Step 2.
5. **Exit:** **IF** EmptySeats == 0, display "Bus Full" and depart. **IF** StudentsWaiting == 0, just depart.



Q18: How does a busy kitchen handle multiple orders fairly?

Ensuring no customer is skipped or delayed unfairly.

Algorithm: Queue / FIFO (First-In, First-Out)

1. **Push:** A waiter punches in a new order. The system places this ticket at the extreme *BACK* of the virtual line (Queue).

2. **Trigger:** A chef finishes cooking and requests the next ticket.

3. **Pop:** The system removes the ticket from the extreme *FRONT* of the Queue (the oldest order).

4. **Execute:** Chef cooks the meal and marks it complete.

5. **Loop:** Repeat Step 2 and 3 continuously. Orders are strictly processed in the order they arrived.



Q19: How does the browser back button work?

Remembering your exact page history after clicking 5 links.

Algorithm: Stack / LIFO (Last-In, First-Out)

1. **Initialize:** Open browser. History Stack is empty.
2. **Push:** User visits Google. Push 'Google URL' onto the top of the Stack.
3. **Push:** User clicks Wikipedia. Push 'Wiki URL' onto the top of the Stack (covering Google).
4. **Pop:** User clicks the 'Back' button. The system Pops (removes) the current top item ('Wiki URL').
5. **Load:** The system looks at what is now on top ('Google URL') and loads that page.



Q20: How do you get ready in the morning?

Why must things happen in a specific order?

Algorithm: Sequence / Ordered Steps

1. **Trigger:** Wake up when the alarm rings at 7:00 AM.
2. **Prep:** Brush teeth and take a shower.
3. **Dress:** Put on clothes. *(Dependency Constraint: Underwear must be put on BEFORE pants. Socks BEFORE shoes).*
4. **Fuel:** Eat breakfast and pack bag.
5. **Exit:** Grab keys, walk out the door, and lock it. Executing out of order (e.g., locking door before exiting) causes a fatal error.

A VISUAL REFERENCE GUIDE

Flowchart Components

Every commonly used flowchart symbol — what it looks like, what it means, and when to use it.



For classroom use — Flowcharting & Process Design

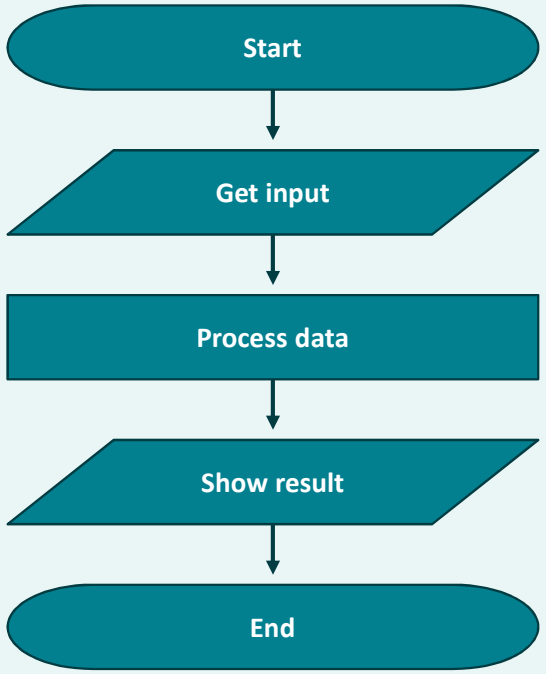
INTRODUCTION

What Is a Flowchart?

A flowchart is a diagram that represents a process, workflow, or algorithm using standardized symbols connected by arrows. Each symbol represents a different kind of step — an action, a decision, an input, or a stopping point.

- Uses a standard, universally recognized set of symbols
- Shows the exact sequence and logic of a process, step by step
- Read from top to bottom, following the arrows

A SIMPLE EXAMPLE



INTRODUCTION

Why Use Flowcharts?

C **Clarity** — Breaks a complex process down into simple, easy-to-follow steps.

S **Shared Communication** — Gives technical and non-technical people a common visual language.

D **Debugging & Analysis** — Makes it easy to spot errors, bottlenecks, or missing steps.

R **Documentation** — Creates a lasting, visual record of how a process or system works.

I **Process Improvement** — Highlights inefficiencies so the workflow can be redesigned and optimized.

COMPONENT 1 / 16

START / END



Terminator

Also known as: Start/End Symbol · Oval

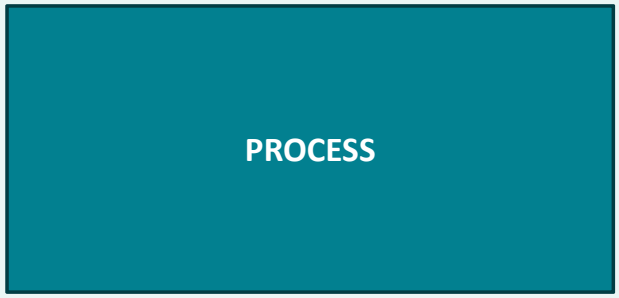
Marks the beginning or the end point of a flowchart. Every flowchart starts and ends with this symbol.

COMMONLY USED FOR

- Marking the start of a process
- Marking the end of a process
- Showing entry and exit points

COMPONENT 2 / 16

ACTION STEP



Process

Also known as: Action / Operation Box

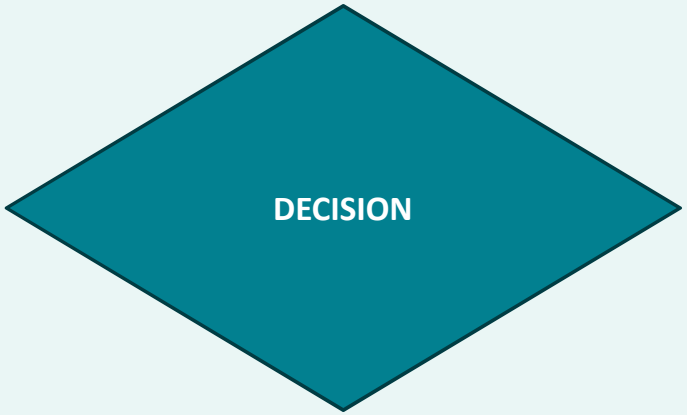
Represents a single step, action, or operation carried out within the process.

COMMONLY USED FOR

- Performing a calculation
- Manipulating or transforming data
- Any general executable instruction

COMPONENT 3 / 16

BRANCHING LOGIC



Decision

Also known as: Diamond

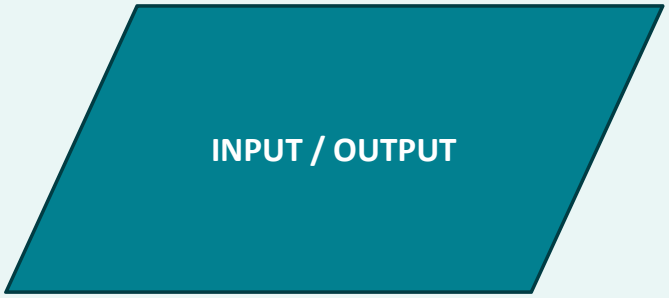
Marks a point where the flow branches based on a yes/no or true/false condition.

COMMONLY USED FOR

- If–else conditions
- Loop continuation checks
- Validation or eligibility checks

COMPONENT 4 / 16

DATA I/O



Input / Output

Also known as: Parallelogram · Data Symbol

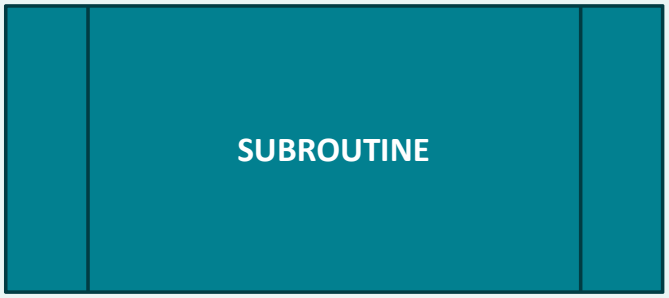
Represents data entering the process (input) or leaving it (output), such as user input or printed results.

COMMONLY USED FOR

- Reading a user's input
- Displaying or printing results
- Reading or writing a data file

COMPONENT 5 / 16

SUBROUTINE



Predefined Process

Also known as: Subroutine · Function Call

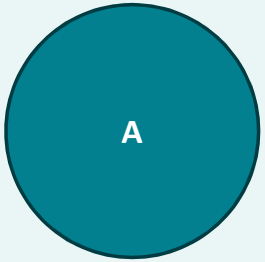
Represents a named process or subroutine that is fully defined in a separate, more detailed flowchart.

COMMONLY USED FOR

- Calling a function or module
- Reusable blocks used elsewhere
- Keeping a high-level chart simple

COMPONENT 6 / 16

NAVIGATION



Connector

Also known as: On-Page Connector · Junction

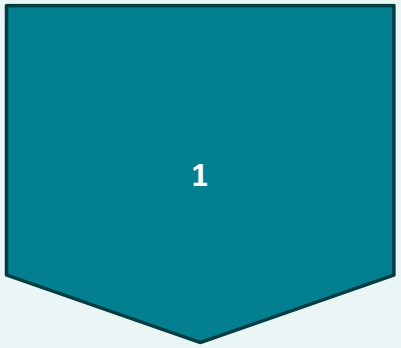
A small circle that links two points of the same flowchart, used to avoid long or crossing flow lines.

COMMONLY USED FOR

- Joining flow across a busy diagram
- Marking a jump point (e.g. “A” to “A”)
- Reducing crossed or tangled lines

COMPONENT 7 / 16

NAVIGATION



Off-Page Connector

Also known as: Page Connector

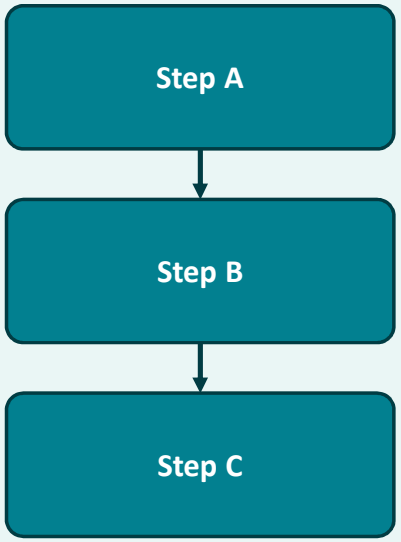
Indicates that the flow continues on a different page of a multi-page flowchart.

COMMONLY USED FOR

- Continuing a chart onto the next page
- Linking sections across page breaks
- Large or complex processes

COMPONENT 8 / 16

CONNECTORS



Flow Lines / Arrows

Also known as: Flowlines · Connectors

Arrows that connect symbols and show the direction the process flows in — the “roads” between every symbol.

COMMONLY USED FOR

- Showing sequence and order of steps
- Indicating a loop (arrow pointing back)
- Showing which branch a decision leads to

COMPONENT 9 / 16

OUTPUT



Document

Also known as: Report Symbol

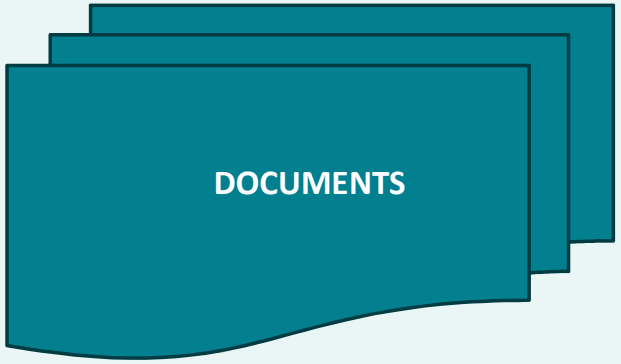
Represents a single printed document or report generated as output by the process.

COMMONLY USED FOR

- A printed report
- A single-page document output
- Written output of a specific step

COMPONENT 10 / 16

OUTPUT



Multiple Documents

Also known as: Batch Documents

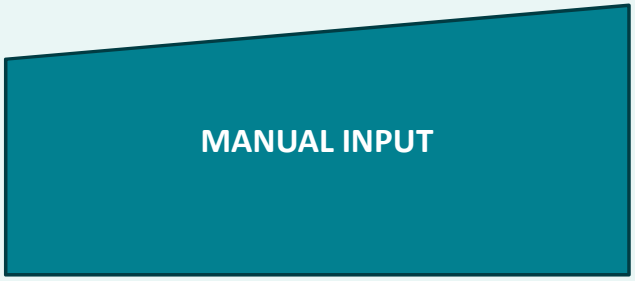
Represents several documents or reports being produced together, as a set.

COMMONLY USED FOR

- Batch or bulk reports
- Multiple copies of the same output
- Bundled paperwork from one step

COMPONENT 11 / 16

INPUT



Manual Input

Also known as: Keying Symbol

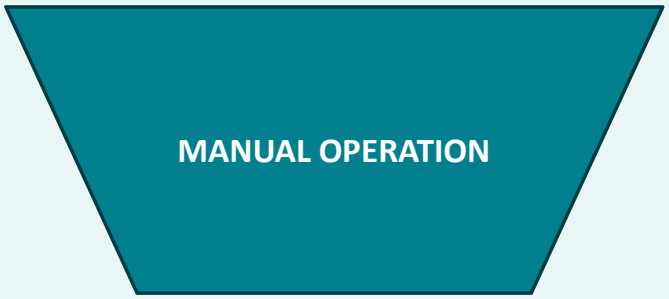
Represents data typed in manually by a person, such as via a keyboard.

COMMONLY USED FOR

- Typing in a value
- Filling in a form field
- Any user-provided keyed input

COMPONENT 12 / 16

ACTION STEP



Manual Operation

Also known as: Manual Task

Represents a step carried out by a person directly, without the help of a computer or machine.

COMMONLY USED FOR

- A human task within the process
- A physical or hands-on work step
- A manual quality check

COMPONENT 13 / 16

SETUP



Preparation

Also known as: Initialization Box

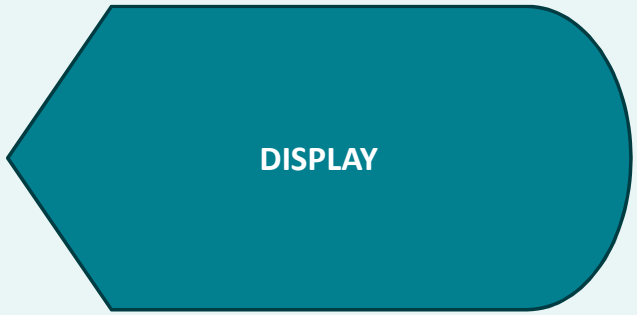
Represents a setup or initialization step performed before the main process begins.

COMMONLY USED FOR

- Initializing a variable
- Setting a loop's starting condition
- Configuration before processing starts

COMPONENT 14 / 16

OUTPUT



Display

Also known as: Screen Symbol

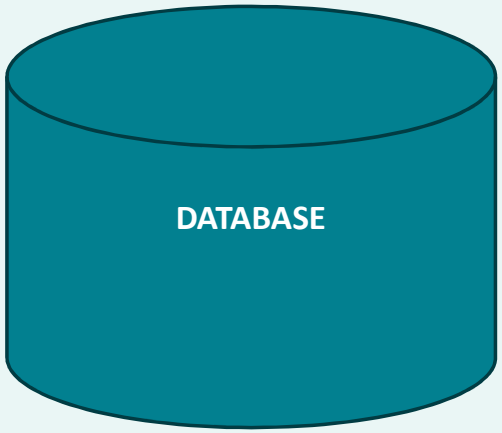
Represents information shown to a user on a screen or monitor, rather than printed.

COMMONLY USED FOR

- Showing a message on screen
- Displaying a calculated result
- Any on-screen-only output

COMPONENT 15 / 16

DATA STORAGE



Stored Data

Also known as: Database · Disk Symbol

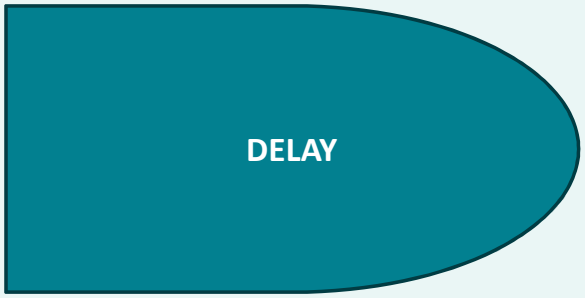
Represents data being saved to, or retrieved from, a database or storage device.

COMMONLY USED FOR

- Saving a record to a database
- Reading previously stored data
- Any persistent data storage step

COMPONENT 16 / 16

TIMING



Delay

Also known as: Wait Symbol

Represents a wait or pause in the process before the flow continues to the next step.

COMMONLY USED FOR

- A required waiting period
- A deliberate pause before the next action
- A timing delay in the process

QUICK REFERENCE

Flowchart Symbol Cheat Sheet



Terminator



Process



Decision



Input / Output



Predefined Process



Connector



Off-Page Connector



Flow Lines / Arrows



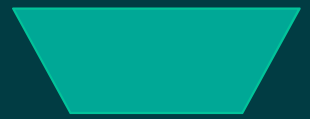
Document



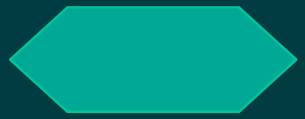
Multiple Documents



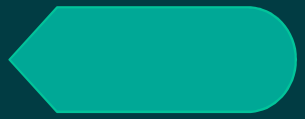
Manual Input



Manual Operation



Preparation



Display



Stored Data



Delay

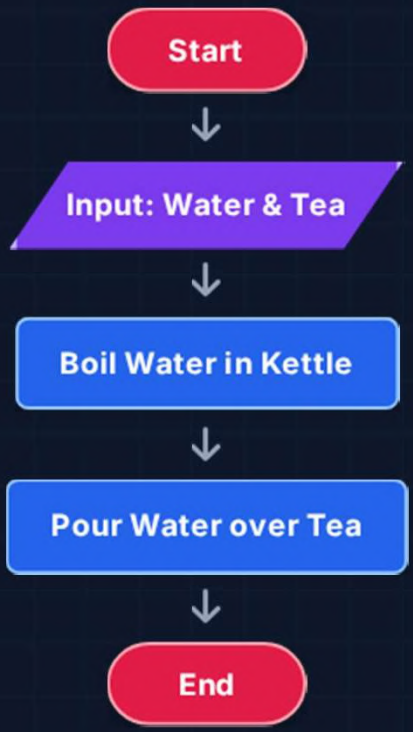


Everyday Algorithms

20 Flowchart Solutions for Daily Logic Problems

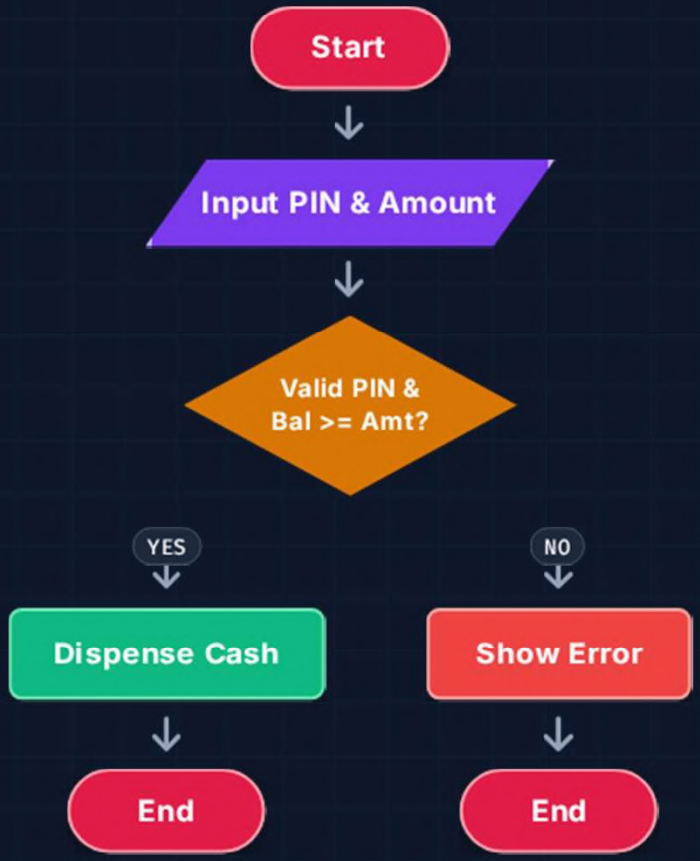
Sequence / Sequential Algorithm

1. Making Tea



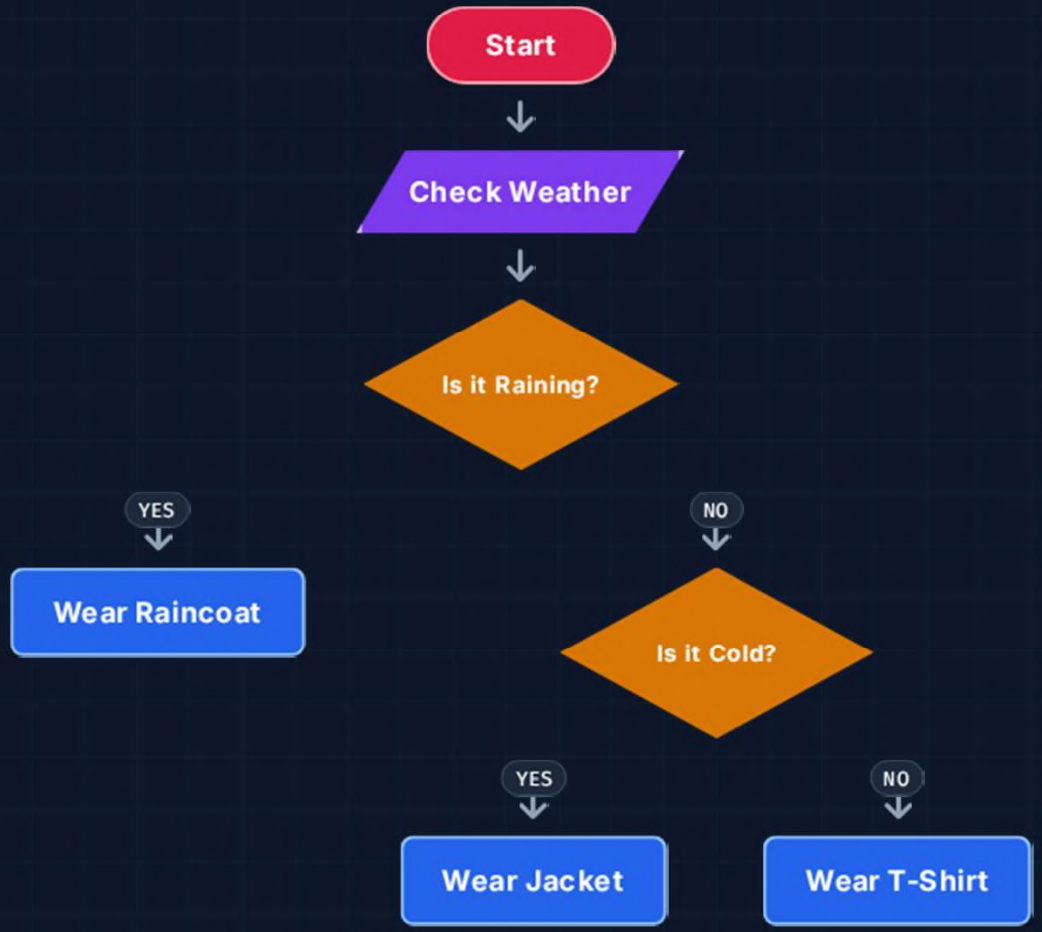
If-Else / Decision Making

2. ATM Withdrawal



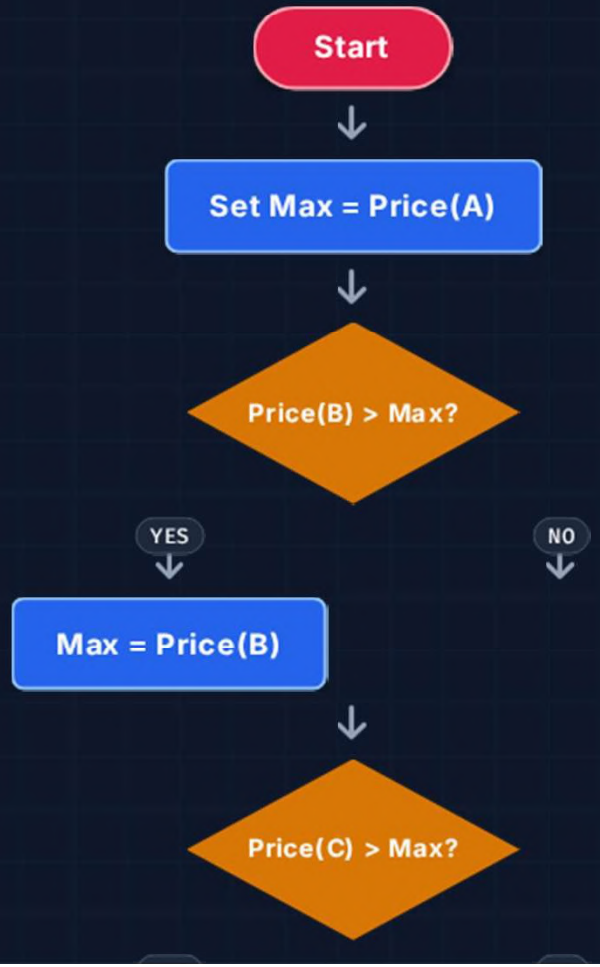
Multiple Conditions

3. Choosing What to Wear



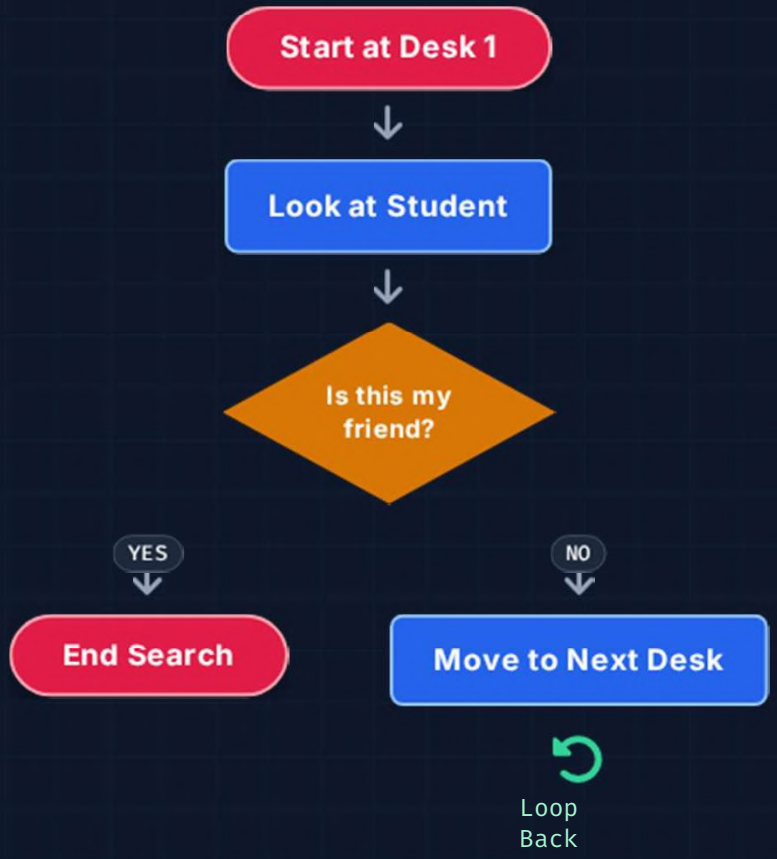
Comparison / Maximum

4. Finding Largest of Three Items (A, B, C)



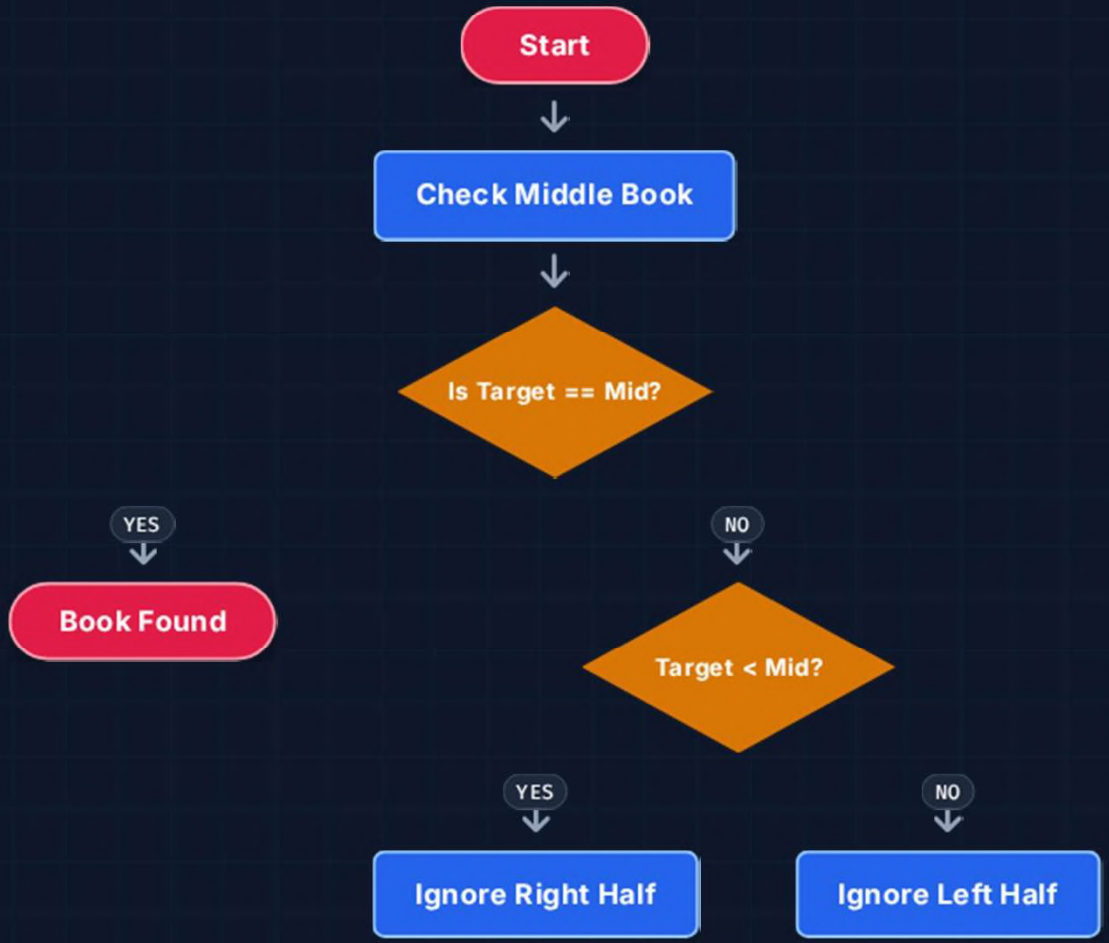
Linear Search

5. Finding a Friend in a Classroom



Binary Search

6. Finding a Book in a Sorted Library



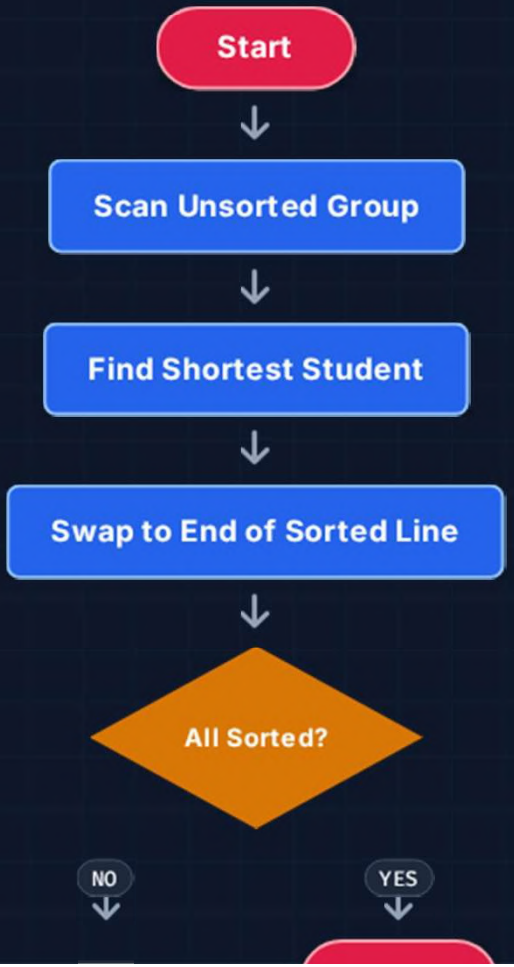
Minimum / Selection

7. Choosing the Shortest Queue



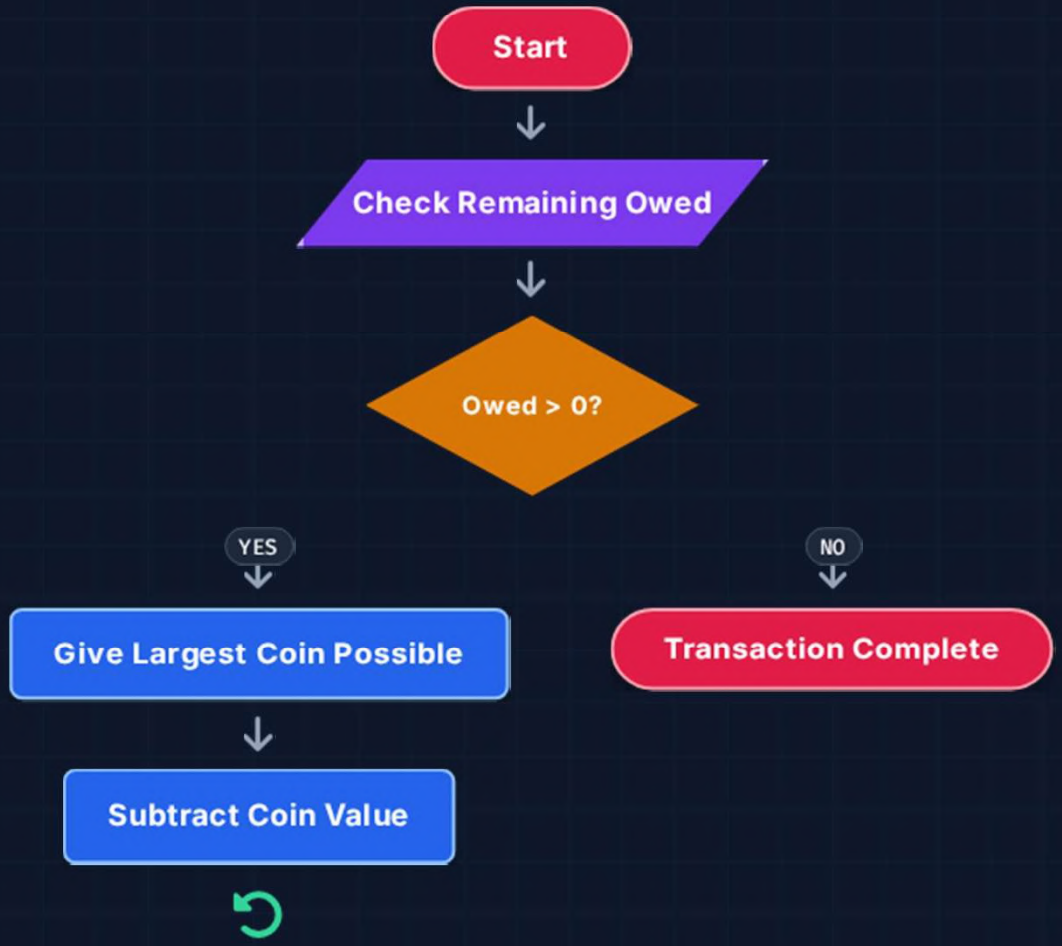
Sorting / Selection Sort

8. Arranging Students by Height



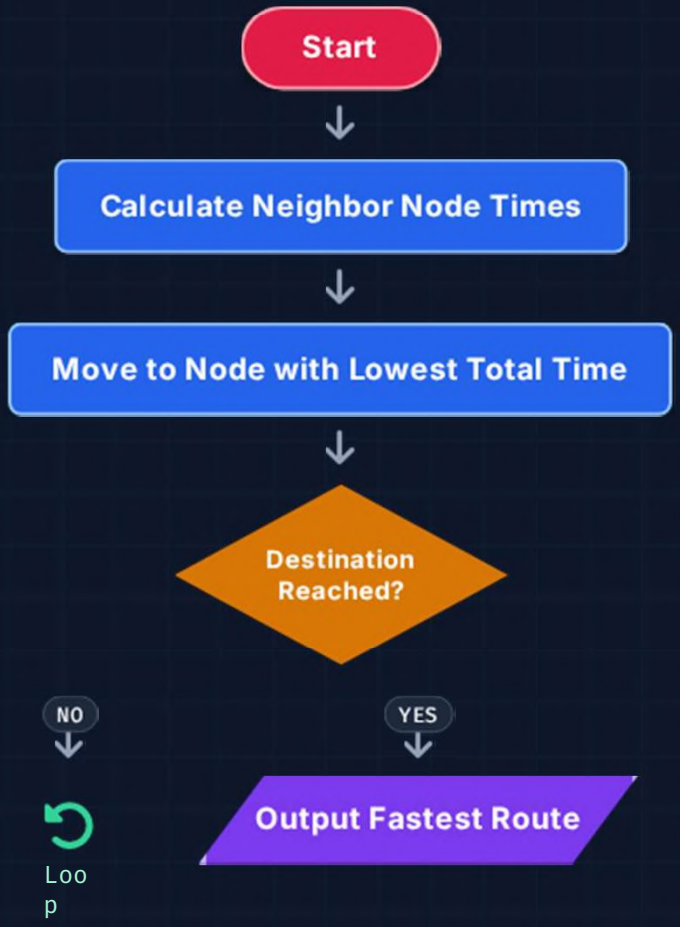
Greedy Algorithm

9. Making Change at a Shop



Shortest Path / Graph Algorithm

10. Google Maps / Navigation



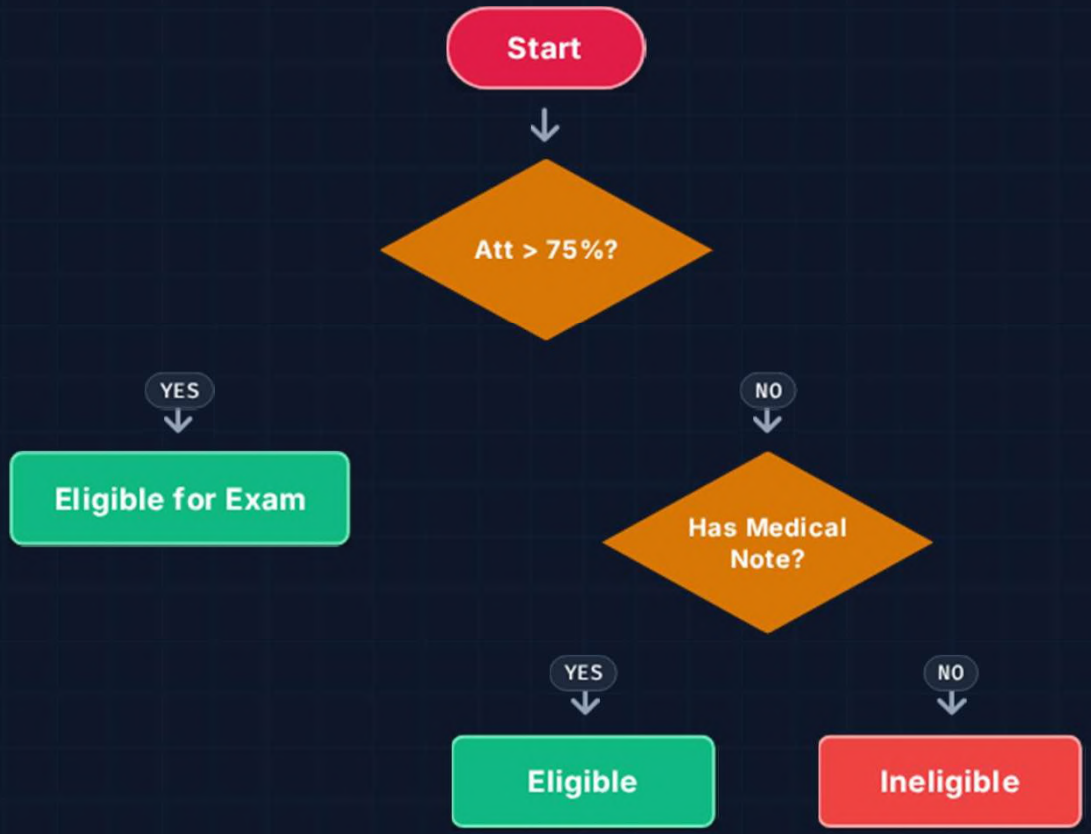
Sequence + Conditions + States

11. Online Food Ordering



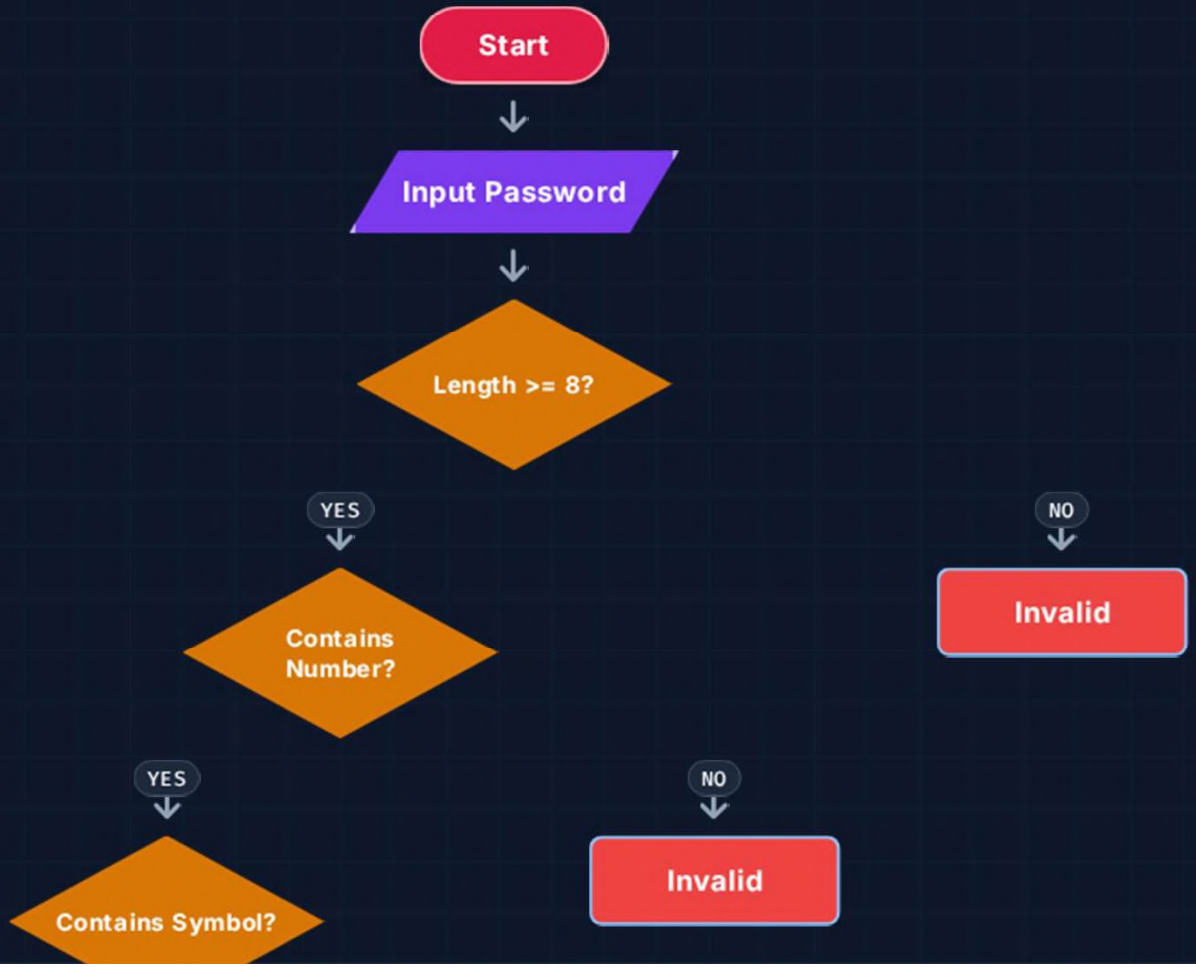
Conditional / Decision Algorithm

12. College Attendance Eligibility



Boolean Logic / Conditions

13. Password Validation



State-Based Algorithm

14. Washing Machine



Loop / Repetition

15. Traffic Signal



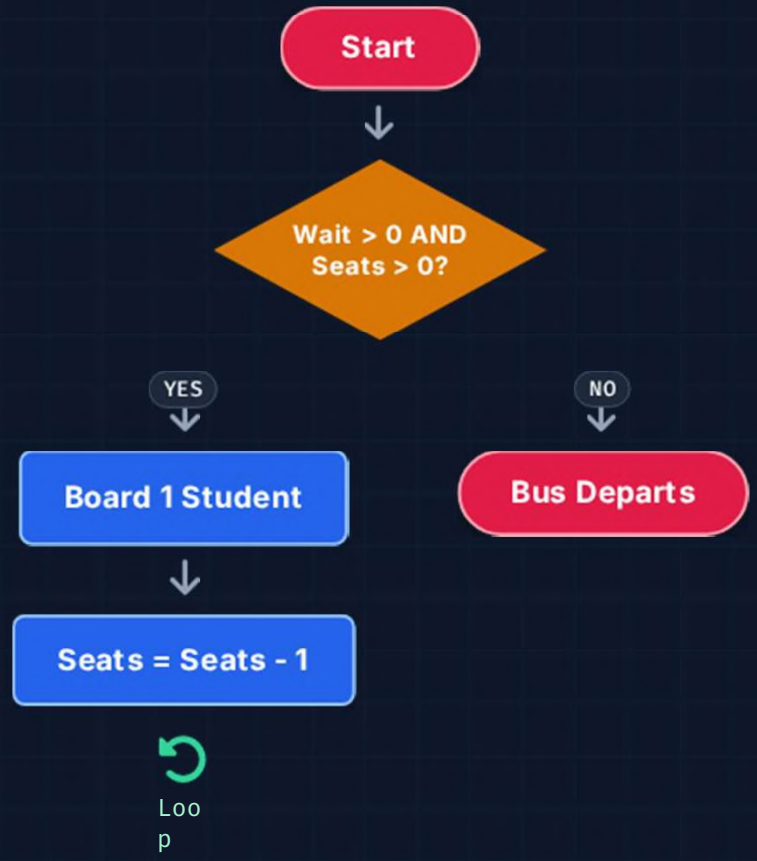
Searching / Indexing

16. Finding a Song on Your Phone



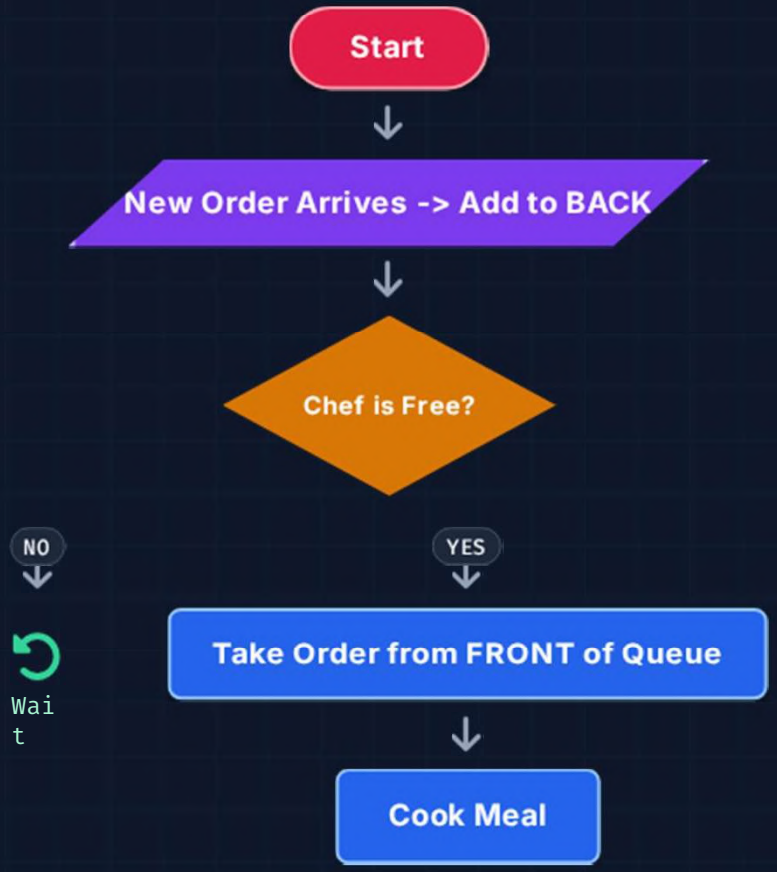
Iteration + Allocation

17. College Bus Seating



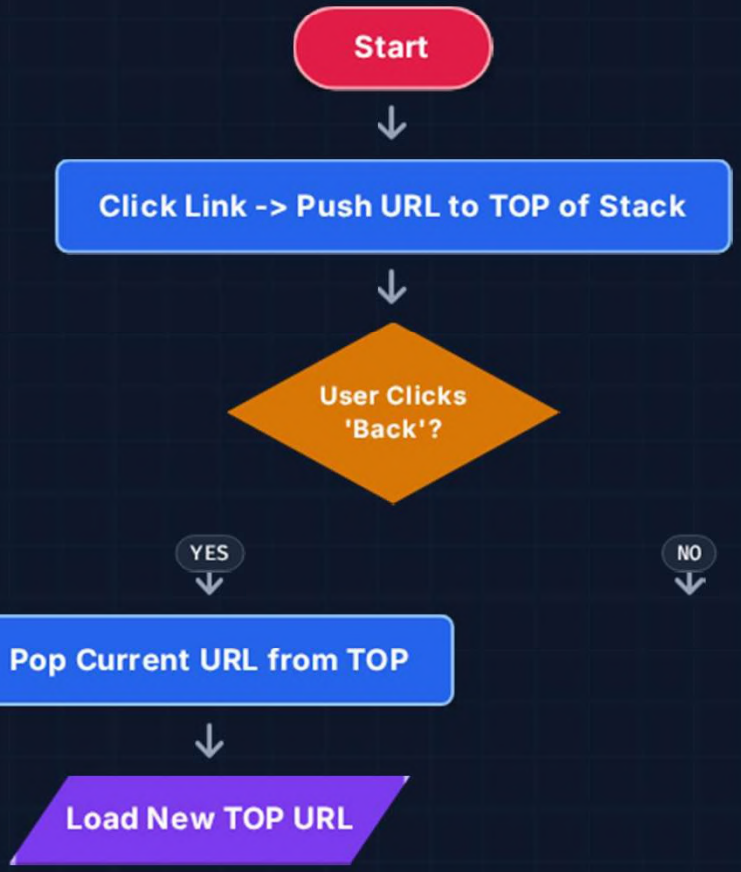
Queue / FIFO (First-In, First-Out)

18. Restaurant Order Queue



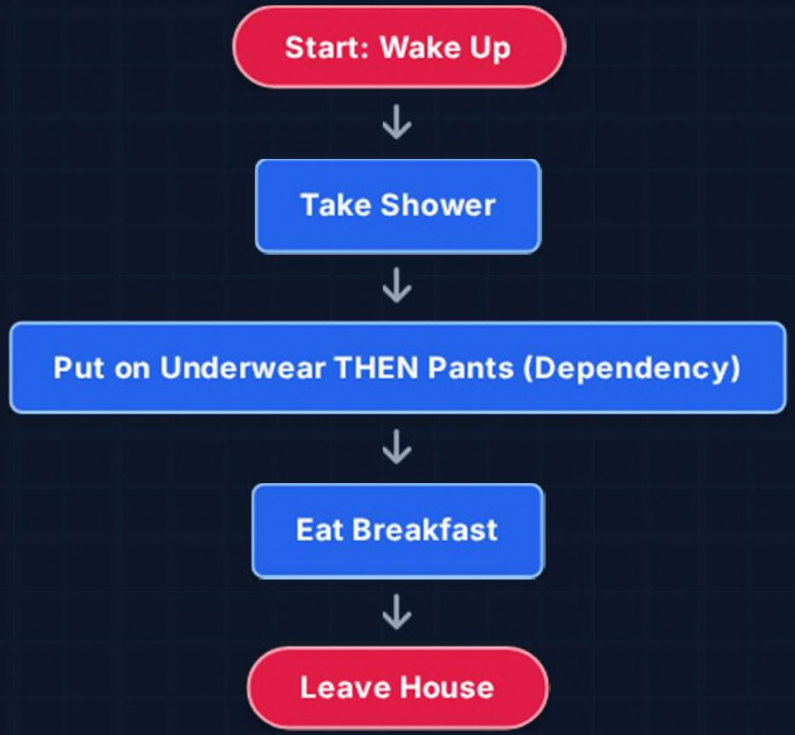
Stack / LIFO (Last-In, First-Out)

19. Browser Back Button



Sequence / Ordered Steps

20. Morning Routine



Flowchart Legend

Standard symbols used in logic design



Start / End Points

Actions / Computations

Yes/No Branching

Input / Output

FLOWCHART SOLUTIONS

20 Everyday Algorithms

The complete flowchart for each of the 20 problems — built from standard flowchart symbols: terminators, processes, decisions, and data I/O.



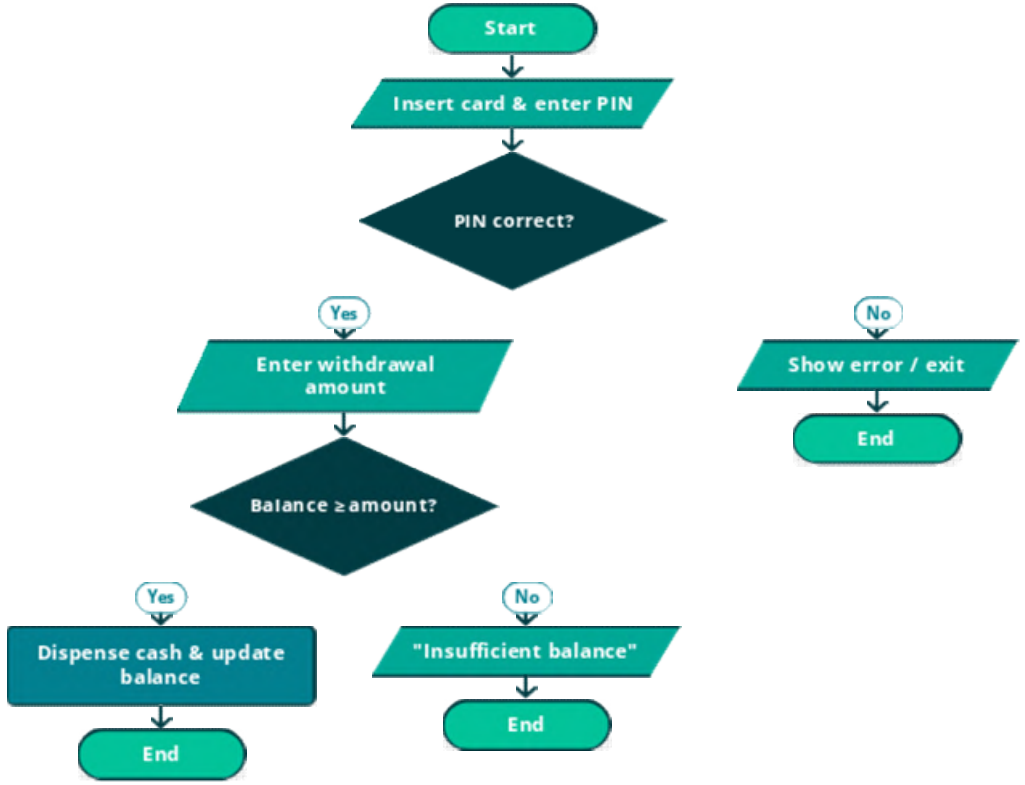
PROBLEM 1 SEQUENCE / SEQUENTIAL ALGORITHM

Making Tea



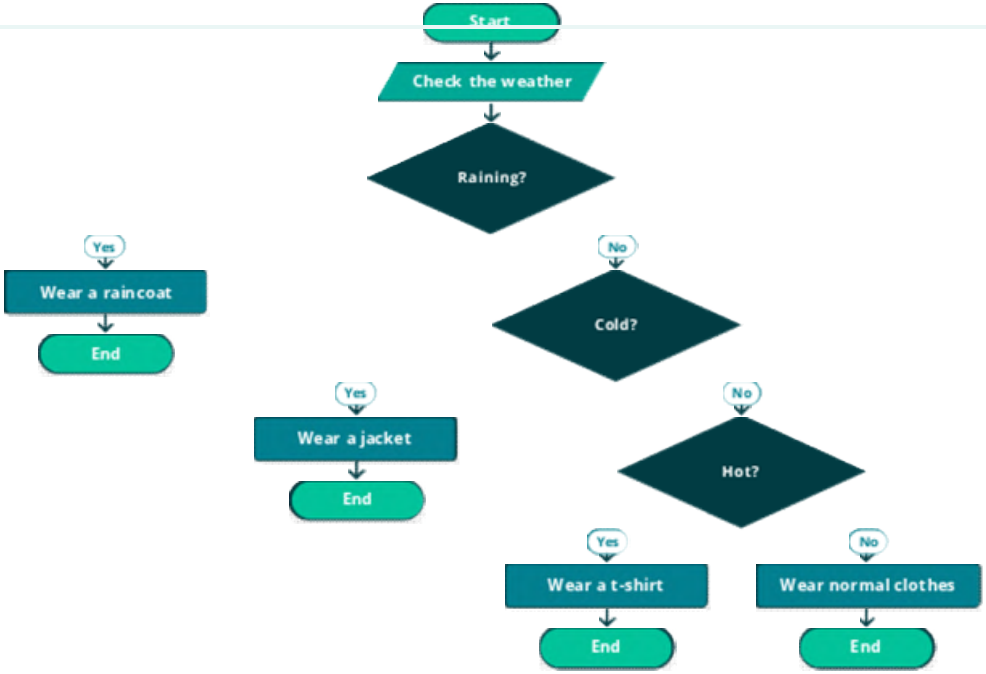
PROBLEM 2 IF-ELSE / DECISION MAKING

ATM Withdrawal



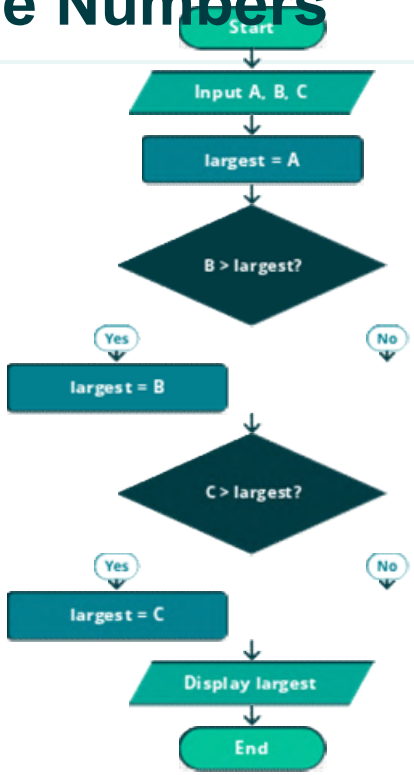
PROBLEM 3 MULTIPLE CONDITIONS

Choosing What to Wear



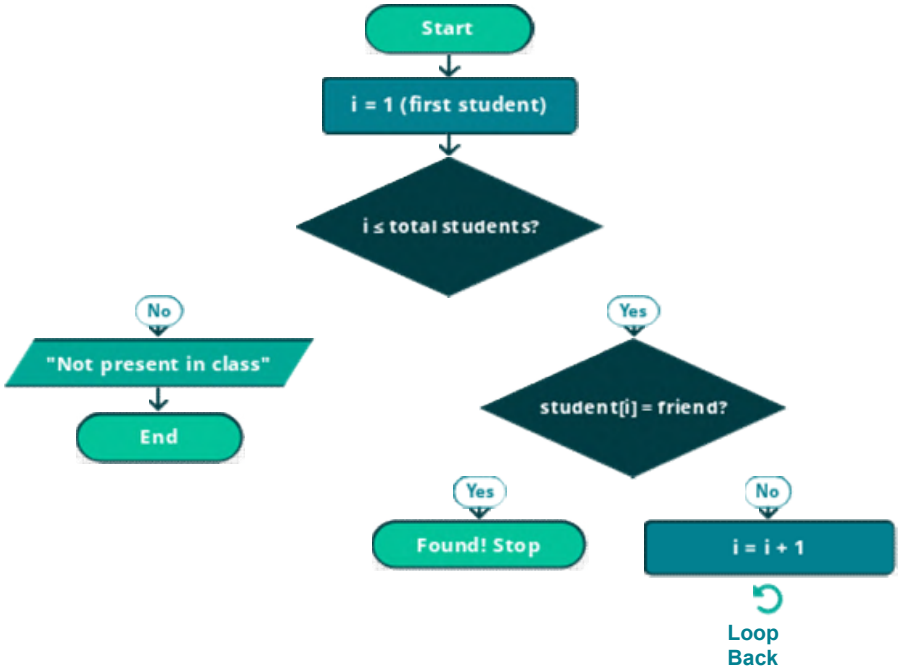
PROBLEM 4 COMPARISON / MAXIMUM

🔢 Finding Largest of Three Numbers



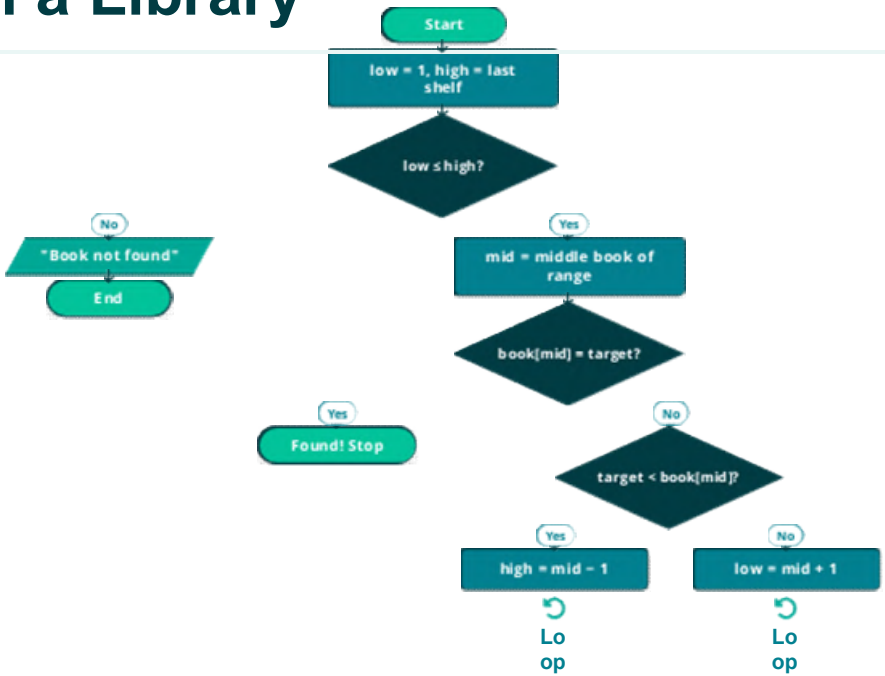
PROBLEM 5 LINEAR SEARCH

🧑‍🎓 Finding a Friend in a Classroom



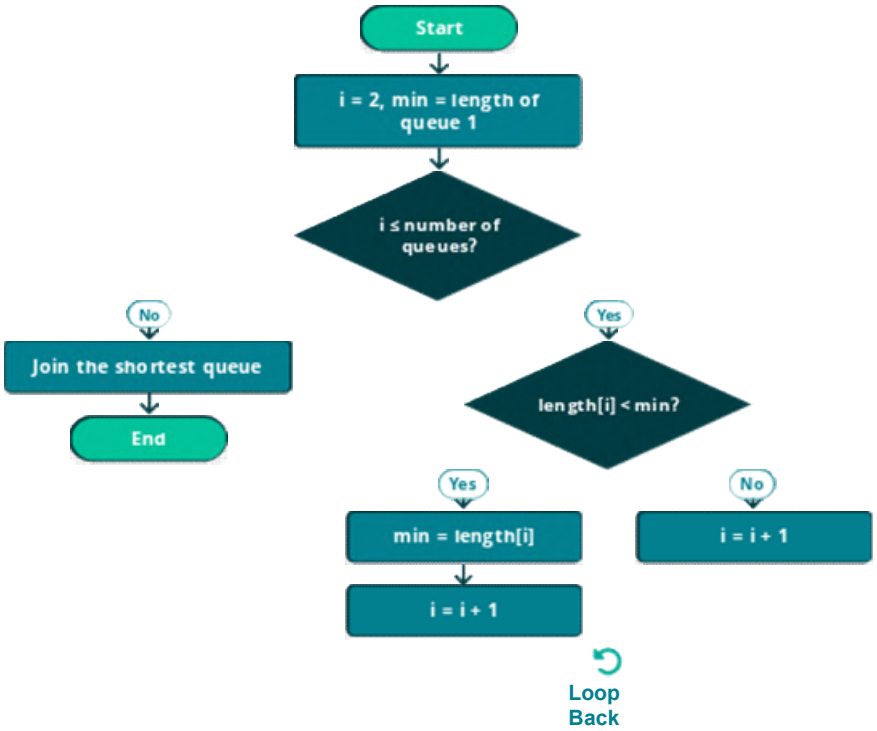
PROBLEM 6 BINARY SEARCH

Finding a Book in a Library



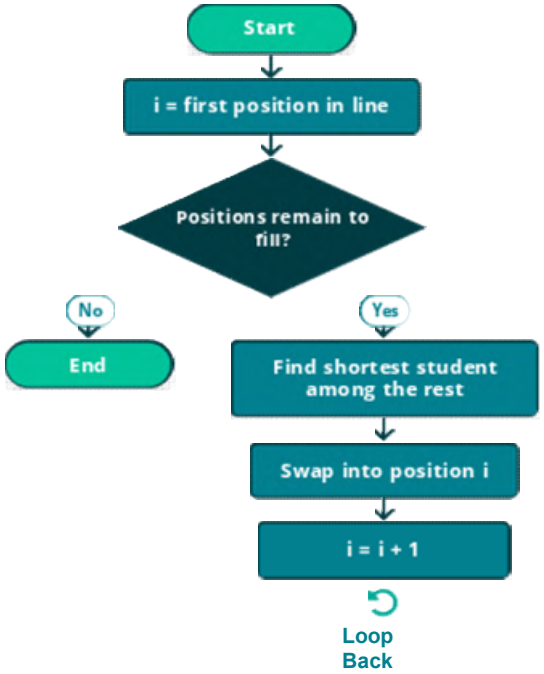
PROBLEM 7 MINIMUM / SELECTION

Choosing the Shortest Queue



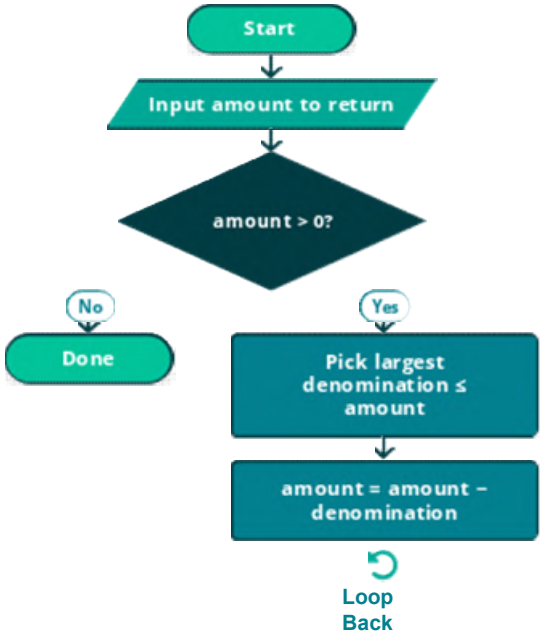
PROBLEM 8 SORTING / SELECTION SORT

Arranging Students by Height



PROBLEM 9 GREEDY ALGORITHM

💰 Making Change at a Shop



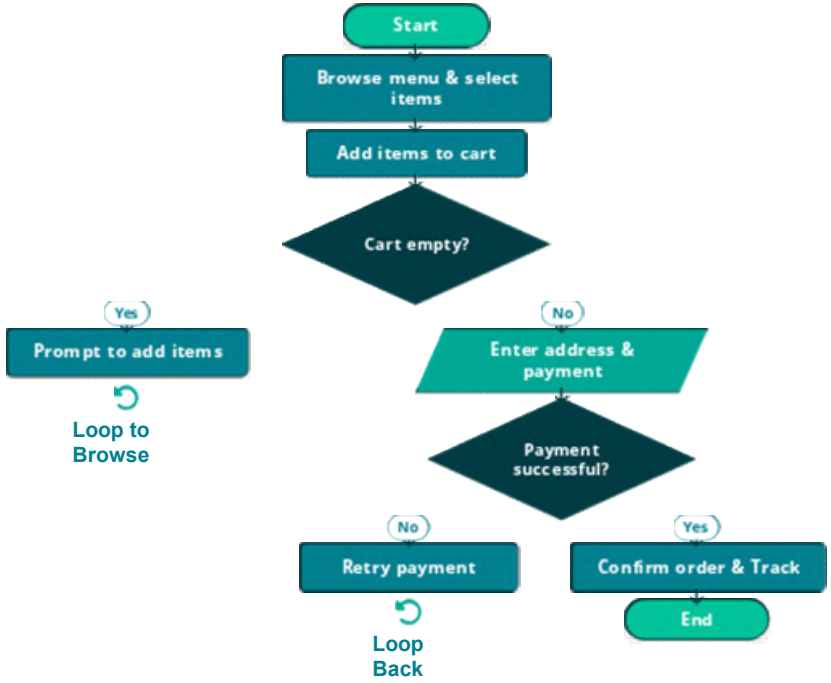
PROBLEM 10 SHORTEST PATH / GRAPH ALGORITHM

🗺️ Google Maps / Navigation



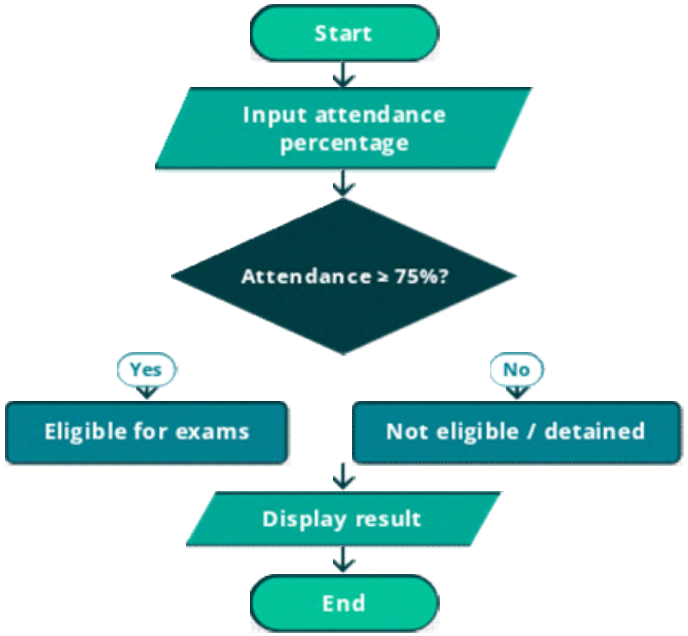
PROBLEM 11 SEQUENCE + CONDITIONS + STATES

Online Food Ordering



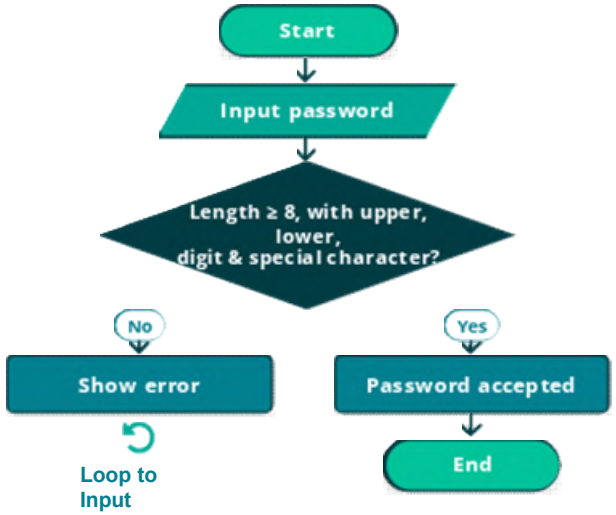
PROBLEM 12 CONDITIONAL / DECISION ALGORITHM

College Attendance Eligibility



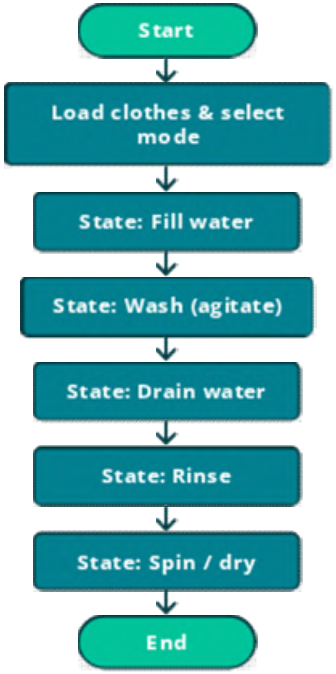
PROBLEM 13 BOOLEAN LOGIC / CONDITIONS

🔐 Password Validation



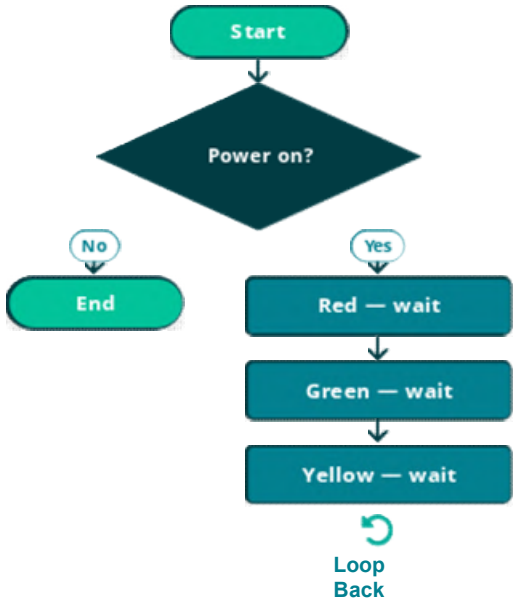
PROBLEM 14 STATE-BASED ALGORITHM

Washing Machine



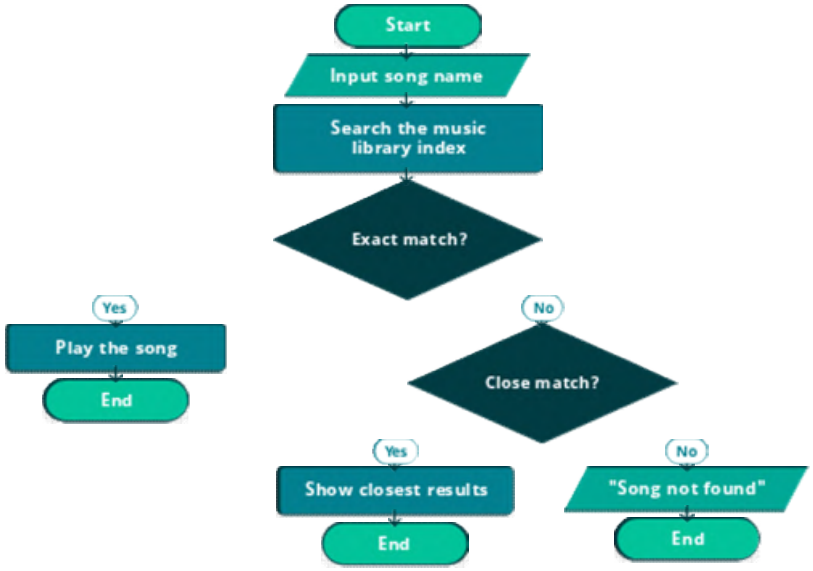
PROBLEM 15 LOOP / REPETITION

🚦 Traffic Signal



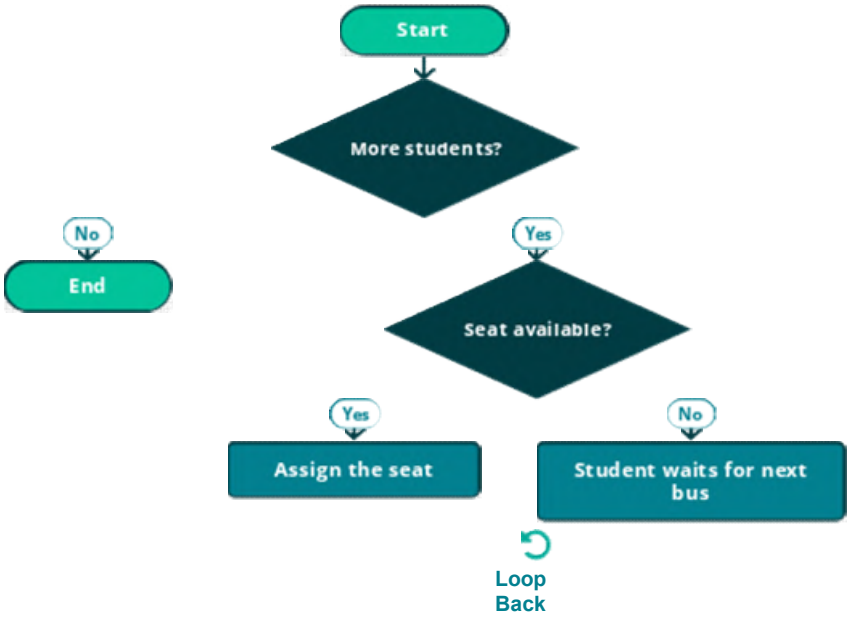
PROBLEM 16 SEARCHING / INDEXING

♪ Finding a Song on Your Phone



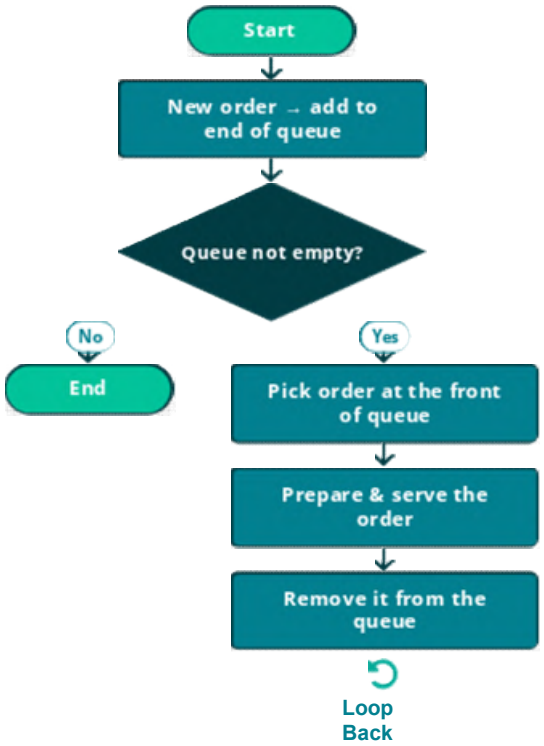
PROBLEM 17 ITERATION + ALLOCATION

College Bus Seating



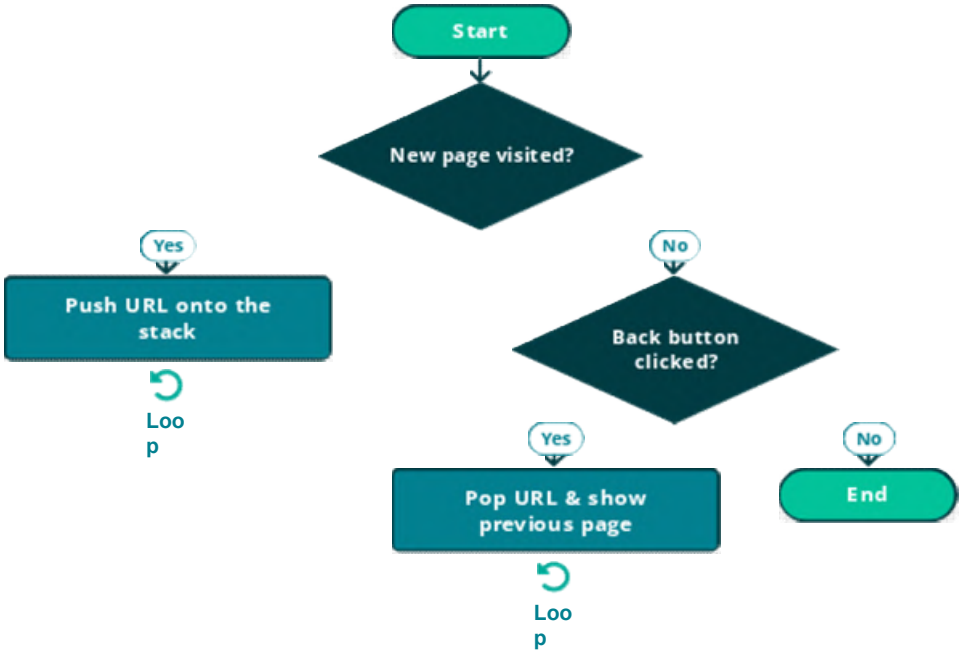
PROBLEM 18 QUEUE / FIFO

🍽️ Restaurant Order Queue



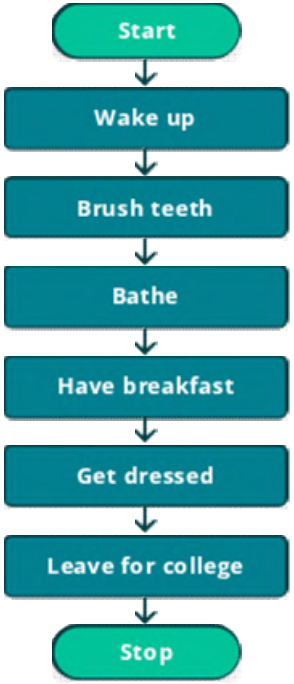
PROBLEM 19 STACK / LIFO

← BACK Browser Back Button



PROBLEM 20 SEQUENCE / ORDERED STEPS

Morning Routine



l • o o fl

p ;Y AV ;q U flff W off ff , Ue

U EC;I fl W n ff c ffi

A8s W k r

e ffi ff ff ff : fl ff fl fl ff fl ff fl ffi flff fl : fl fl ffi ffi

ff As • ffi fl ffi • ffiffffl fl• fl fl ffi fl ff fl fl : ff ffiff ffi fl•

flffi fl : fl fl fl• fl A

p fl fl ff : fl fl ff ff fl ffi • ; fl Ap fl ffi fl flffi fl
• fl ffi fl fl fl fl• : ff fffl• fl A

B8a n r r

U • ; fl fl fl fl ff fl flff ffi ff fl ff • Ap
fl flffi fl N

s 7a U Mb fl • ff fl ffi• fl ffi fl : ff ff ff A

V 7t U Mb fl fffl fl : fl fl ff fl ffiff • fl fl A

C8s s 7a U MJ m

p p ;Y fl flff .fl fl ; 0• ; ff ff fl fl Ap
ffiifl fl : fl • flff • Ap fl ff ff • ff flff fl • fl ffiffi ffi fl • A

p ff ffi ff ff Aa fl ff • • ffi • ; ffi : fl ffi • ; ffi fl •
ffi ff ff ff : ff fl • fl fl ffi ffi ff ffi ff ffi l A

D8r 7 7r Mb s 7a

n AMk a

Y fl fl • ff fl A
z mx
W

n BMr f

efi fl fl fl • ff Az my
Py P P
W

n CMk a

V fl • ff Az m, y
, , x , P, , P
, , W

n DMf

s ff ffi fl ff :
• fl ff ff fl A

E8s W k

> **r W**
>
> p ;Y ffi ff ffi fl AU ffi fl ;ff fl ffi ff ffi fl ff fl ff ff fl AV
> fl fl . A A fffl fl• fl fl fl qe ffi Q ffi flffi fl ffi fl ffi
> fl ffi fl fffl fl • fl ffi fl ffi fffl fl • fl A

F8s 7a b M i n

W ffi fl • ffi fl j fl fl hfl fl l ff .j hl 0 Aq fl ;ffi ff : ffi fl •
ff fl As fl fl N

- **k r Mj** hl o U fl
- **i Ak Mp** e → l ff → p fl i ffi e ff → n b fl
- **i Bk 2n 3Mp** fl → o ; ffin fl → r ff a • ffffi

k fl fl fl ff ffi flff fl ff ffi `Tokenization` A

H8r s 7a s

V fffl ; ff ff • ff ; ff fl ffi ffi r fl A
U • fl ffi ff fl fl • fl fl ffi fl fl • A

z *m*U • fl • ffffi ff fl fl fl fl ffi fl j hl fffl • ffi
A

I 8U s 7a a

W u

p fl ff fl ff ff fl fffl ffi ffifl A
V fffl fl ff ffi ffi : ff
fl fl fl fl ffi fl fl A

a

k ff fl ff fl ffi : fl flffi fffl
fl ffi • ffi ff ; ffi
ffi fl ffi fl fl ffi A

L8i s 7a a

a b Mp ; : ff fl• ff ffi fl Au fffl fl ffi
fl • A

i p MV fffl ffi fl ff ffi ffi ff ff ; fl : fl ff ffi
ff fl ffiffi ff fl fffl A

A: 8s V 7t U MJ m

p V ;q fl flff fl flffi Ae • ffi fl ffi fl fffl fl : fl fl ff
Ak ff ; ff fl • fl ffi ffi fl ffi fl ffi A

p ff ff fl ffi ff : fl ff fl Ae fl ff • ffi • ff A

AA8r 7 7r Mb V 7t

n AMV W

effi fl ffi• ffi ; Az m

W

n BMW

c • fl ff fl ffi Az m

x W

n CMf

h ffi fl • Az m

W

n DMs 7i U

s fl fl ff ff fl fffl

ff ffi • A

AB8p c n

s W n

V ; ffi ff fl ff fl ff: fl ff AV fffl fl ; ff •
; fl fffl fl ffi ff • ffAp ff fl fl • fl fl• fl fffl
• fl ffiflff ff A

AC8V 7t b Mk i c

e fl • ffi fl i flff h fl fl ff fl ff . fl fl p b l p ff 0AU ;ffi fl flff
ffi• fl fl Ae flffi • ; N

- **i AMV** ffi• fl ff fffl fl ff fl ffi fffl fl fl A
- **i BMV** • fl fffl fl ffiflff fl ff . AA `ReLU` : `Sigmoid` 0A
- **i CMV** • fl fffl ffihfl fl• flff . AA `DenseLayer` 0A
- **i DMb** fl : fl fl `Model` ff fl fl fl ffi fl fl A

AD8a V 7t s

s ;ffi %ø • % fl : • ; a AU ffi fl fl ff ff ffi
ff fffl fl fffl fl fl • ; ffi A

p fl fl fl fl ff DCC) fl fl fffl flffff fl • fl ffi fl fl
A

AE8U V 7t a

b s p

V fffl ff fl• ff ffi ffiifl fl
: ffi fffl : ffi ff
fl fffl A

i W

p ff ffi • fl fl fl fl ffi
ff • fl : ffi fl fffl flffff fl
ffi fl • ff A

AF8i V 7t a

f e MV fffl ff fl • fl : fl ff fl • flff ; Ae flff ff fl A

i u MY fffl ffi ff fl ; fl : fl fl fl fl ff ffiffi flff fl fl ff ff fl A

AH8a W Ms a

p ff ff • fl flffiffffl fl m fl U fl ff .mU0 ff N

s 7a Mq efl ffi fl fl ffi AW fl fl ffi ff fl ffi fl A	**V 7t MW** fl fl ffi ff fl ffi Aq efl ffi fl fl ffi fl A
s 7a V a Mh ff • fl ffi fl : ff fffl • ff A	**V 7t V a Me** fl • fl ffi fl : flff A

Al 8r r Mv t s 7a

p ;Y ffi• ff N

p fl ffi ff fl ffi flff fl ff fl ffi ffi • A

u fl • ffi fl ff fl ffifl fffl fl ffi ff ff fl A

u fl flff fl ff ffO ffi fl % ff %• ffi fl A

AL8r r Mv t V 7t

V ;q ffi• ff N

u fl • ffi ff ff : oYg : Ul e . fl flff fl fl fffl fl ff • fl 0A

p fl ff : ffi fl fl fffl fl fl • A

u fl • ffifl fl• ff ffi • fl fl• ff A

B: 8s k r Ms e (k 7m (U

e ff fl fl fl ffiflffi fl ff ffi fl ff : fl fl fl flff ff Ap ffi fl fffl ffi
ffi% ffffi ;k % %d • ff⁄ffi A

U ff ff fl fl ff ff .p ;Y 0 fl fl ff fl fl Ap : ffiifl ff
• ffi fl ffi ff fffl : ; fffl fl fl .V ;q 0Aa fl : ; fl ffi • ;
ffffi : fl fl • ;ffi fl ffi • A

INTRODUCTION TO C PROGRAMMING

C Tokens, Keywords & Identifiers

The building blocks of every C program

KEYWORDS | IDENTIFIERS | CONSTANTS | OPERATORS

LECTURE • CONCEPTS • EXAMPLES • PRACTICE

Learning outcomes

Define a C token and explain why tokens matter.

Classify the major categories of tokens in a C program.

Recognize C keywords and explain their role.

Create valid identifiers and identify naming errors.

Analyze a short C statement by separating its tokens.

What is a token?

A token is the smallest meaningful unit recognized by the C compiler during lexical analysis.

Think of words

A program is built from small units—much like a sentence is built from words.

Compiler view

The compiler groups characters into tokens before parsing the program.

Why it matters

Correct token spelling and separation help the compiler understand your code.

The main categories of C tokens

Keywords

Reserved words with predefined meaning.

int, if, return

Identifiers

Names created by the programmer.

main, total, count

Constants

Fixed values.

10, 3.14, 'A'

Strings

Character sequences in double quotes.

"Hello"

Operators & punctuators

Operations and structural symbols.

+, ==, ;, ()

Tokenization: read the statement

```
int total = price + 5;
```

int — keyword

total, price — identifiers

=, + — operators

5 — integer constant

; — punctuator (statement terminator)

Keywords: reserved words

Keywords are part of the C language vocabulary.

Each keyword has a specific meaning defined by the language.

A keyword cannot be used as an identifier.

C is case-sensitive: int is a keyword; Int is a different spelling.

Common C keywords: data and types

Basic types

char
int
float
double

Type modifiers

short
long
signed
unsigned

Type qualifiers

const
volatile

Other type words

void
struct
union
enum
typedef

Common C keywords: control flow

Selection

if
else
switch
case
default

Loops

for
while
do

Jump statements

break
continue
return
goto

Common C keywords: storage and functions

Storage-class keywords: auto, register, static, extern

Function-related keyword: return

Other commonly encountered keywords: sizeof, _Alignof (C11), _Atomic (C11)

The exact keyword set depends on the C standard being used.

Keyword examples in context

```
int main(void) {
    int score = 90;
    if (score >= 50) {
        return 1;
    } else {
        return 0;
    }
}
```

Keywords shown:

int • void
if • else
return

Identifiers: programmer-defined names

An identifier names an entity in a program, such as a variable, function, array, or structure.

Variables

```
int studentCount;
```

Functions

```
float calculateMean(void);
```

Arrays

```
int marks[5];
```

Structures

```
struct Student { ... };
```

Rules for valid identifiers

May contain letters (A–Z, a–z), digits (0–9), and underscore (_).

The first character must be a letter or underscore—not a digit.

No spaces or punctuation such as @, #, or hyphen.

A keyword cannot be used as an identifier.

Names are case-sensitive: total, Total, and TOTAL are different.

Valid vs. invalid identifiers

`student_name`	Valid	Letters + underscore
`count2`	Valid	Digit allowed after first character
`_index`	Valid	Begins with underscore*
`2count`	Invalid	Begins with a digit
`total-price`	Invalid	Hyphen is not allowed
`float`	Invalid	Reserved keyword

*Avoid leading underscores in ordinary application code; some forms are reserved to the implementation.

Naming style: make identifiers readable

Descriptive

Use totalMarks rather than x when the meaning is known.

Consistent

Choose one style and apply it across the program.

Meaningful

Use verbs for functions: calculateTotal().

Maintainable

Avoid cryptic abbreviations and unnecessary names.

Constants, strings, and punctuators

Constants

42
0.75
'A'

Represent fixed values.

String literals

"C programming"

Double quotes enclose text.

Punctuators

; , ( ) { } [ ]

Help structure declarations and statements.

Operators are tokens too

Arithmetic

+ - * / %

Relational

< <= > >=

Equality

== !=

Logical

&& || !

Assignment

= += -= *=

Whitespace and token boundaries

```
int sum = a + b;
int sum=a+b;
```

Both statements contain the same tokens.

Spaces and line breaks often improve readability.

Whitespace separates adjacent tokens when needed.

Whitespace inside a keyword or identifier changes its spelling.

Comments are ignored by the compiler during tokenization.

Guided practice: identify the tokens

```
float average = (a + b) / 2.0;
```

Which words are keywords?

Which names are identifiers?

Identify the operators and punctuators.

Which token is a floating-point constant?

Quick check: true or false?

1. A keyword can be used as a variable name.
2. count and Count are the same identifier.
3. An identifier may contain a digit after its first character.
4. The semicolon is a punctuator.
5. 3.14 is a string literal.

Key takeaways

Tokens are the smallest meaningful units of a C program.

The main token categories include keywords, identifiers, constants, string literals, operators, and punctuators.

Keywords are reserved; identifiers are chosen by the programmer.

Follow identifier rules and use meaningful names.

Practice by reading a statement and labeling each token.

Next: declarations, expressions, and how the compiler parses statements.

INTRODUCTION TO C PROGRAMMING

Variables & Constants

Storing, using, and protecting data in C

VARIABLES

CONSTANTS

DATA TYPES

CONCEPTS • C CODE • EXAMPLES • PRACTICE

Learning outcomes

Explain what variables and constants represent in a C program.

Declare and initialize variables using appropriate data types.

Distinguish assignment from initialization.

Use const-qualified objects and #define macros appropriately.

Recognize scope, lifetime, and common data-handling errors.

Why do programs need variables?

Programs process information. Variables give that information a name so it can be stored, updated, and used.

Input

A user's age or a sensor reading.

Processing

Intermediate values such as a total or average.

Output

A result that the program displays.

What is a variable?

A variable is a named object used to store a value.

Its type determines what kind of value it can represent and how it is handled.

A variable may be assigned a new value during program execution.

Example: int score = 85;

```
int score = 85;
score = 92;   // score now stores 92
```

Declaration, initialization, assignment

Declaration

int count;

Introduces a variable and its type.

Initialization

int count = 0;

Gives its initial value.

Assignment

count = 5;

Stores a new value after declaration.

Basic syntax

```
data_type identifier = initial_value;
```

data_type — such as int, char, float, or double

identifier — a valid variable name

= — assignment operator

; — ends the declaration statement

Choosing a data type

int

Whole numbers

int students = 45;

char

A character

char grade = 'A';

float

Single-precision real

float temp = 24.5f;

double

Double-precision real

double pi = 3.14159;

Using variables in expressions

```
int length = 8;
int width = 5;
int area = length * width;
printf("Area = %d", area);
```

Result:

Area = 40

The expression uses the values stored in length and width.

Updating a variable

```
int count = 0;
count = count + 1;
count += 1;
++count;
```

Each statement changes count.

The final value is 3.

+= and ++ are common shorthand forms.

Variable naming and good practice

Use meaningful names: totalMarks is clearer than tm.

Follow identifier rules: begin with a letter or underscore; then use letters, digits, or underscores.

C is case-sensitive: total and Total are different names.

Initialize variables before reading their values.

Avoid names that differ only by capitalization.

What is a constant?

A constant is a value intended to remain fixed while a program runs.

Literal constant

Examples: 10, 2.5, 'A'

Named constant

A meaningful name represents a fixed value.

Why use them?

Improve readability and reduce accidental changes.

Literal constants in C

Integer

25
-7
0

Floating-point

3.14
6.0f
1e3

Character

'A'
'7'
'\n'

String literal

"Hello"
"C language"

The const qualifier

```
const double TAX_RATE = 0.18;
const int MAX_STUDENTS = 60;
```

const makes the object non-modifiable through that identifier.

Initialize a const object when it is defined.

Attempting to assign a new value to it violates C constraints.

The name is still an object—not a preprocessor macro.

Using const in a calculation

```
const double PI = 3.141592653589793;
double radius = 2.0;
double area = PI * radius * radius;
printf("Area = %.2f", area);
```

Output:

Area = 12.57

The #define directive

```
#define MAX_TRIES 3
#define TAX_RATE 0.18

int attempts = 0;
```

A preprocessor directive defines a macro before compilation.

A simple object-like macro substitutes its replacement text.

Macros do not create typed C objects.

Use parentheses in function-like macro definitions when needed.

const vs. #define

const object

Has a C type and an object identity.

Example: const int limit = 10;

#define macro

Preprocessor text substitution; no declared object type.

Example: #define LIMIT 10

Scope and lifetime: a first look

Block scope

A local variable declared inside { } is generally visible only within that block.

File scope

A name declared outside functions has file scope.

Lifetime

Automatic local objects usually exist during block execution; static objects persist for the program's lifetime.

Common mistakes to avoid

Using a variable before giving it a valid value.

Assigning to a const-qualified object.

Choosing a type that cannot represent the needed value.

Confusing = (assignment) with == (equality comparison).

Forgetting the semicolon after a declaration.

Assuming #define creates a typed variable.

Practice: predict the output

```
int x = 4;
int y = 3;
x = x + y;
y = x * 2;
printf("%d %d", x, y);
```

Think about:

1. **What is x after line 3?**
2. **What is y after line 4?**
3. **What will printf display?**

Quick check & recap

What is the difference between declaration and initialization?

Why should variables be initialized before use?

How does const differ from #define?

Name two integer or floating-point data types.

Remember: choose meaningful names, appropriate types, and constants for fixed values.

Next topic: expressions, operators, and type conversion.

INTRODUCTION TO C PROGRAMMING

Data Types & Memory Allocation

How C represents data and reserves memory

DATA TYPES | SIZE & RANGE | MEMORY

CONCEPTS • CODE • MEMORY DIAGRAMS • PRACTICE

Learning outcomes

Classify fundamental, derived, and user-defined C data types.

Explain how a type influences representation, operations, and storage.

Use sizeof to determine object and type sizes on a given system.

Distinguish automatic, static, and dynamically allocated storage.

Apply malloc, calloc, realloc, and free safely at an introductory level.

Why data types matter

A data type tells C how to interpret a value and what operations are valid for it.

Meaning

Is the value a whole number, character, or real number?

Representation

How is the value represented in memory?

Operations

Which operations and conversions are appropriate?

C data type families

Fundamental

char, integer types, floating types, _Bool

Derived

Arrays, pointers, function types

User-defined forms

struct, union, enum; typedef creates an alias

Integer types

char, short, int, long, and long long represent integer values.

signed and unsigned variants affect the representable range.

Exact sizes are implementation-dependent, subject to C's minimum requirements.

Use a type that safely represents the values your program needs.

```
short year = 2026;
unsigned int count = 250;
long long population = 8000000000LL;
```

Floating-point types

float

Typically lower precision; often 4 bytes on common systems.

double

Typically higher precision; often 8 bytes.

long double

May provide extended precision; size and precision vary.

Character and Boolean types

```
char grade = 'A';
_Bool isReady = 1;
// In C99 and later:
#include <stdbool.h>
bool passed = true;
```

char stores an integer character code; its signedness is implementation-defined.

_Bool stores 0 or 1.

Type modifiers and qualifiers

Modifiers: signed, unsigned, short, and long.

Qualifiers: const, volatile, and (since C11) atomic-related forms.

Modifiers affect type properties; qualifiers affect how objects may be accessed.

Use standard headers and the language version supported by your compiler.

The sizeof operator

```
printf("int: %zu bytes\n", sizeof(int));
printf("double: %zu bytes\n", sizeof(double));
int values[10];
printf("array: %zu bytes\n", sizeof values);
```

sizeof produces a size_t value: print it with %zu.

It reports size in bytes—not a fixed number of bits.

For an array in the same scope, sizeof array gives the total array size.

Memory is byte-addressed

A byte is the unit measured by sizeof. C guarantees a byte is at least 8 bits.



Illustration only: a particular object may occupy multiple bytes.

A variable occupies storage

```
int score = 42;
char grade = 'A';
```

Illustrative memory layout



The actual byte count and addresses depend on the implementation.

Arrays: contiguous elements

```
int marks[4] = {70, 82, 91, 65};
printf("%zu\n", sizeof marks);
```

marks[0] = 70	marks[1] = 82	marks[2] = 91	marks[3] = 65

Array elements are stored contiguously; sizeof marks gives the full array size here.

Structures and unions

struct

Each member has its own storage; padding may be inserted for alignment.

union

Members share storage; the union is large enough for its largest member, with possible padding.

Storage duration: automatic and static

Automatic

Typical local variables exist while their block is executing.

Static

Objects declared static, and file-scope objects, have program-long storage duration.

Important

Scope is where a name is visible; storage duration is how long the object exists.

Dynamic memory: heap allocation

```
int *data = malloc(5 * sizeof *data);
if (data == NULL) {
    /* handle allocation failure */
}
/* use data[0] ... data[4] */
free(data);
data = NULL;
```

Include:

#include <stdlib.h>

Dynamic storage is requested at runtime and must be released when no longer needed.

malloc, calloc, realloc, free

malloc

Allocates a block; contents are uninitialized.

calloc

Allocates an array-like block and initializes all bytes to zero.

realloc

Resizes a block; use a temporary pointer to avoid losing the original on failure.

free

Releases a dynamically allocated block.

Safe memory allocation habits

Check whether malloc/calloc/realloc returned NULL.

Calculate allocation sizes carefully; guard against multiplication overflow.

Do not access memory outside the allocated object.

Free each allocated block exactly once when finished.

Do not use a pointer after free; set it to NULL when that helps prevent mistakes.

Common mistakes

Assuming int is always 4 bytes or long is always 8 bytes.

Using the wrong printf format specifier for a type.

Reading an uninitialized automatic variable.

Writing beyond an array's bounds.

Forgetting to free dynamically allocated memory—or freeing it twice.

Practice: reason about size

```
int a[6];
printf("%zu\n", sizeof a);
printf("%zu\n", sizeof a / sizeof a[0]);
```

What does the first expression measure?

What does the second expression calculate?

Why might the byte count differ between systems?

Key takeaways

Data types define interpretation and constrain valid operations.

Use sizeof to inspect sizes on the system where the program runs.

C object sizes and memory layouts can vary by implementation.

Understand scope, storage duration, and dynamic allocation as distinct concepts.

Allocate carefully, check results, and release dynamic memory responsibly.

Next topic: operators, expressions, and type conversion.

INTRODUCTION TO C PROGRAMMING

Arithmetic Operators & Example Programs

Performing calculations in C—clearly and correctly

OPERATORS

EXPRESSIONS

C PROGRAMS

CONCEPTS • WORKED EXAMPLES • OUTPUTS • PRACTICE

Learning outcomes

Identify C arithmetic operators and their purposes.

Evaluate arithmetic expressions using precedence and associativity.

Explain integer division, remainder, and type conversion.

Use unary plus/minus and increment/decrement safely.

Write, trace, and test small arithmetic programs.

What is an arithmetic operator?

An arithmetic operator performs a mathematical operation on one or more operands.

Operands

Values or variables used in an expression.

Operator

The symbol that specifies the operation.

Expression

A combination that produces a value.

Arithmetic operators at a glance

+ Addition

Adds two operands.

8 + 3 → 11

- Subtraction

Subtracts right from left.

8 - 3 → 5

* Multiplication

Multiplies operands.

8 * 3 → 24

/ Division

Divides left by right.

8 / 3 → 2 (integers)

% Remainder

Remainder for integer operands.

8 % 3 → 2

Addition and subtraction

```
int a = 15, b = 6;
printf("Sum = %d\n", a + b);
printf("Difference = %d\n", a - b);
```

Output:
Sum = 21
Difference = 9

The + and - operators are binary here: each uses two operands.

Multiplication and division

```
int a = 17, b = 5;
printf("Product = %d\n", a * b);
printf("Quotient = %d\n", a / b);
```

Output:
Product = 85
Quotient = 3

With integer operands, / discards the fractional part (truncates toward zero).

Remainder operator: %

```
int quotient = 17 / 5;   // 3
int remainder = 17 % 5;  // 2
```

% works with integer operands, not floating-point operands.

For positive integers: dividend = divisor × quotient + remainder.

The divisor must not be zero.

Integer division vs. floating-point division

Integer operands

7 / 2 → 3

The fractional part is discarded.

Floating operand

7.0 / 2 → 3.5

Usual arithmetic conversions apply.

Force real division

(double)7 / 2 → 3.5

Cast before dividing.

Unary plus and unary minus

```
int n = 8;
printf("+n = %d\n", +n);
printf("-n = %d\n", -n);
```

Output:
+n = 8
-n = -8

Unary + leaves the value unchanged; unary - produces its negation (subject to type range).

Increment and decrement

++x

Pre-increment: increment x, then yield its new value.

x++

Post-increment: yield the old value, then increment x.

--x / x--

Decrement by one; prefix and postfix differ in value yielded.

Increment: trace the result

```
int x = 4;
int a = ++x;  // x=5, a=5
int b = x++;  // b=5, then x=6
printf("%d %d %d", a, b, x);
```

Output:
5 5 6

Compound assignment

```
int total = 10;
total += 5;  // total = total + 5
total -= 2;  // total = total - 2
total *= 3;  // total = total * 3
total /= 2;  // integer division
```

Values:
15 → 13
→ 39 → 19

Operator precedence

Parentheses ( ) group expressions and make intent clear.

Unary operators bind more tightly than * and /.

* / % are evaluated before + and -.

Operators of the same arithmetic level generally associate left to right.

Use parentheses instead of relying on memory.

Precedence: worked example

```
int result = 4 + 3 * 2;
// 3 * 2 first, then +
// result = 10

int grouped = (4 + 3) * 2;
// grouped = 14
```

Remember:
4 + 6 = 10

7 × 2 = 14

Example 1: sum and average

```c
#include <stdio.h>
int main(void) {
    int a = 12, b = 18, c = 15;
    int sum = a + b + c;
    double average = sum / 3.0;
    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);
    return 0;
}
```

Output:
Sum = 45
Average = 15.00

Example 2: area and perimeter

```c
#include <stdio.h>
int main(void) {
    double length = 8.5, width = 4.0;
    double area = length * width;
    double perimeter = 2 * (length + width);
    printf("Area = %.2f\n", area);
    printf("Perimeter = %.2f\n", perimeter);
    return 0;
}
```

Output:
Area = 34.00
Perimeter = 25.00

Example 3: convert minutes

```c
#include <stdio.h>
int main(void) {
    int totalMinutes = 135;
    int hours = totalMinutes / 60;
    int minutes = totalMinutes % 60;
    printf("%d hours, %d minutes\n", hours, minutes);
    return 0;
}
```

Output:
2 hours,
15 minutes

Example 4: swap using arithmetic?

A temporary variable is the clearest and safest general method.

Arithmetic swap can overflow for signed integers; avoid it in ordinary code.

Prefer this readable approach:

```
int temp = a;
a = b;
b = temp;
```

Common errors and safe habits

Dividing by zero: undefined behavior; validate the divisor.

Expecting 7 / 2 to produce 3.5: use floating-point arithmetic when needed.

Using % with floating-point operands: not allowed in C.

Confusing = with ==; they have different purposes.

Avoid multiple modifications of the same variable in one expression.

Practice: predict the output

```
int a = 9, b = 4;
printf("%d\n", a / b);
printf("%d\n", a % b);
printf("%d\n", a + b * 2);
```

Try it first:

What are the three lines?

Recap: arithmetic in C

Know the operators: +, -, *, /, %, unary +/-, ++, --.

Integer division truncates; floating-point division can retain fractions.

Precedence controls grouping; parentheses improve clarity.

Trace prefix/postfix operations carefully.

Test programs with ordinary, boundary, and invalid inputs.

Next topic: relational and logical operators.

INTRODUCTION TO C PROGRAMMING

Relational & Logical Operators

Making comparisons and combining conditions in C

COMPARE

COMBINE

DECIDE

CONCEPTS • TRUTH TABLES • C PROGRAMS • PRACTICE

Learning outcomes

Identify relational and logical operators in C.

Evaluate comparisons and interpret their results.

Use &&, ||, and ! to build compound conditions.

Explain short-circuit evaluation and operator precedence.

Write and trace decision-making programs using conditions.

Why do we need these operators?

Programs often need to compare values and decide whether a condition is true or false.

Compare

Is a mark at least 40?

Combine

Is the mark valid and attendance sufficient?

Decide

Should the program display “Pass”?

Relational operators

<

Less than

5 < 8 → true

>

Greater than

9 > 4 → true

<=

Less than or equal to

4 <= 4 → true

>=

Greater than or equal to

7 >= 9 → false

Equality operators: == and !=

== Equal to

Checks whether two values are equal.

5 == 5 → true

!= Not equal to

Checks whether two values differ.

5 != 3 → true

Remember: = assigns a value; == compares two values.

Relational expressions return an integer result

In C, a relational or equality expression evaluates to 1 when true and 0 when false.

These results can be stored in an int variable or used directly in a condition.

Example: int result = (10 > 3); stores 1.

```
int result = (10 > 3);
printf("%d", result);  // prints 1
```

Relational operators: worked examples

```
int a = 10, b = 6;
printf("%d\n", a > b);   // 1
printf("%d\n", a == b);  // 0
printf("%d\n", a != b);  // 1
printf("%d\n", a <= 10); // 1
```

Output:

1
0
1
1

Logical operators

&& AND

True only when both operands are true.

|| OR

True when at least one operand is true.

! NOT

Reverses the truth value.

Truth table: logical AND (&&)

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

AND is true only when both conditions are true.

Truth table: logical OR (||)

A	B	A || B
0	0	0
0	1	1
1	0	1
1	1	1

OR is false only when both conditions are false.

Truth table: logical NOT (!)

A	!A
0	1
1	0

NOT reverses a condition: true becomes false, and false becomes true.

Combining conditions

```
int age = 20;
int hasID = 1;
if (age >= 18 && hasID) {
    printf("Entry allowed");
}
```

Both conditions must be true:

age >= 18
hasID

Short-circuit evaluation

For A && B, if A is false, B is not evaluated.

For A || B, if A is true, B is not evaluated.

This can prevent unnecessary work and protect expressions from unsafe evaluation.

Do not rely on a skipped expression to perform an essential side effect.

```
if (denominator != 0 && numerator / denominator > 2) {
    printf("Condition met");
}
```

Operator precedence

Arithmetic operators are evaluated before relational comparisons.

Relational operators bind more tightly than equality operators.

Logical NOT (!) binds more tightly than &&, which binds more tightly than ||.

Use parentheses to make compound conditions clear.

```
a + 2 > b && ready || force
```

Example 1: even or odd

```c
#include <stdio.h>
int main(void) {
    int n = 17;
    if (n % 2 == 0)
        printf("Even\n");
    else
        printf("Odd\n");
    return 0;
}
```

Output:
Odd

Example 2: pass or fail

```
#include <stdio.h>
int main(void) {
    int marks = 72;
    int attendance = 85;
    if (marks >= 40 && attendance >= 75)
        printf("Pass\n");
    else
        printf("Fail\n");
    return 0;
}
```

Output:
Pass

Example 3: largest of two numbers

```
#include <stdio.h>
int main(void) {
    int a = 24, b = 31;
    if (a > b)
        printf("a is larger\n");
    else if (b > a)
        printf("b is larger\n");
    else
        printf("Both are equal\n");
    return 0;
}
```

Output:
b is larger

Example 4: range check

```
#include <stdio.h>
int main(void) {
    int score = 86;
    if (score >= 80 && score <= 100)
        printf("Grade A\n");
    else
        printf("Outside A range\n");
    return 0;
}
```

Output:
Grade A

Common mistakes

Writing = instead of == in a comparison.

Using & or | instead of logical && or || when combining conditions.

Writing a mathematical range as 40 <= score <= 100; use score >= 40 && score <= 100.

Forgetting that 0 is false and any nonzero scalar value is true in a condition.

Ignoring precedence or relying on unclear compound expressions.

Practice: predict the output

```
int x = 5, y = 10;
printf("%d\n", x < y && y < 20);
printf("%d\n", x == 5 || y == 3);
printf("%d\n", !(x > y));
```

Try first:

What are the three results?

Recap: compare, combine, decide

Relational operators compare values; equality operators test sameness or difference.

Logical operators combine or reverse conditions.

C comparison and logical expressions produce 0 or 1.

Short-circuit evaluation can skip the right operand.

Use parentheses and test boundary cases in decision-making programs.

Next topic: if, if–else, and nested decision statements.

Type Conversions in C

CHANGE

CONVERT

CALCULATE

Understanding how C converts values between different data types.

CONCEPTS • IMPLICIT • EXPLICIT • EXAMPLES • PRACTICE

Learning outcomes

UNDERSTAND

Identify implicit and explicit type conversion.

TRACE

Predict the type and value after conversion.

CAST

Use (data_type) correctly in expressions.

APPLY

Recognize data loss and common mistakes.

TYPE CONVERSIONS • LEARNING OUTCOMES

Why do we need type conversion?

COMPARE

Different variables can have different data types.

CALCULATE

Expressions may combine int, float, and double values.

DIVIDE

Integer division may need conversion for a decimal answer.

CONTROL

A cast lets the programmer choose the type used in an expression.

CONVERSION HELPS C HANDLE VALUES OF DIFFERENT TYPES

Common data types

char

Character values such as 'A'.

int

Whole-number values.

float

Floating-point values with fractional parts.

double

Floating-point values with greater precision than float.

SIMPLIFIED VIEW: char → int → float → double

Implicit type conversion

Implicit conversion is **automatic**. The compiler performs the conversion without a cast written by the programmer.

```
int marks = 75;
double result = marks;
printf("%.1f", result);
```

WHAT HAPPENS?

The integer value 75 is converted to a double value for the assignment.

IMPLICIT = AUTOMATIC CONVERSION

Implicit conversion: worked example

```
int a = 10;
float b = 2.5;
float result = a + b;
printf("%.2f", result);
```

OUTPUT

12.50

The integer participates in floating-point arithmetic.

MIXED TYPES → CONVERSION → CALCULATION

Widening conversion

IDEA

A value moves to a type that can generally represent a broader range or fractional information.

EXAMPLE

int x = 25; double y = x;

RESULT

The value is represented as 25.0 when used as a double.

WIDENING • GENERALLY LOWER RISK OF INFORMATION LOSS

Narrowing conversion

IDEA

A value moves to a type with less range or precision.

EXAMPLE

double price = 99.99; int amount = price;

RESULT

amount becomes 99; the fractional part is discarded.

NARROWING • CHECK FOR INFORMATION LOSS

Data loss during conversion

FRACTION

double → int can discard the fractional part.

PRECISION

A conversion can reduce the precision of a numeric value.

RANGE

A destination type may not represent a large source value correctly.

CONVERSION DOES NOT GUARANTEE THAT INFORMATION IS PRESERVED

Integer division

```
int a = 5, b = 2;
printf("%d", a / b);
```

OUTPUT

2

Both operands are integers, so the division is integer division. The fractional part is not retained.

5 / 2 → 2 WHEN BOTH OPERANDS ARE int

Floating-point division using a cast

```
int a = 5, b = 2;
float result;
result = (float)a / b;
printf("%.2f", result);
```

OUTPUT

2.50

CAST BEFORE THE DIVISION WHEN A DECIMAL RESULT IS REQUIRED

Explicit type conversion

Explicit conversion is also called **type casting**.
The programmer specifies the desired type.



SYNTAX: (data_type) expression

Casting: simple example

```
int x = 10;
int y = 4;
float result;
result = (float)x / y;
printf("%.2f", result);
```

OUTPUT

2.50

The cast affects this expression; x remains declared as int.

EXPLICIT CASTING GIVES CONTROL OVER THE EXPRESSION

Conversion in assignment

WIDENING

int n = 25; double d = n; d stores 25.0.

NARROWING

double x = 15.8; int n = x; n becomes 15.

ASSIGNMENT CAN TRIGGER IMPLICIT CONVERSION

Character and integer conversion

```
char ch = 'A';
int n = ch;
printf("%d", n);
```

KEY IDEA

A character has an integer character code. The numeric value depends on the execution character set; ASCII is common.

CHARACTER ↔ INTEGER REPRESENTATION

Mixed-type expressions

```
int a = 10;
double b = 3.5;
double result = a + b;
```

WHAT HAPPENS?

C applies conversion rules so the operands can participate in the same arithmetic operation.

MIXED OPERANDS → CONVERSION RULES → ARITHMETIC

A common casting mistake

WRONG

(float)(5 / 2) 5 / 2 happens first → 2 Then 2 becomes 2.0

RIGHT

(float)5 / 2 5 becomes 5.0 first Then 5.0 / 2 → 2.5

CAST BEFORE INTEGER DIVISION — NOT AFTER

Common mistakes

- Using 5 / 2 when a decimal answer is expected.
- Casting after integer division instead of before it.
- Converting double to int without considering the discarded fraction.
- Ignoring range and precision when choosing a destination type.
- Assuming a cast changes the declared type of the original variable.

CHECK THE TYPE USED AT EACH STEP

Practice: predict the result

```
int a = 7, b = 2;
float x = a / b;
float y = (float)a / b;
printf("%.1f\n", x);
printf("%.1f\n", y);
```

TRY FIRST

What are the two outputs? Why are they different?

PREDICT → TRACE → THEN RUN THE PROGRAM

Recap: convert, calculate, control

IMPLICIT

Automatic conversion performed by the compiler.

EXPLICIT

Use (data_type) to request a conversion.

WATCH

Narrowing conversions may lose information.

NEXT

if, if–else, and nested decision statements.

TYPE CONVERSIONS • RECAP • NEXT TOPIC

01 Type Conversion in C

Understand what changes, when it changes, and why the output changes.

TYPE CONVERSION = CHANGE OF DATA TYPE

When C uses a value in a different numeric type, the language may convert that value automatically. A programmer can also request a conversion explicitly using a cast.

IMPLICIT	EXPLICIT	PROMOTION	CASTING	PRECISION

■ **By the end, you should be able to predict the result of mixed-type expressions before running the program.**

02 Learning Objectives

What you should be able to explain after this lesson

- Define type conversion and explain why it is required in C.
- Differentiate implicit conversion from explicit type casting.
- Explain how C handles mixed-type arithmetic expressions.
- Predict integer and floating-point division results.
- Identify truncation, precision loss, range issues, and signedness concerns.

■ **Don't just memorize the output — explain WHY the output occurs.**

03 What Exactly Is Type Conversion?

A value is used in another data type for an operation or assignment.

int 25 ■■■■■■■■■■■■→ double 25.0

A C expression may contain operands of different types. Before an operation is performed, C may convert one or more operands so that the operation can be carried out using compatible types.

The declared type of the original variable does not necessarily change. A converted value is produced for that particular context.

```
int x = 25;
double y = x;

// x is still int
// y stores 25.0
```

■ **Converting a VALUE is not the same as changing the declared TYPE of a variable.**

04 Why Is Conversion Necessary?

Different types have different representation and arithmetic behavior.

NUMERIC REPRESENTATION	int stores whole-number values; float and double support fractional values.	int → double

- Mixed-type expressions need compatible operands before an operation can be performed.
- Assignment may require the right-hand value to be converted to the destination type.
- Programmers may intentionally convert a type to control the precision of a calculation.
- Type conversion is therefore part of normal expression evaluation in C.

■ **Example: marks / number_of_subjects should normally produce a decimal average.**

05 Conversion Categories

Two common ways to think about conversion

IMPLICIT / AUTOMATIC	The compiler performs the conversion according to C's rules.	int → double
EXPLICIT / CASTING	The programmer requests the conversion using a cast.	(int)9.8

Another useful distinction is widening versus narrowing. Widening generally moves toward a type capable of representing a broader range; narrowing may lose information.

■ **Conversion can occur in expressions, assignments, function calls, and other contexts defined by C.**

06 Implicit Type Conversion

The compiler performs the conversion automatically.

Implicit conversion occurs when C's rules require an operand or value to be converted to a compatible type.

```
int age = 20;
double d;
d = age;

printf("%.1f", d);
// 20.0
```

20 (int) → 20.0 (double)

- No cast is written by the programmer.
- The assignment context requires the integer value to be represented as double.

■ **Think: “C decides the conversion because the context requires it.”**

07 Integer → Floating Point

A common conversion in calculations involving decimals

75 (int) ■■■■■■■■■→ 75.0 (float)

An integer can generally be represented as a floating-point value. The converted value can then participate in floating-point arithmetic.

```
int marks = 75;
float x = marks;

printf("%.2f", x);
// 75.00
```

- marks remains an int.
- x is a float containing the converted value.

■ **This is a typical widening-style conversion.**

08 Floating Point → Integer

A narrowing conversion can discard information

When a finite floating-point value is converted to an integer type, the fractional part is discarded by truncation toward zero, provided the result is representable.

12.89 → 12	Fraction .89 is discarded.
-12.89 → -12	Truncation is toward zero.

```
float price = 12.89;
int rupees = (int)price;

// rupees = 12
```

■ **Casting is NOT rounding. (int)12.9 gives 12, not 13.**

09 Integer Division: The Big Trap

The types of the operands determine how / is evaluated.

```
int a = 5;
int b = 2;
float result = a / b;

printf("%.2f", result);
// 2.00
```

5 / 2 → integer division → 2
2 → assignment to float → 2.0

Because both operands are int, C performs integer division. The fractional part is discarded before the result is assigned to result.

■ **A float variable on the LEFT does not automatically make a / b floating-point division.**

10 Getting 2.5 Correctly

Convert an operand before division occurs

```
int a = 5;
int b = 2;
float result = (float)a / b;

// 5.0 / 2 → 2.5
```

CAST FIRST → OPERATE SECOND

(float)5 / 2 → 5.0 / 2 → 2.5

The cast changes the type of a in this expression. One floating-point operand is enough to make the division floating-point.

- The original variable a is still an int.

■ **Rule: if you need a fractional division result, make at least one operand floating point BEFORE / is evaluated.**

11 Explicit Type Casting

The programmer specifies the target type

The general syntax is: (type-name) expression. The expression is converted to the specified type for that use.

```
float x = 9.8;
int y = (int)x;

printf("%d", y);
// 9
```

(type) expression

(float)a
(int)price
(double)total

- The cast does not permanently change x.
- (int)x creates an integer value from x.

■ **Use a cast when the intended conversion is clear and possible information loss is acceptable.**

12 Where You Put the Cast Matters

The position of the cast can change the result

```
int a = 7, b = 2;

float x = (float)a / b;
float y = (float)(a / b);
```

x = 3.5	Cast before division.	7.0 / 2
y = 3.0	Division before cast.	7 / 2 → 3

(float)a / b → 7.0 / 2 → 3.5
(float)(a / b) → 7 / 2 → 3 → 3.0

■ **Parentheses determine which expression is converted.**

13 Assignment Conversion

The destination type matters during assignment

```
int a = 25.9; // 25
float b = 10; // 10.0
double c = 7.5f; // 7.5
```

During assignment, the value produced by the right-hand expression is converted to the type required by the left-hand object.

- Floating → integer may discard the fractional part.
- Integer → floating generally preserves the numerical value, subject to the target type's representation.
- Compiler warnings can reveal suspicious or unintended conversions.

■ **Always look at BOTH sides of an assignment: source type and destination type.**

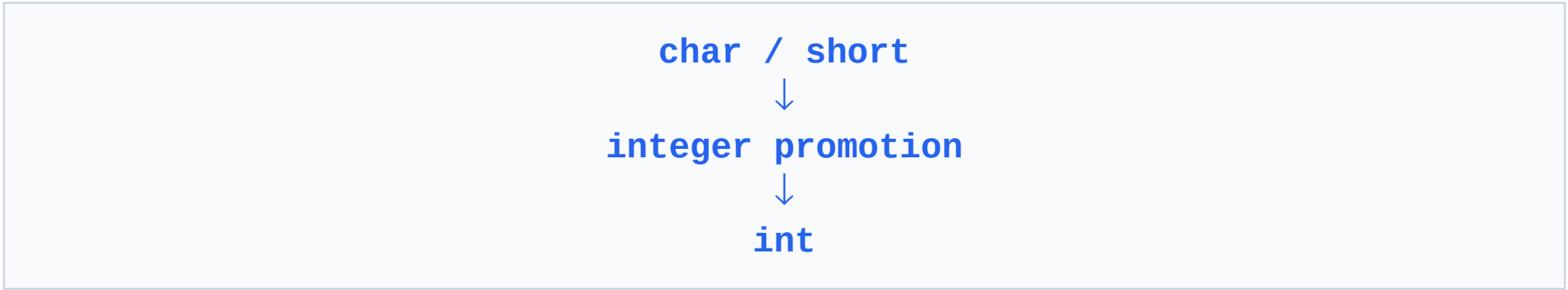
14 Integer Promotions

Small integer types often become int during arithmetic

C applies integer promotions to certain integer types, especially char and short, before many arithmetic operations.

```
char c = 10;
short s = 20;

int result = c + s;
// result = 30
```



- On typical implementations, char and short are promoted to int when int can represent all their values.

■ **Promotion helps C perform arithmetic using a standard integer type.**

15 Usual Arithmetic Conversions

C makes mixed arithmetic operands compatible

int + float → floating-point calculation
int + double → double calculation
float + double → double calculation

```
int i = 10;
float f = 2.5f;
float r = i + f;
// r = 12.5
```

The complete standard rules are detailed: integer promotions happen first, followed by conversions based on type rank and signedness.

- For beginners, the key idea is: C converts operands so the arithmetic operation can be performed consistently.

■ **Always identify the operand types before predicting the result.**

16 Character and Integer Conversion

char is an integer type in C

A char object stores a character representation, but char is also an integer type. Character constants such as 'A' have integer values in the execution character set.

```
char ch = 'A';
int n = ch;

printf("%d", n);
// Common ASCII result: 65
```

'A'	Character representation.
65	Common ASCII integer value.

ASCII is widely used, but portable C code should not assume every implementation uses ASCII.

■ **This is why character arithmetic such as ch + 1 is possible in C.**

17 Precision, Range & Information Loss

Conversions can change or discard information

PRECISION LOSS	A target type may not represent every value exactly.	Floating-point representation
TRUNCATION	Floating → integer removes the fractional part.	12.99 → 12
RANGE ISSUE	The destination type may not represent the source value.	Very large values need care
SIGNEDNESS	Signed and unsigned types can interpret values differently.	Check the destination type

■ **Good practice: choose a destination type that can safely represent the values you expect.**

18 Mixed Expression Walkthrough

Trace a complex expression from the inside out

```
int a = 10;
float b = 2.5f;
double c = 2.0;

double result = a + b * c;
```

Step 1 — b * c combines float and double, so C converts operands for compatible floating-point arithmetic.

Step 2 — a is then combined with the floating-point result, so a is converted as required.

Step 3 — the final value is stored in the double variable result.

10 + (2.5 × 2.0)

↓

10 + 5.0

↓

15.0

■ **Teaching/debugging technique: break a long expression into smaller steps and inspect each intermediate result.**

19 Common Mistakes

Five mistakes to check before submitting a program

- ■ 5 / 2 is assumed to be 2.5 even though both operands are int.
- ■ (int)12.9 is assumed to round to 13.
- ■ A float variable on the left is expected to change integer division.
- ■ The cast is placed after the operation instead of before it.
- ■ Precision, range, or signedness issues are ignored.

■ **Debugging tip: print intermediate values and identify the type of every operand.**

20 Quick Quiz + Practice

Pause → predict → explain → run

```
int a = 5, b = 2;

printf("%d\n", a / b);
printf("%.1f\n", (float)a / b);
printf("%.1f\n", (float)(a / b));
printf("%d\n", (int)8.9);
```

Predict the outputs:
? ? ? ?

■ **Answer: 2 | 2.5 | 2.0 | 8**

Practice: Read two integers and print their average correctly as a floating-point value.

■ **Challenge: Explain why (float)(a / b) is different from (float)a / b.**

21 Final Summary

The rules students should remember

1. IMPLICIT	Compiler performs conversion automatically.	int → double
2. EXPLICIT	Programmer requests conversion using a cast.	(float)a
3. INTEGER DIVISION	int / int produces an integer result.	5 / 2 → 2
4. CAST BEFORE OPERATION	Convert before / when decimals are needed.	(float)5 / 2 → 2.5
5. WATCH INFORMATION LOSS	Truncation, precision, range and signedness matter.	12.9 → 12

UNDERSTAND THE TYPE → TRACE THE OPERATION → PREDICT THE OUTPUT

01 Formatted Input & Output in C

How C programs read data from users and display results clearly.

INPUT → PROCESS → OUTPUT

Formatted I/O allows a C program to accept values in a specified format and display values using controlled formatting.

printf()	scanf()	FORMAT SPECIFIERS	ESCAPE SEQUENCES	FIELD WIDTH

■ **Core idea: the format string tells C what kind of data is being read or displayed and how it should be represented.**

02 Learning Objectives

What students should understand after this lesson

- Explain the purpose of formatted input and formatted output.
- Use printf() to display integers, floating-point values, characters, and strings.
- Use scanf() to read different types of data.
- Select the correct format specifier for each data type.
- Use width, precision, flags, and escape sequences in output.
- Avoid common scanf() and printf() mistakes.

03 What is Formatted I/O?

Input and output are controlled using a format string.

USER INPUT → scanf() → VARIABLES
VARIABLES → printf() → SCREEN

Formatted input means that the program specifies the expected type and arrangement of input data.

Formatted output means that the program controls how values are converted to text and displayed.

```
#include <stdio.h>

int age;
scanf("%d", &age);
printf("Age = %d", age);
```

■ **The header file provides printf(), scanf(), and other standard I/O functions.**

04 printf() — Formatted Output

printf() sends formatted text and values to the standard output.

printf() can display ordinary text, variable values, and special formatting instructions.

```
int age = 20;
float marks = 85.5;

printf("Age = %d\n", age);
printf("Marks = %.1f", marks);
```

- Text outside %... appears as ordinary text.
- A conversion specification beginning with % tells printf() how to format a value.

■ **Think: printf() = “prepare the output exactly the way I want it to look.”**

05 printf() Syntax

Understand the basic structure before using advanced formatting.

```
printf("format string", argument1, argument2, ...);
```

```
printf("Name: %s\n", name);
printf("Age : %d\n", age);
printf("CGPA: %.2f\n", cgpa);
```

The format string may contain ordinary characters, escape sequences, and conversion specifications.

- The arguments after the format string supply the values that replace the conversion specifications.

■ **The number and types of arguments should match the conversion specifications.**

06 Important Format Specifiers

Choose the specifier according to the data type.

%d / %i	Signed decimal integer	int
%u	Unsigned decimal integer	unsigned int
%f	Floating-point output	double argument to printf
%c	Single character	int value representing a character
%s	String	character array / char *

■ **For printf(), float arguments are promoted to double because of default argument promotions.**

07 Integer Output

Formatting whole numbers with printf()

```
int age = 21;
int marks = 95;

printf("Age = %d\n", age);
printf("Marks = %d", marks);
```

%d → signed decimal integer

- %d is commonly used for int values.
- %i can also be used for signed integer conversion in printf().
- Width and alignment can be added when displaying tables.

■ **Example: printf("%5d", 42); reserves a field of at least 5 character positions.**

08 Floating-Point Output

Control the number of digits shown after the decimal point.

```
double pi = 3.14159265;

printf("%f\n", pi);
printf("%.2f\n", pi);
printf("%.4f\n", pi);
```

%f → default precision
%.2f → 2 digits after decimal
%.4f → 4 digits after decimal

Precision in %f controls the number of digits after the decimal point.

■ **For printf(), %f is used for floating-point output; a float argument is promoted to double.**

09 Character & String Output

Use %c for one character and %s for a string.

```
char grade = 'A';
char name[] = "Girish";

printf("Grade = %c\n", grade);
printf("Name = %s", name);
```

%c	Displays one character	'A'
%s	Displays characters until '\0'	"Hello"

■ **A C string is a sequence of characters terminated by the null character '\0'.**

10 Escape Sequences

Special characters inside a string

\n	New line	printf("Hello\nWorld");
\t	Horizontal tab	printf("A\tB");
****	Backslash	printf("C:\\Temp");
\"	Double quote	printf("\"C\"");
\'	Single quote	printf("'A'");

■ **Escape sequences begin with a backslash and represent characters that are difficult or special to write directly.**

11 Field Width

Control the minimum number of character positions used for output.

```
printf("%5d\n", 42);
printf("%8.2f\n", 12.5);
printf("%10s\n", "C");
```

%5d → minimum width = 5
%8.2f → width = 8, precision = 2

If the value is shorter than the specified width, spaces are added. The width is a minimum, not a maximum.

■ **Useful for producing aligned columns and simple console tables.**

12 Alignment & Flags

Make formatted output easier to read.

```
printf("%5d\n", 42); // right aligned
printf("%-5d\n", 42); // left aligned
printf("%05d\n", 42); // zero padded
```

%5d	Right alignment by default	42
%-5d	Left alignment	42
%05d	Pad with zeros	00042

■ **Flags modify how a conversion is displayed. The exact effect depends on the conversion specification.**

13 scanf() — Formatted Input

scanf() reads formatted data from standard input.

scanf() examines the input according to its format string and stores converted values into variables.

```
int age;
float marks;

scanf("%d", &age);
scanf("%f", &marks);
```

KEY IDEA

scanf() → read → convert → store

- For most ordinary variables, scanf() needs the address of the variable.

■ **That is why &age; and &marks; are written in scanf().**

14 scanf() Syntax & Address Operator

The second part students must understand: &

```
scanf("format", &variable;);
```

```
int n;
scanf("%d", &n);
```

&n; means “the address of n”. scanf() needs an address so that it can place the converted input value into the variable.

scanf("%d", &n;)	Correct for int	stores input into n
scanf("%d", n)	Usually incorrect	n is a value, not an address

■ **Exception: character arrays used as strings are passed without &, e.g. scanf("%s", name).**

15 scanf() Format Specifiers

Input specifiers must match the variable type.

%d	Read int	int n; scanf("%d", &n;);
%f	Read float	float x; scanf("%f", &x;);
%lf	Read double	double x; scanf("%lf", &x;);
%c	Read one character	char ch; scanf(" %c", &ch;);
%s	Read a word/string	char name[30]; scanf("%29s", name);

■ **Notice the important difference: scanf() uses %f for float and %lf for double.**

16 Reading Multiple Values

One scanf() can read multiple values.

```
int age;
float marks;
char grade;

scanf("%d %f %c", &age, &marks, &grade);
```

The format string describes the expected sequence. Whitespace in a scanf() format can match zero or more whitespace characters in the input.

Input: 20 85.5 A

↓

age=20 marks=85.5 grade='A'

■ **The order of conversion specifications should match the order of the arguments.**

17 Reading Strings & Spaces

Why %s does not read a complete sentence

```
char name[30];
scanf("%29s", name);
```

%s reads a sequence of non-whitespace characters and stops at the first whitespace.

Input: Hello World

%s → "Hello"
stops at space

To read a complete line including spaces, fgets() is generally safer and more appropriate.

```
char line[100];
fgets(line, sizeof(line), stdin);
```

■ **Use a field width such as %29s with a 30-character array to reduce the risk of overflowing the array.**

18 Common scanf() Problems

Small mistakes can cause confusing input behavior.

- ■ Using the wrong format specifier for the variable type.
- ■ Forgetting & for ordinary scalar variables.
- ■ Using %s without a field width for a fixed-size character array.
- ■ Expecting %s to read spaces.
- ■ Mixing scanf() and character input without understanding leftover newline characters.

■ For robust programs, check the return value of scanf() to verify that the expected input was successfully converted.

19 Complete Example + Summary

Put formatted input and output together.

```
#include <stdio.h>

int main() {
int age;
float marks;
char grade;

printf("Enter age, marks and grade: ");
scanf("%d %f %c", &age, &marks, &grade);

printf("\nAge : %d\n", age);
printf("Marks : %.2f\n", marks);
printf("Grade : %c\n", grade);
return 0;
}
```

- printf() controls how values are displayed.
- scanf() controls how input is converted and stored.
- Format specifiers must match the expected type.
- Use width and precision to make output readable.

■ **Practice task: Create a student-details program using int, float, char and string input.**

01 Control Statements in C

Controlling the flow of execution in a C program.

SEQUENCE → SELECTION → ITERATION → JUMP

Control statements determine which statements execute, when they execute, and how many times they execute.

■ **A C program does not always execute from top to bottom without decisions or repetition.**

02 Learning Objectives

What students should understand after this lesson

- Explain the purpose of control statements in C.
- Distinguish sequential, selection, iteration, and jump statements.
- Write and trace if, if-else, nested if, else-if, and switch.
- Use while, do-while, and for loops correctly.
- Explain break, continue, goto, and return.

03 What Are Control Statements?

Statements that change the normal flow of execution

```
NORMAL FLOW
Statement 1 → Statement 2 → Statement 3
```

```
CONTROLLED FLOW
Statement 1 → DECISION → Path A / Path B
```

By default, C executes statements sequentially. Control statements allow a program to make decisions, repeat operations, or jump to another part of the program.

■ **Without control statements, useful programs would have very limited decision-making and repetition.**

04 Types of Control Statements

Four major categories

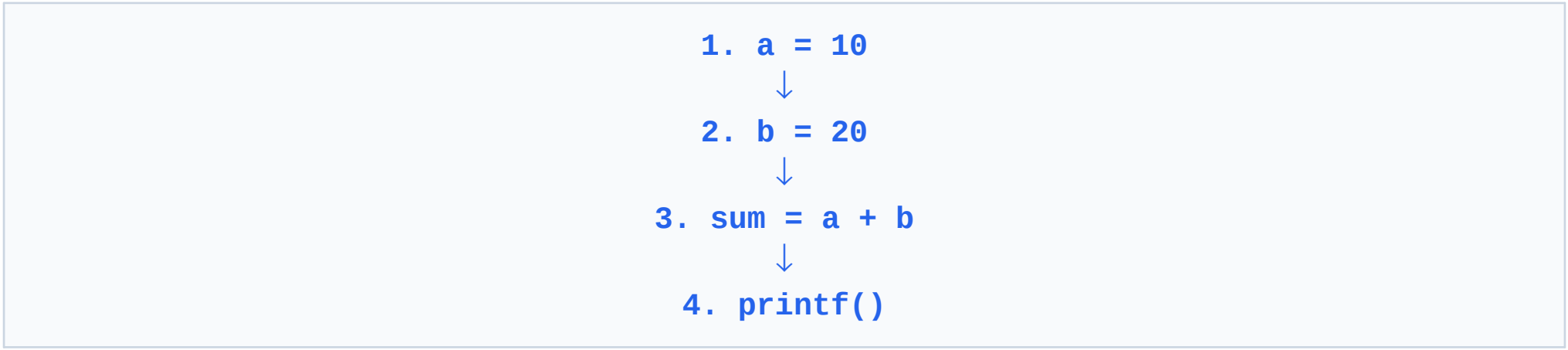
SEQUENCE	Statements execute in the written order.	a → b → c
SELECTION	Choose a path based on a condition.	if / switch
ITERATION	Repeat a block while a condition permits.	while / do-while / for
JUMP	Transfer control to another point.	break / continue / goto / return

05 Sequential Control Flow

The default execution model

If there is no decision, loop, or jump, C normally executes one statement after another.

```
int a = 10;
int b = 20;
int sum = a + b;
printf("%d", sum);
```



06 Conditions in C

A condition produces a true or false decision

Conditions are commonly formed using relational operators such as <, >, <=, >=, == and !=, often combined with logical operators.

```
age >= 18
marks >= 40
x == 0
(a > 10 && b < 20)
```

- In C, zero represents false.
- A non-zero value represents true.

■ **This rule is important when tracing if and loop conditions.**

07 Relational Operators

Building conditions

<	Less than	a < b
>	Greater than	a > b
<=	Less than or equal	a <= b
>=	Greater than or equal	a >= b
==	Equal to	a == b
!=	Not equal to	a != b

08 Logical Operators

Combining multiple conditions

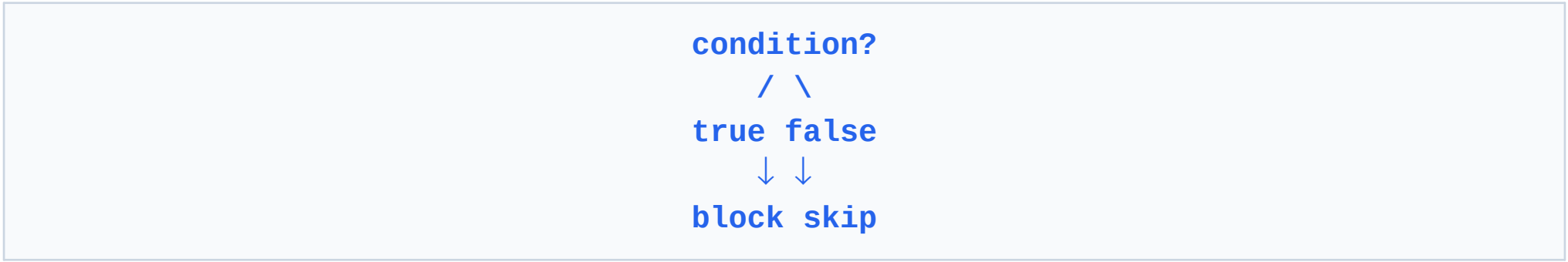
`&&`	Logical AND	Both conditions must be true
`||`	Logical OR	At least one condition must be true
`!`	Logical NOT	Reverses true/false

```
age >= 18 && citizen == 1
marks >= 40 || grace == 1
!(x == 0)
```

■ **Use parentheses to make complex conditions easier to read.**

09 The if Statement

One-way selection




```
if (condition)
{
statement;
}
```

The statements inside the if block execute only when the condition evaluates to true.

■ **If the condition is false, control moves to the statement after the if block.**

10 if Example: Positive Number

A simple decision problem

```
int n;
scanf("%d", &n);

if (n > 0)
printf("Positive");
```

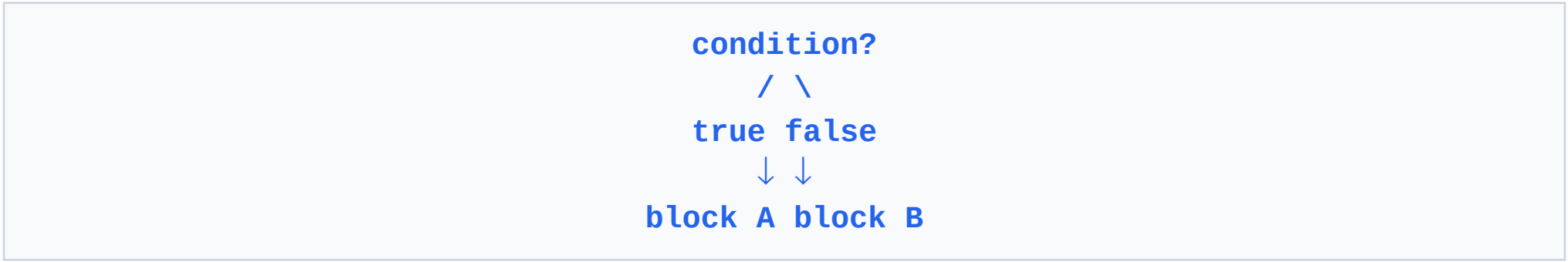
n > 0 ?
YES → print Positive
NO → do nothing

- The condition n > 0 is evaluated first.

■ **Single-statement if blocks may omit braces, but braces are recommended for clarity.**

11 The if-else Statement

Two-way selection




```
if (condition)
{
statement1;
}
else
{
statement2;
}
```

Exactly one of the two blocks executes.

■ **Use if-else when the program has two alternative outcomes.**

12 if-else Example: Even or Odd

Using the remainder operator

```
int n;
scanf("%d", &n);

if (n % 2 == 0)
printf("Even");
else
printf("Odd");
```

The expression n % 2 produces the remainder after division by 2.

n % 2 == 0

YES → Even

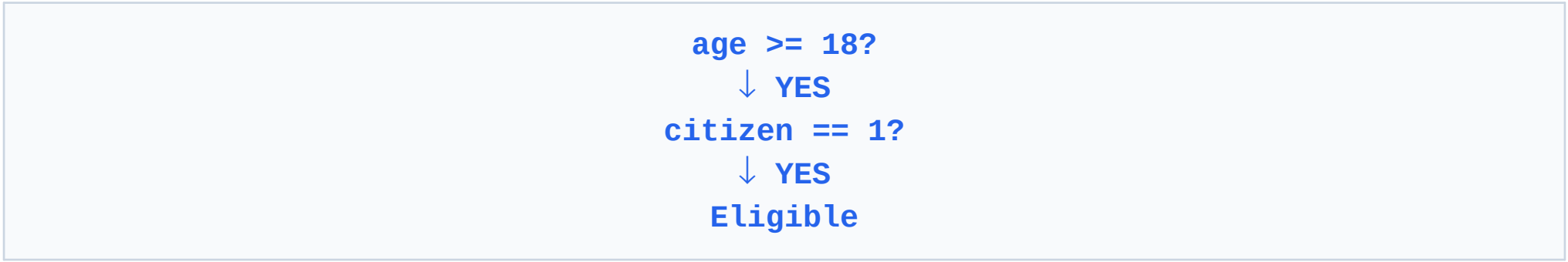
NO → Odd

13 Nested if

An if statement inside another if

```
if (age >= 18)
{
if (citizen == 1)
printf("Eligible");
}
```

The inner condition is evaluated only when the outer condition is true.



■ **Nested decisions are useful when one decision depends on another.**

14 else-if Ladder

Choosing among multiple alternatives

```
if (marks >= 90)
grade = 'A';
else if (marks >= 75)
grade = 'B';
else if (marks >= 60)
grade = 'C';
else
grade = 'D';
```

Conditions are checked from top to bottom. When one becomes true, its block executes and the remaining ladder is skipped.

■ **Order matters. Write ranges in a logical sequence.**

15 else-if Example: Electricity Bill

Multiple ranges in one problem

```
    if (units <= 100)
    bill = units * 2;
    else if (units <= 200)
    bill = 200 + (units-100)*3;
    else
    bill = 500 + (units-200)*5;
```

- First range: 0–100 units.
- Second range: 101–200 units.
- Third range: above 200 units.

■ **Range-based problems are ideal for practicing else-if ladders.**

16 Common if Mistakes

Errors students should recognize

- ■ Using = instead of == in a comparison.
- ■ Forgetting braces when adding multiple statements.
- ■ Writing conditions in the wrong order.
- ■ Creating overlapping or impossible ranges.

```
if (x = 5) // assignment
if (x == 5) // comparison
```

■ **Remember: = assigns a value; == compares two values.**

17 The switch Statement

Multi-way selection using an expression

switch is useful when one expression is compared against several constant case values.

```
switch (choice)
{
case 1: printf("Add"); break;
case 2: printf("Delete"); break;
default: printf("Invalid");
}
```

expression
↓
case 1 / case 2 / case 3 / default

■ **switch is especially common in menu-driven programs.**

18 switch Components

Understand every part

switch	Controls the selection expression.	switch(choice)
case	Labels a possible constant value.	case 1:
break	Exits the switch.	break;
default	Runs when no case matches.	default:

19 switch and break

Why break is important

```
switch (n)
{
case 1:
printf("One");
break;
case 2:
printf("Two");
break;
}
```

When a matching case is found, statements execute from that point. break transfers control out of the switch.

■ **Without break, execution can continue into the next case. This is called fall-through.**

20 switch Fall-Through

An important behavior to understand

```
switch (n)
{
case 1:
printf("A");
case 2:
printf("B");
}
```

If n is 1, the program prints A and then continues into case 2 because there is no break after case 1.

case 1 → A → case 2 → B

■ **Fall-through can be intentional, but accidental fall-through is a common error.**

21 if-else vs switch

Choosing the appropriate selection statement

if-else	Best for ranges and complex logical conditions.	marks >= 90
switch	Useful for discrete constant choices.	choice = 1, 2, 3

switch is not a general replacement for if-else. It is designed around matching a controlling expression against case labels.

■ **For ranges such as marks >= 90, an if-else ladder is usually more natural.**

22 Introduction to Loops

Iteration repeats a block of statements

```
condition → body → update → condition → ...
```

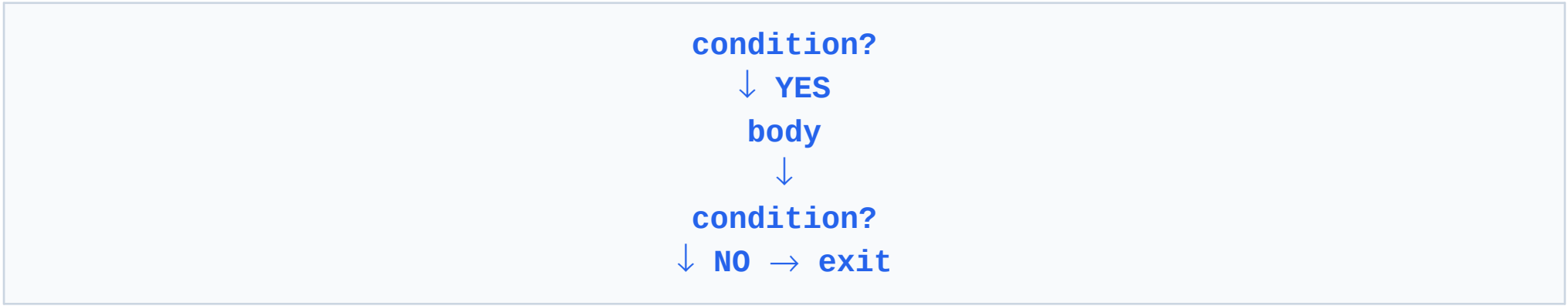
A loop is useful when the same task must be performed repeatedly, such as printing numbers, processing array elements, or reading multiple records.

- A loop normally has a condition controlling whether another iteration occurs.

■ **Three common loops in C: while, do-while, and for.**

23 while Loop

Entry-controlled repetition




```
int i = 1;
while (i <= 5)
{
printf("%d ", i);
i++;
}
```

The condition is checked before each iteration. If it is false initially, the body may execute zero times.

24 while Loop: Trace It

Learn to dry-run a loop

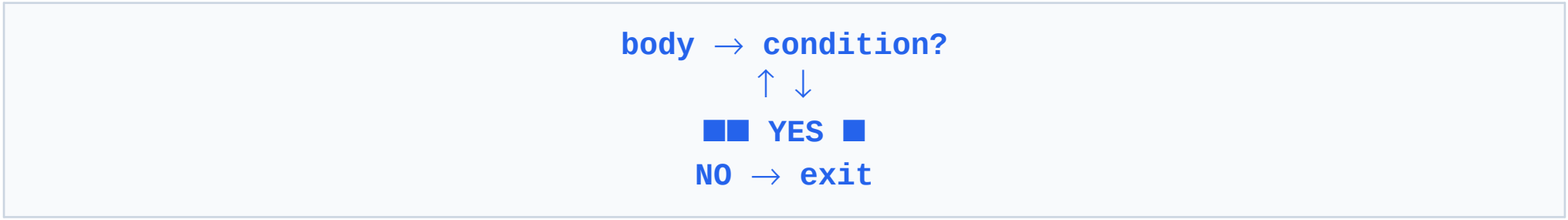
```
int i = 1;
while (i <= 3)
{
printf("%d ", i);
i++;
}
```

i=1 → print 1 → i=2
→ print 2 → i=3
→ print 3 → i=4
→ stop

■ **Trace the variable before checking the condition and after updating it.**

25 do-while Loop

Exit-controlled repetition




```
int i = 1;
do
{
printf("%d ", i);
i++;
} while (i <= 5);
```

The body executes first, so a do-while loop always executes at least once.

■ **Notice the semicolon after while(condition);**

26 while vs do-while

The key difference is when the condition is checked

while	Condition checked before body.	May execute 0 times
do-while	Condition checked after body.	Executes at least 1 time

```
while: condition → body

do-while: body → condition
```

■ **For menu prompts where the user must see the menu at least once, do-while can be useful.**

27 for Loop

Compact loop for initialization, condition, and update

```
for (initialization; condition; update)
          ↓ ↓ ↓
      START TEST CHANGE
```

```
for (int i = 1; i <= 5; i++)
{
printf("%d ", i);
}
```

The for loop places the three main control parts in one line.

■ **Typical pattern: initialize → test → execute → update → test again.**

28 for Loop Anatomy

Understand exactly when each part runs

```
for (i = 1; i <= 5; i++)
printf("%d ", i);
```

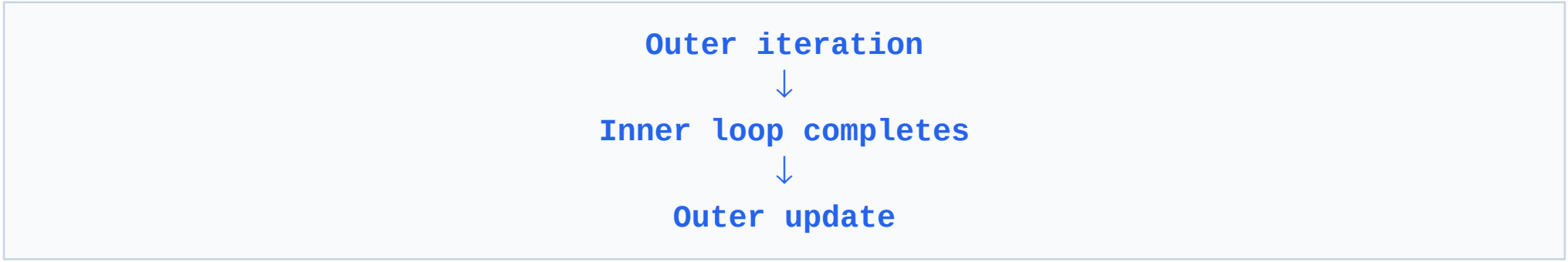
1. i = 1
↓
2. i <= 5 ?
↓ YES
3. body
↓
4. i++
↓
5. condition again

■ **Initialization happens once. The condition is tested repeatedly. The update runs after each body execution.**

29 Nested Loops

A loop inside another loop

```
for (int i = 1; i <= 3; i++)
{
for (int j = 1; j <= 3; j++)
printf("* ");
printf("\n");
}
```



For each outer-loop iteration, the inner loop completes all its iterations.

■ **Nested loops are widely used for patterns, matrices, and two-dimensional data.**

30 break Statement

Immediately exits the nearest loop or switch

```
for (int i = 1; i <= 10; i++)
{
if (i == 5)
break;
printf("%d ", i);
}
```

1 2 3 4 → i==5 → BREAK → exit

break terminates the nearest enclosing loop or switch and transfers control to the statement after it.

■ **Use break when continuing the remaining iterations is unnecessary.**

31 continue Statement

Skip the current loop iteration

```
for (int i = 1; i <= 5; i++)
{
if (i == 3)
continue;
printf("%d ", i);
}
```

1 → print
2 → print
3 → skip
4 → print
5 → print

continue skips the remaining statements in the current iteration and proceeds to the next iteration.

■ **Output: 1 2 4 5**

32 goto and return

Other jump-style control statements

```
    goto label;

    label:
    printf("Done");
```

goto transfers control to a labeled statement within the same function. It should be used carefully because excessive use can make program flow difficult to follow.

```
    return 0;
```

return exits the current function and can send a value back to the caller.

■ **Structured if and loops are usually easier to understand than extensive goto usage.**

33 Putting It All Together

Choose the control statement based on the problem

Decision	Choose among alternatives.	if / else-if / switch
Repetition	Repeat a block.	while / do-while / for
Early exit	Stop a loop or switch.	break
Skip iteration	Move to next iteration.	continue
Function exit	Leave a function.	return

■ **Ask: Is the task choosing, repeating, skipping, or exiting?**

34 Practice Problems

Apply the concepts

- 1. Read a number and display whether it is positive, negative, or zero.
- 2. Read marks and display a grade using an else-if ladder.
- 3. Create a calculator menu using switch.
- 4. Print numbers from 1 to N using a for loop.
- 5. Find the sum of first N natural numbers using while.
- 6. Print a multiplication table using a loop.
- 7. Print a star pattern using nested loops.

■ **Challenge: combine selection, iteration, and break in one program.**

35 Common Mistakes + Summary

Final checklist

- ■ Use == for comparison, not =.
- ■ Update loop variables to avoid unintended infinite loops.
- ■ Remember break in switch when fall-through is not intended.
- ■ Remember the semicolon after do-while.
- ■ Check loop boundaries carefully: < versus <=.

```
SELECTION → if / else-if / switch
ITERATION → while / do-while / for
JUMP → break / continue / goto / return
```

■ **Final rule: trace the condition → identify the path → execute → update → repeat if required.**

01 Jump Statements in C

break • continue • goto

CHANGE THE NORMAL FLOW OF EXECUTION

Jump statements transfer control from one point of a program to another instead of simply continuing with the next sequential statement.

■ **In this lesson we study three important jump statements: break, continue and goto.**

02 Learning Objectives

What you should be able to do

- Explain the purpose of jump statements in C.
- Use break to terminate loops and switch statements.
- Use continue to skip selected loop iterations.
- Use goto with labels and understand its restrictions.
- Trace programs containing jump statements.
- Select the most suitable statement for a programming problem.

03 What Are Jump Statements?

Changing the normal sequence of execution

```
NORMAL FLOW
Statement 1 → Statement 2 → Statement 3
```

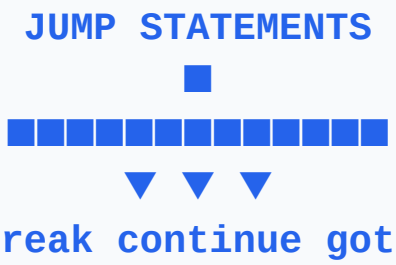
```
JUMP FLOW
Statement 1 → JUMP → another point
```

C normally executes statements sequentially. A jump statement changes this flow by transferring control.

■ **Jump statements are useful when normal sequential or repetitive execution must be interrupted or redirected.**

04 Types of Jump Statements

Three statements covered in this lesson



break and continue are mainly associated with loops and switch. goto transfers control to a label within the same function.

■ **Remember: break = exit, continue = skip, goto = jump to label.**

05 break — Introduction

Terminate the nearest loop or switch

The break statement immediately terminates the nearest enclosing for, while, do-while, or switch statement.

```
for (int i = 1; i <= 10; i++)
{
if (i == 5)
break;
printf("%d ", i);
}
```

1 → 2 → 3 → 4 → BREAK → EXIT

■ **Output: 1 2 3 4**

06 break — Syntax

Simple form

```
if (condition)
    break;
```

break is commonly placed inside an if statement so that the loop is terminated only when a particular condition becomes true.

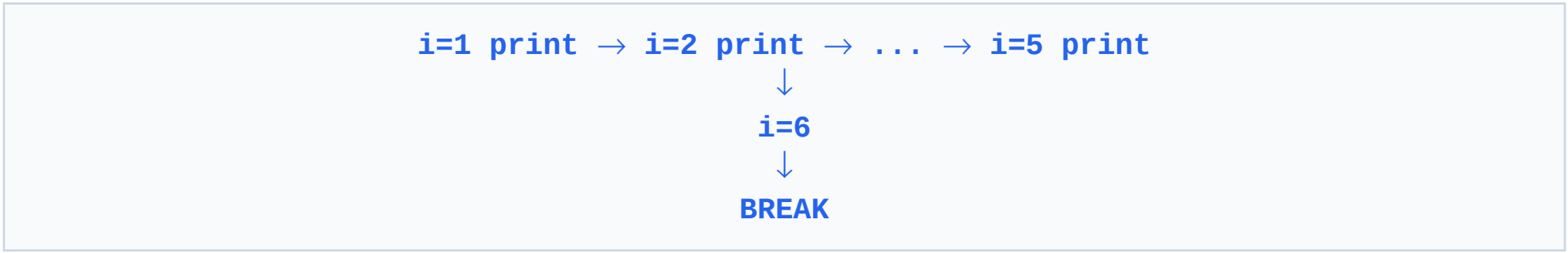
```
while (condition)
{
if (exit_condition)
break;
statements;
}
```

■ **break does not terminate the complete program; it exits only the nearest loop or switch.**

07 break with for Loop

Stop a counting loop early

```
for (int i = 1; i <= 10; i++)
{
if (i == 6)
break;
printf("%d ", i);
}
```



The loop condition permits values up to 10, but break stops execution when i becomes 6.

■ **Output: 1 2 3 4 5**

08 Tracing break

Dry-run example

```
int i;
for (i = 1; i <= 5; i++)
{
if (i == 3)
break;
printf("%d ", i);
}
```

i=1 → print 1
i=2 → print 2
i=3 → condition TRUE → break

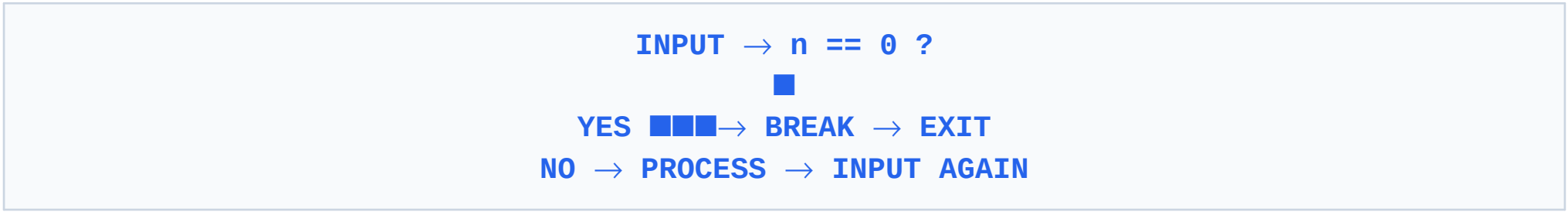
At i = 3, the condition becomes true. The printf statement is not executed and the loop ends.

■ **Output: 1 2**

09 break with while Loop

Controlled exit from an indefinite loop

```
int n;
while (1)
{
scanf("%d", &n);
if (n == 0)
break;
printf("You entered %d\n", n);
}
```



while(1) creates an always-true loop. break provides the controlled exit when the sentinel value 0 is entered.

10 break with switch

break prevents unwanted fall-through

```
switch (choice)
{
case 1:
printf("Add");
break;
case 2:
printf("Delete");
break;
default:
printf("Invalid");
}
```

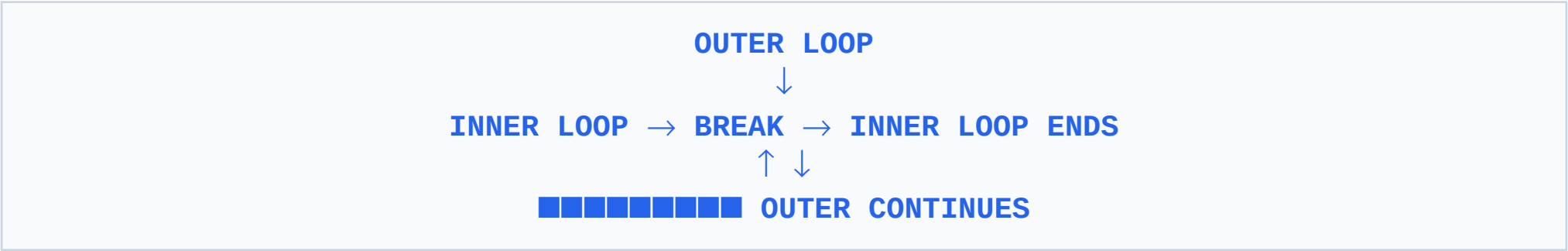
In switch, break transfers control to the statement after the switch block.

■ **Without break, execution can continue into the next case.**

11 break in Nested Loops

Which loop does break terminate?

```
for (int i = 1; i <= 3; i++)
{
for (int j = 1; j <= 3; j++)
{
if (j == 2)
break;
printf("%d%d ", i, j);
}
}
```



A break statement exits only the nearest enclosing loop. Here, the inner loop ends but the outer loop continues.

12 Important Points about break

Rules to remember

- break can be used with for, while, do-while and switch.
- It exits only the nearest enclosing loop or switch.
- It is often used with an if condition.
- It does not terminate the entire program.
- In nested loops, one break does not automatically exit every loop.

■ **Memory trick: BREAK means STOP the current loop/switch.**

13 continue — Introduction

Skip the current iteration

continue skips the remaining statements of the current loop iteration and moves toward the next iteration.

```
for (int i = 1; i <= 5; i++)
{
if (i == 3)
continue;
printf("%d ", i);
}
```

1 print → 2 print → 3 SKIP → 4 print → 5 print

■ **Output: 1 2 4 5**

14 continue — Syntax

Skip selected iterations

```
if (condition)
   continue;

remaining statements;

next iteration
```

When the condition is true, continue skips the statements that follow it in the current iteration.

```
for (...)
{
if (skip_condition)
continue;
process();
}
```

■ **continue means “skip this turn”, not “stop the loop”.**

15 continue with for Loop

Print only selected values

```
for (int i = 1; i <= 10; i++)
{
if (i % 2 == 0)
continue;
printf("%d ", i);
}
```

```
1 PRINT
2 SKIP
3 PRINT
4 SKIP
5 PRINT
6 SKIP
...
```

Even numbers satisfy i % 2 == 0, so those iterations skip printf().

■ **Output: 1 3 5 7 9**

16 Tracing continue

Understand exactly what is skipped

```
for (int i = 1; i <= 5; i++)
{
if (i == 3)
continue;
printf("%d ", i);
}
```

i=1 → print
i=2 → print
i=3 → CONTINUE → skip printf()
i=4 → print
i=5 → print

■ **Output: 1 2 4 5**

17 continue with while Loop

Important difference from for

```
int i = 0;
while (i < 5)
{
i++;
if (i == 3)
continue;
printf("%d ", i);
}
```

In a while loop, continue transfers control to the condition check. Therefore the loop-control variable must be updated correctly.

■ **Be careful: a misplaced continue can create an infinite loop.**

18 break vs continue

The most important comparison

```
break → LOOP ENDS
continue → CURRENT ITERATION ENDS
```

break completely terminates the nearest loop. continue leaves the current iteration but allows the loop to continue.

■ **Memory trick: break = STOP; continue = SKIP.**

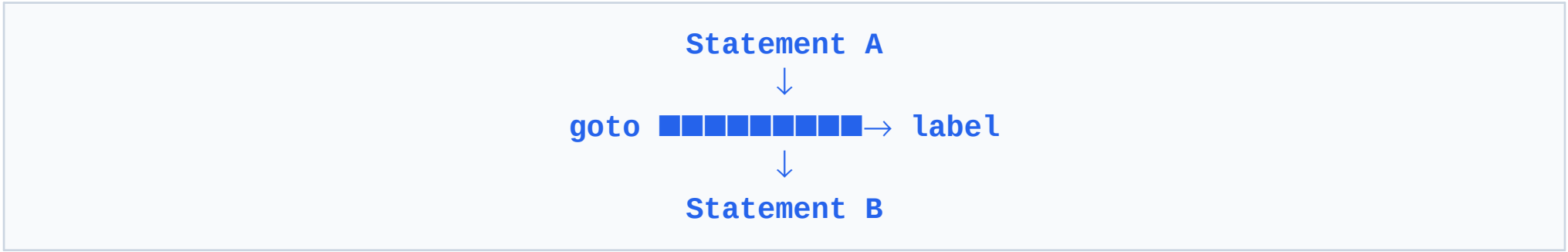
19 goto — Introduction

Unconditional jump to a label

goto transfers program control directly to a labeled statement within the same function.

```
    goto label;

    label:
    statement;
```



■ **Unlike break and continue, goto can jump to a named location.**

20 goto — Syntax

Two components: goto and label

```
goto label;

label:
statement;
```

A label is an identifier followed by a colon. The goto statement specifies the destination label.

```
    printf("Start\n");
    goto message;
    printf("Skipped\n");

    message:
    printf("Done\n");
```

■ **The label and goto must belong to the same function.**

21 Simple goto Example

Observe the changed execution order

```
#include <stdio.h>

int main()
{
printf("A\n");
goto skip;
printf("B\n");

skip:
printf("C\n");
return 0;
}
```

A → goto skip → C

B is skipped

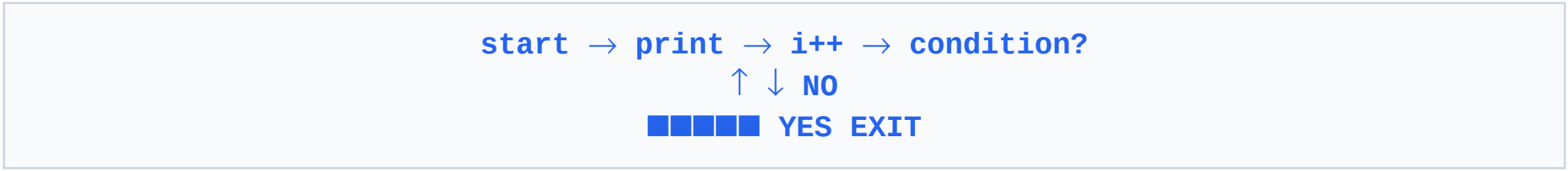
■ **Output: A C**

22 goto with Conditional Logic

Using goto for repeated execution

```
int i = 1;

start:
printf("%d ", i);
i++;
if (i <= 5)
goto start;
```



The program jumps back to start while the condition remains true.

■ **This demonstrates goto, but a loop is usually clearer for repetition.**

23 goto and Labels

Rules for using labels

- A label is followed by a colon: label:
- goto label; transfers control to that label.
- The destination label must be inside the same function.
- Labels can appear before a statement.
- Too many jumps can make the program difficult to understand.

■ **Choose meaningful labels when goto is genuinely necessary.**

24 Why goto Is Discouraged

Readability matters

- Frequent jumps make program flow difficult to trace.
- Many labels can produce confusing “spaghetti code”.
- Loops and conditions usually express structured logic more clearly.
- Debugging becomes harder when control jumps unpredictably.

■ **Understanding goto is important; using it everywhere is not.**

25 When Can goto Be Useful?

A balanced view

- Some low-level or systems programs use goto for centralized error handling.
- It can sometimes simplify cleanup paths involving multiple resources.
- Its use should be limited and carefully structured.

For beginner programs, prefer for, while, do-while, if, switch, break and continue whenever they express the logic clearly.

26 break vs continue vs goto

Compare all three

```
break → EXIT nearest loop/switch
continue → NEXT loop iteration
goto → NAMED LABEL
```

Each statement changes control flow differently. Selecting the right one makes the program easier to read and maintain.

■ **Use the simplest control statement that clearly expresses your intention.**

27 Common Student Mistakes

Check these before executing

- ■ Thinking continue terminates the loop.
- ■ Thinking break skips only one iteration.
- ■ Forgetting that break exits only the nearest loop.
- ■ Creating an infinite while loop after continue.
- ■ Forgetting break in switch when fall-through is not intended.
- ■ Using goto when a simple loop would be clearer.

28 Output Prediction — 1

Practice dry-run

```
for (int i = 1; i <= 6; i++)
{
if (i == 3)
continue;
if (i == 5)
break;
printf("%d ", i);
}
```

1 → PRINT
2 → PRINT
3 → CONTINUE
4 → PRINT
5 → BREAK

■ **Predict the output before running the program: 1 2 4**

29 Output Prediction — 2

Think carefully about break

```
int i = 1;
while (i <= 6)
{
if (i == 4)
break;
printf("%d ", i);
i++;
}
```

The condition is checked before each iteration. When i becomes 4, break executes before printf().

■ **Predict the output: 1 2 3**

30 Practice Programs

Apply the three jump statements

- 1. Read integers until 0 is entered; use break to stop.
- 2. Print 1 to 50 but skip multiples of 5 using continue.
- 3. Print numbers until 35 and terminate using break.
- 4. Create a menu-driven program that exits using break.
- 5. Write a simple goto program that jumps over one statement.
- 6. Rewrite a goto-based repetition using while.

■ **Challenge: explain why the loop version is easier to read.**

31 Final Summary

Key points to remember

```
break → STOP
continue → SKIP
goto → JUMP
```

break terminates the nearest loop or switch. continue skips the current loop iteration. goto transfers control to a named label within the same function.

■ **Good programming practice: prefer clear, structured control flow and use goto sparingly.**